

O'REILLY®

Training Data for Machine Learning

Human Supervision from Annotation
to Data Science

**Early
Release**

Raw & Unedited

Compliments of

Kili



Anthony Sarkis

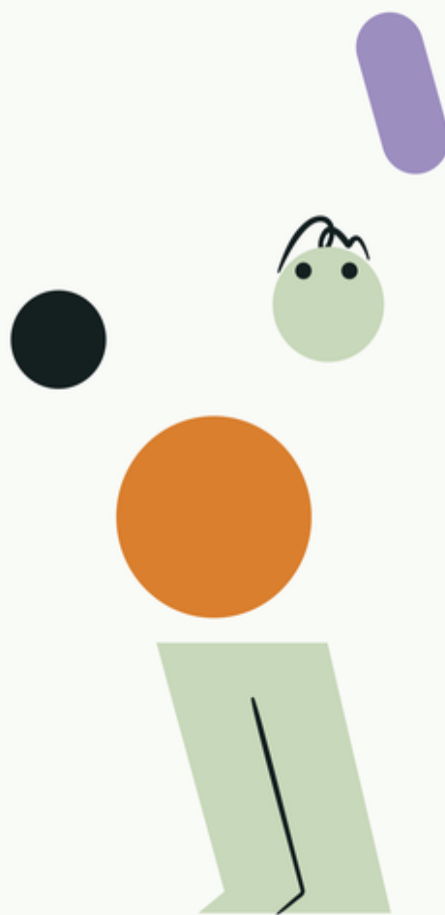
Kili Technology

AI has a lot to learn

to be trustworthy

to be accurate

to go faster to production



Kili

DATA TRAINING PLATFORM FOR ENTERPRISE AI
kili-technology.com

Training Data for Machine Learning

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

Anthony Sarkis

Training Data for Machine Learning

by Anthony Sarkis

Copyright © 2021 Anthony Sarkis. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North,
Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: Rebecca Novack

Development Editor: Jill Leonard

Production Editor: Elizabeth Faerm

Copyeditor: TK

Proofreader: TK

Indexer: TK

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: Kate Dullea

October 2022: First Edition

Revision History for the Early Release

- 2021-05-21: First release
- 2021-10-13: Second release
- 2022-02-23: Third release

See <http://oreilly.com/catalog/errata.csp?isbn=9781492094524> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. Training Data for Machine Learning Models, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author, and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

This work is part of a collaboration between O'Reilly and Kili Technology. See our [statement of editorial independence](#).

978-1-492-09445-6

[LSI]

Chapter 1. Training Data

Introduction

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 1st chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at jleonard@oreilly.com.

What is Training Data?

Training Data is the control of a Supervised System.

Training Data controls the system by defining the ground truth goals for the creation of Machine Learning models. This involves technical representations, people decisions, processes, tooling, system design, and a variety of new concepts specific to Training Data. In a sense, a Training Data mindset is a paradigm upon which a growing list of theories, research and standards are emerging.

Model

A Machine Learning (ML) Model that is created as the end result of a ML Training Process.



Figure 1-1. Diagram or screenshot of common supervision interface.

Training Data is not an algorithm, nor is it tied to a specific machine learning approach. Rather it's the definition of what we want to achieve. A fundamental challenge is effectively identifying and mapping the desired human meaning into a machine readable form.

The effectiveness of training data depends primarily on how well it relates to the human defined meaning and how reasonably it represents real model usage. Practically, choices around Training Data have a huge impact on the ability to train a model effectively.

Training Data makes sense when a set of conditions are true. For example, training data for a parking lot detection system may have very different views. If we create a training data set based on a top down view (see left side of Figure 1.2) and then attempt to use the image on the right we will get unexpected results. That's why it's important that the data we use to train a system closely matches the data our trained system would see in production.

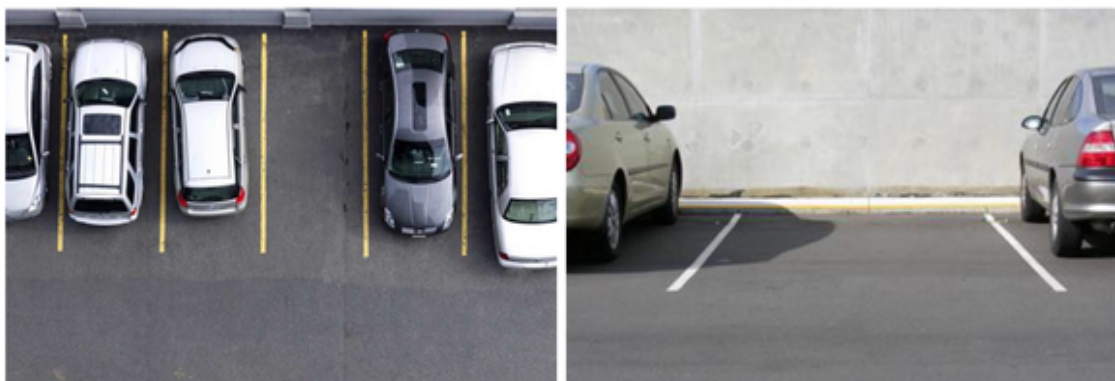


Figure 1-2. If the left is your training data and right is your use case, you are in trouble!

See figure 1-2—A machine learning system trained only on images from a top-down view as in the left image has a hard time running in an environment where the images are from a front view front as in the right image. Our system would not understand the concept of a car and parking lot from a front view if it has never seen such an image during training.

Good Robot, Bad Robot

Training a machine to understand and intelligently interpret the world may feel like a monumental task. But there's good news! The algorithms behind the scenes do a lot of the heavy lifting. Our primary concern with training data can be summed up as defining what's good, what should be ignored, and what's bad.

Of course real Training Data requires a lot more than a head nod or head shake. We must find a way to transform our rather ambiguous human terminologies into something the machine can understand.

Thinking of Training Data as Code

One way to think of Training Data is as a higher level programming language. In the same way we use languages like Python to be more expressive than Assembly code¹, we can use Training Data to be more expressive than Python. Instead of trying to define a Strawberry as hundreds of lines of if statements, and manually identifying key features,

we can simply draw and label the Strawberry as such. Similar to how we just tell a child “that’s a strawberry”.

This is distinct from using an application, such as a Low Code system, because it is directly programming meaning to raw data. It’s real programming, just at a higher level of abstraction. Like code, if the Training Data is changed, then the results of the system are changed. From a data science perspective, training data is integral to the machine learning process because it’s the input for Training and the ground truth upon which to measure results.

As a brief comparison, traditional software programs often have Business Logic, or logic that only makes sense in the business context exterior to the program. Similarly training data relates to a context exterior to itself, often the real world. While business logic is inferred, Training Data is literally directly mapped to that exterior context of raw data and related assumptions. It’s human meaning encoded in a form ready for consumption by a machine learning algorithm. Training Data is Code.

Concepts Introduction

There are two general categories of training data: Classic and Supervised. The general focus of this book is on Supervised. We will contrast Supervised to Classic later in more detail. The following presentation of concepts is intended to be introductory to provide a baseline understanding around definitions and assumptions. These themes will be explored in greater detail throughout the book.

Representations

Let’s imagine we are working on a Strawberry picking system. We need to know what a strawberry is, where it is, and how ripe it is. Let’s introduce a few new terms to help us work more efficiently:

Label

A Label, also called a class² Other names include: Label template, represents the highest level of meaning. For example, a label can be “Strawberry” or “Leaf”. For those technical folks, you can think of it as a Table in a database. This is usually attached³ to a specific Instance.

Instance

A single example. It is connected to a label to define what it is. And usually contains positional or spatial information defining where it is. Continuing the technical example, this is like a row in a database. An Instance may have many Attributes.

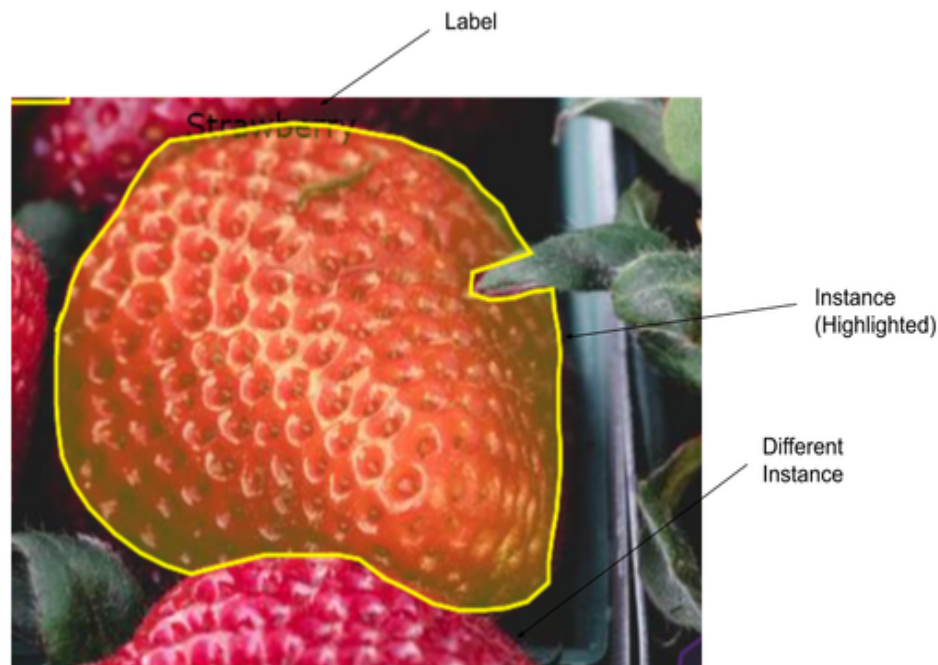
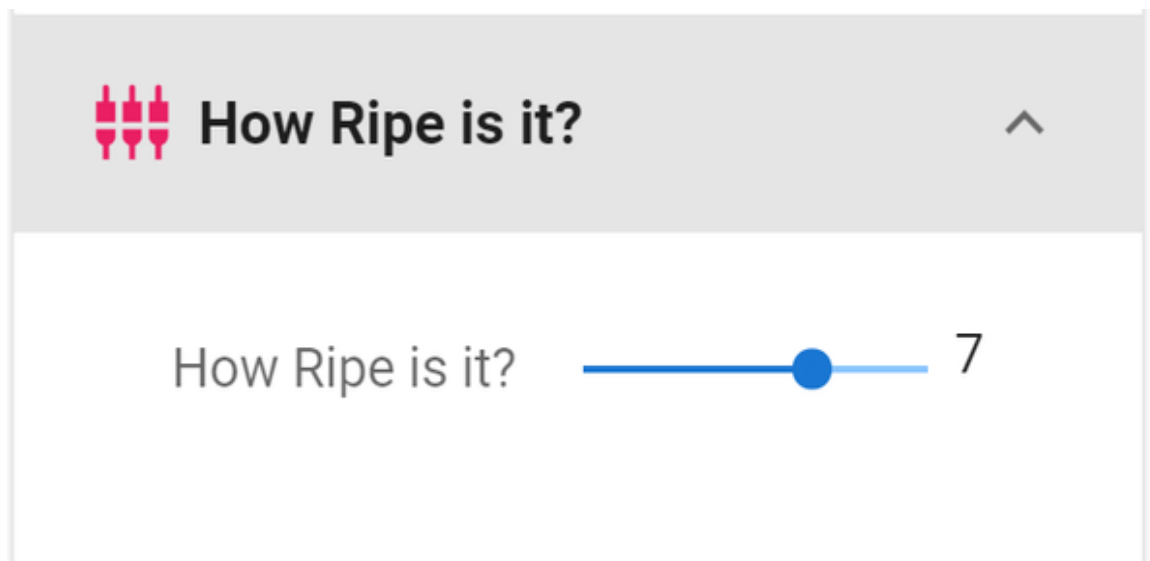


Figure 1-3. Diagram showing labeled and not labeled instances.

Attributes

Attributes are things unique to a specific Instance. Imagine you only want the system to pick Strawberries of a certain ripeness. You may represent the ripeness as a slider or you may also have a multiple choice question answer about ripeness. From the database perspective this is somewhat akin to a column. This choice will affect the speed of supervision. A single instance may have many unique attributes, for example in addition to ripeness there may be disease identification, produce quality grading, etc.

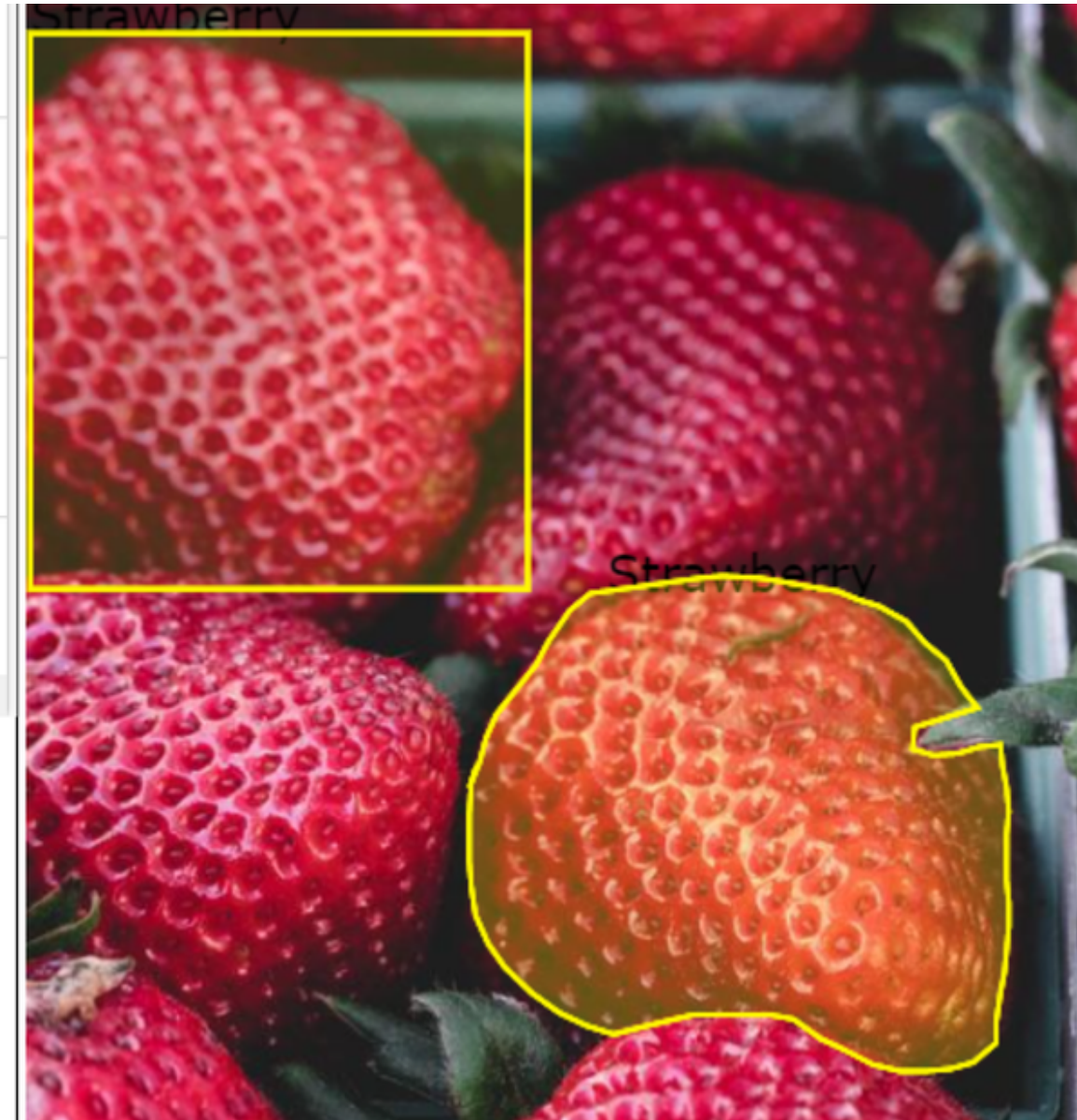


Choices

Your choices here are as much part of Training Data as doing the literal supervision. You also make choices on what type of **Spatial** representation to use.

Spatial

We can represent the Strawberries as boxes, polygons, and other options. In later chapters I will discuss these trade-offs.



There are other choices which will be detailed in later chapters. From a system design perspective there can be choices around what type of raw data to use, such as image, audio, text, or video. And sometimes even combining multiple modalities together. Angle, size, and other attributes may also apply here. Unlike attributes generally, spatial locations are singular in nature. An object is usually only in one place at a given moment in time.⁴

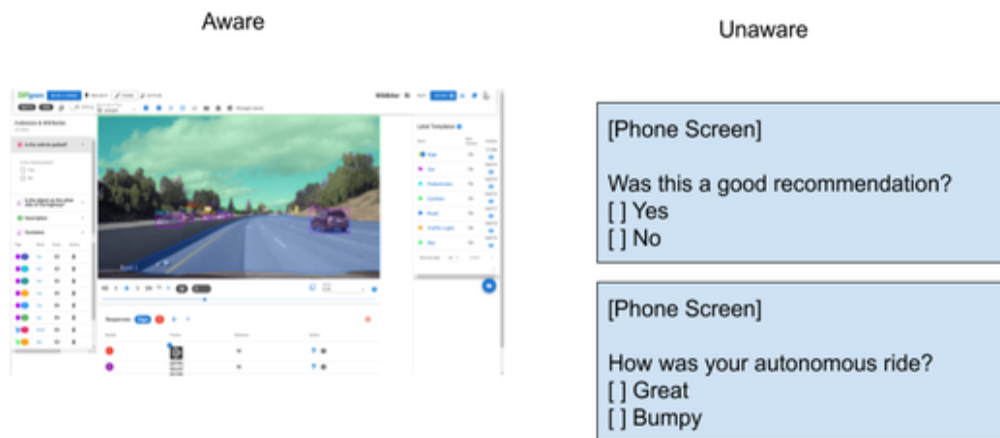
Who Supervises the Data

For our strawberry system, who supervises the data may matter depending on the context. For example, if the system is to be installed in a grocery store, perhaps a grocery store employee is best able to identify if the strawberries are “OK” to sell or not. Whereas an automated farm installation may require greater precision.

This leads us to considerations around who supervises the data. Their backgrounds, biases, incentives and more. What incentives are the people who supervise the data given? How much time is being spent per sample? When is the person doing it? Are they doing 100 samples at once?

As a supervisor, you may work for a firm that specializes in data supervision, or you may be hired to one company, or you may be a subject matter expert. Finally, the supervision may come directly from an end user, who is potentially unaware they are even doing supervision.

Generally if an end user is doing the supervision the volume and depth of supervision will be lower. However it may be more timely, and more specific. Consider that both can be used together. For example an end user suggesting something was “bad” may be used as a flag to initiate further direct supervision.



This is all in the context of explicit (or direct) supervision of the data. Someone is directly viewing the data and taking action. This contrasts with

classic training data where the data is implicitly observed “in the wild” and not editable by humans.

We will cover this in more detail in Chapter 3.

Sets of Assumptions

Imagine you are an Olympic runner training to run in the set of conditions that are expected to be present at the Olympics. It’s likely if you are training for a specific event, say the 100 Meter, then you will train only for the 100 Meter, and not for the 400, 800, or High Jump. Because while similar, those events are outside of the scope of what you expect to encounter—the 100 Meter.

Training Data is very similar. We define a set of assumptions and expect the Training Data to be useful in that context - and only in that context. Similar to the above we can start with high level assumptions. Our strawberry picker is assumed to be on a commercial strawberry field. Then, like the 100M specificity, we can get into more specific assumptions. For example perhaps we assume the camera will be mounted to a drone, or a ground based robot, or the system will be used during the day.

Assumptions are in any system, however, they take on a different context with Training Data because of the inherent randomness involved. Somewhat surprisingly, human analogies around Training (for sports, work etc.) are actually very applicable to Training Data.

Randomness

Let’s zoom in on this human centric example of training for the Olympics. I can train how to do something for all my life, such as beating an Olympic record - and still not be 100% certain that I will be able to do it. In fact for many things, I can probably only be certain I won’t be able to do it. The intuition that I am not guaranteed to be an olympian is clear. Getting a similar intuition around AI training is part of the challenge.

This is especially hard because we typically think of computer systems as being deterministic, meaning if I run the same operation twice I will get the same result. The way AI models get trained is not deterministic. The world in which AIs operate is not deterministic. The processes around creation of training data involve humans and are not deterministic. Therefore at the heart of training data is an inherent randomness. Much of the work with training data, especially around the abstractions, is defining what is and is not possible in the system. Essentially trying to reign in and control the randomness into more reasonable bounds.

We create training data to cover the expected cases. What we expect to happen. And as we will cover in more depth later, we primarily use rapid retraining of the data to handle the expected randomness of the world.

Processes and Process Automation

Defining a process is one of the most fundamental ways to set up guardrails around randomness. Even the most basic supervision programs require some form of process. This defines where the data is, who is responsible for what, the status of tasks, etc. Quality assurance in this context is a mini artform with many competing approaches and opinions. We will discuss multiple levels of sophistication here, going from manual to fully automated, self healing, multi-stage pipelines.

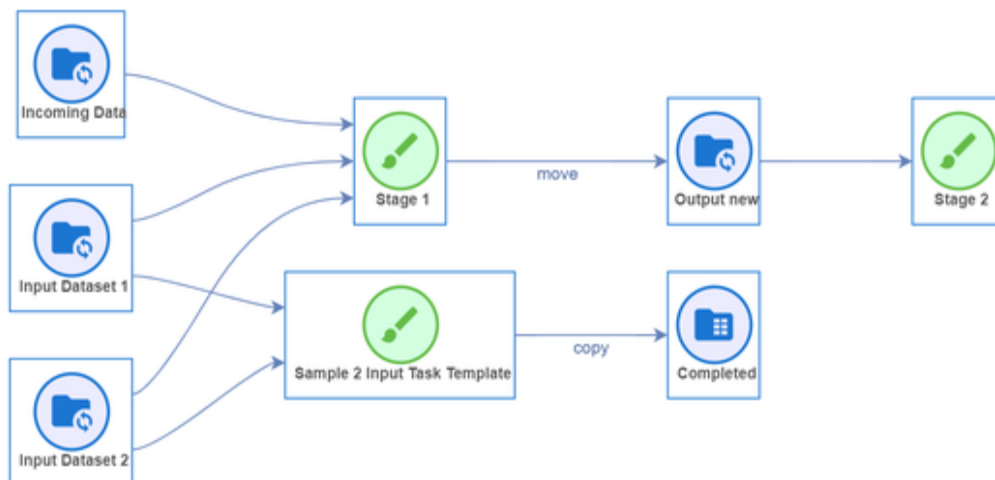


Figure I-4. Process visual

Supervision Automation and Tooling

As soon as you understand the basics of training data you will quickly realize there is an obvious bottleneck: Doing the literal annotation. There have been many approaches to speed up this core part of the process. We will explore the processes and trade-offs. Choices here can quickly become very complex. It's one of the most often misunderstood parts but is an important part of Training Data.

Dataset Construction & Maintenance

When creating a model, a common practice is to split an original set into three subsets, it's called Train/Val/Test. The idea is to have a set that's trained on, a second set that's withheld from training to Validate on, and a 3rd set that's held in reserve until work is complete, to do a single use final test on.

But where did the "Original" set come from?! This is the concern of the Training Data context, constructing the Original set.

Usually there is more raw data then can be annotated. So part of the basic selection process is choosing which of the raw samples to be annotated. There are normally multiple Original sets. In fact many projects quickly grow to have hundreds of these Original sets. More generally, this is the concern of overall organization and structure of the Datasets, including selecting which samples are to be included in which sets.

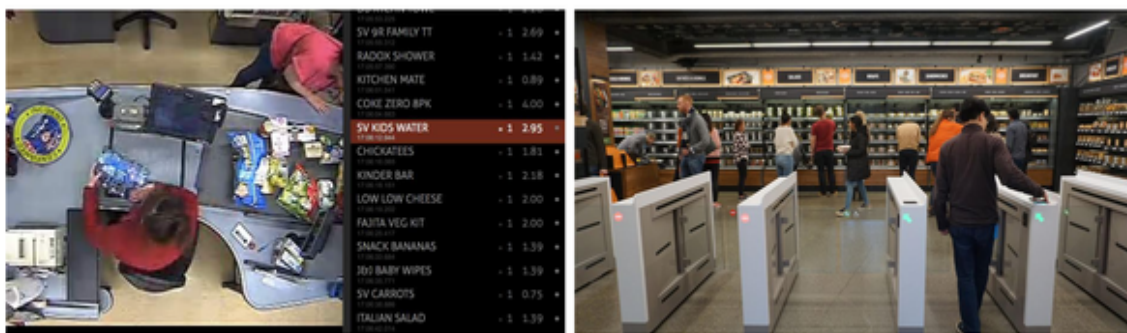
Relevancy

Continuing the theme of validation of existing data, how do we know if our data is actually relevant to the real world? For example, we could get a perfect 100% on a test set, but if the test set isn't relevant to the real world data then we are in trouble! There is currently no known direct test that can define if the data is relevant to the production data - only time will truly tell. This is similar to the way a traditional system can be loaded tested for x number of users, but the true usage pattern will always be different. Of

course we can do our best to plan for the expected use and design the system.

Integrated System Design

There are usually many choices of how to design the data collection system in supervised learning. For example, for a grocery store, a system could be positioned over an existing checkout register, such as to prevent theft, or aid in faster checkout. Or, a system could be designed to replace the checkout entirely, such as placing many cameras throughout and tracking shopper actions. There is no right or wrong answer here - the primary thing to be aware of is that the Training Data is tied to the context it's created in. In later sections we will cover system design here in much more depth.



Left, approach example of "Integration"; Right, example of "New Process"

Figure 1-5. Goal of showing left an existing check out process, vs on right a novel "checkout free" process

Raw Data Collection

Beyond the high level system design perspective, the collection and storage of raw data is generally beyond the scope of this book. This is part because of the vast array of options for raw data. Raw data can be real life sensors, it can be screenshots of webpages, pdf scans, etc. Virtually anything that can be represented as an image, video, or text can be used for raw data.

What-To-Label

As part of Dataset Construction we know we need to create a smaller set from a larger set of raw data - but how? This is the concern of What-To-

Label. Generally, the goal of these approaches is to find “interesting” data. For example if I have thousands of images that are similar, but 3 are obviously different, I may get the most bang for my buck if I start with the 3 most different ones.

Iterations

In traditional programming we iterate on the design of functions, features, and systems. In Training Data there is a similar form of iteration on all of the concepts we are here in discussing. The models themselves are also iterative, for example they may be retrained on a predetermined frequency, such as daily. Two of the biggest competing approaches here are the “Fire and forget” and the “Continual Retrain”. In some cases it may be impractical to retrain, and so a single, final model is created.

Transfer Learning

The idea of transfer learning is to start off from an existing knowledge base before training a new model.

Transfer learning is used to dramatically speed up training new models. From a Training Data view, transfer learning introduces challenges around bias. Because we are indirectly using training data from that prior model training. If there was undesirable bias in that model, it may carry over to our new case. Essentially it creates a dependency on that prior training data set. Dependencies are an unavoidable reality of software, but it’s important to be aware of them and surface the trade-offs clearly.

Per Sample Judgement Calls

Ultimately a human will supervise each sample, generally one sample at a time. In all of this we must not forget the decisions each person makes has a real impact on the final result. There are no easy solutions here. There are tools available such as taking averages of multiple opinions, requiring examinations etc.

Often people, including experts, simply have different opinions. To some extent these unique judgements can be thought of as a new form of intellectual property. Imagine an oven with a camera. A chef who has a signature dish could supervise a training dataset that in a sense reflects that chef's unique taste. This is a light intro to the concept that the line between system and user content becomes blurred with training data in a way that's still developing.

Ethical & Privacy Considerations

First, it's worth considering that some forms of supervised data are actually relatively free of bias. For example, it's hard for me to imagine any immediate ethical or privacy concerns from our strawberry picking dataset. There are however very real and very serious ethical concerns in certain contexts. This is not an ethics book and there are already some relatively extensive books on the effects of automation more generally. However, I will touch on some of the most immediate and practical concerns.

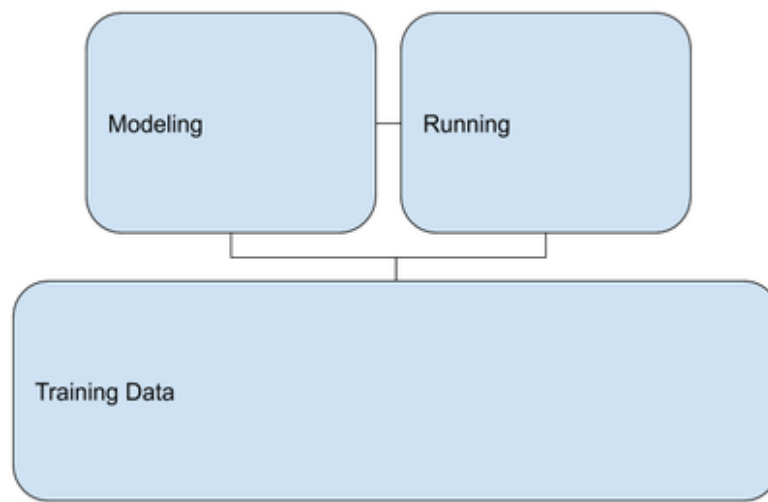
Technical Specifics

There are a variety of technical specifics, such as formats and representations that I will cover in some detail. While generally these representations have a "flavor of the month" feel, I will cover some of the currently popular ones and speak to the general goals the formats are aiming to achieve.

Why Training Data Matters for Supervised Learning

Now that we have a high level understanding of what Training Data is, let's consider why it matters.

[Visual showing Training Data as foundation for AI system]



Training Data is the foundation for successful supervised learning. Machine Learning is about learning from data. Historically, this meant datasets in the form of logs, or similar tabular data such as “Anthony viewed a video.”

Dataset

A dataset is like a folder. It usually has the special meaning that there are both “raw” data (such as images) and annotations in the same place. For example a folder of 100 images plus a text file that lists the annotations.

These systems continue to have significant value - however - they have some limits. They won’t help us build systems to interpret a computerized tomography (CT) Scan, understand football tactics, or drive a car. As models require less and less data to be effective, it puts more emphasis on creating application specific training data.

The idea behind Supervised Learning is generally a human expressly saying “here’s an example of what a player passing a ball” looks like. “Here’s what a tumor looks like”. “This section of the apple is rotten”.

Control

The question in any system: control.

How Training Data Controls the Model

Where is the control? In normal computer code this is human written logic in the form of loops, if statements, etc. This logic defines the system.

In Machine Learning I define features of interest and a dataset. The algorithm generates a model which is effectively the control logic. I exercise control by choosing features.

In a Deep Learning system, the algorithm does its own Feature Selection. The algorithm attempts to determine what features are relevant to a given goal. That goal is defined by Training Data. In fact, Training Data is the entire definition of the goal.

Here's how it works. An internal part of the algorithm, called a loss function, describes a key part of how the algorithm can learn a good representation of this goal. This is not the goal itself. The algorithm uses this loss function to determine how close it is to the goal defined in the training data. The training data is the “ground truth” for correctness of the model's relationship to the human defined goal.⁵

Dependencies

In traditional software development there is a degree of dependency between the end user and the engineering. The end user cannot truly say if the program is “correct”, and neither can the engineer. Their definitions of correctness may be similar, but are most likely not exactly equal. It's hard for an end user to say what they want until a “prototype” of it has been built. Therefore both the end user and engineer are dependent on each other. This is called a circular dependency. The ability to improve the software comes from the interplay between both.

With Training Data, the AI Supervisors, control the meaning of the system when doing the literal supervision. The Data Scientists control it when choosing abstractions such as label templates.

For example, if I as a supervisor were to label a tumor as cancerous when in fact it's benign, I would be controlling the output of the system in a detrimental way. In this context, it's worth understanding there is no validation possible to ever 100% eliminate this control. Engineering cannot, in a reasonable time frame, look at all the data.

Historical Aside

There used to be this assumption that Data Science knew what 'correct' was. The theory was that they could define some examples of correct, and then as long as the human supervisors generally stuck to that guide, they knew what correct was. The problem is, how can an english speaking data scientist know if a translation to french is correct? How can a data scientist know if a doctor's medical opinion on an X-Ray image is correct? The short answer is - they can't. As the role of AI systems grows subject matter experts increasingly exercise control on the system that supersedes Data Science.⁶

To understand why this goes beyond the “garbage in garbage out” phrase. Consider that in a traditional program, while the end user may not be happy, the engineer can, through a concept called unit tests, at least guarantee that the code is “correct”.

This is impossible in the context of training data, because the controls available to engineering, such as a validation set, are still based on the control executed by the individual AI supervisors.

Note: Classic cases where there's existing data that can't be edited. Context of changing the underlying data (unlike say sales statistics that are fixed).

Further, the AI supervisors are generally bound by the control exerted by engineering in defining the abstractions they are allowed to use. It's almost as though anything an end user writes, starts to become part of the fabric of the system itself.

This blurring of the lines between “content” and “system” is important. This is distinctly different from classic systems. For example, on a social

media platform, your content may be the value, but it's still clear what is the literal system (the box you type in, the results you see, etc) and the content you post (text, pictures, etc).

While this entire book is about the concepts around (control) of training data - it's worth understanding that:

- Training Data abstractions define Data Sciences control Not just algorithm selection
- Training Data literals define Supervisors control. Their control can supersede Data Science

Context Matter: Imagine a Perfect System

Let's imagine for a moment we have a Deep Learning algorithm that is perfect. For any given examples reasonably similar to our training set, say traffic lights, it will automatically detect said traffic lights 100% of the time, without failure. Is it perfect?

Unfortunately, our perfect system is not really perfect.

Because after celebrating our victory at detecting traffic lights - we realize we not only want to detect the traffic light, but also if it's red, red left, green, green left etc...

The system is perfect at detecting the abstractions we defined. Therefore the abstractions matter as much as the accuracy of the detections. It's worth pausing here and considering - even if the algorithm is perfect - there's still a need to understand the training data.

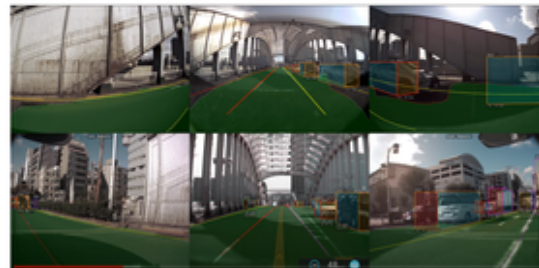
Continuing this example, we go back and update our Training Data with the new classes (red, green etc). And again we hit a problem. We realize that sometimes the light is occluded. Now we must train for occlusion. Oops and we forgot night time, and snow, and the list goes on.

Contexts in Training Data: Classic and Supervised

Classic and Supervised are the two major complementary camps within Training Data. Supervised has recently been in the limelight, in part because there are more degrees of freedom. The most key difference is exactly that - in the classic context there is only indirect human control, where as in the supervised context there is direct human control. This is not to diminish the continued importance of classic training data. It's best to think of them as different tools covering different problems rather than competing approaches.

Classic vs New

Owner	Country	File_Date	IPC_Class
Company A	US	6/18/2008	H05H13
Company A	EP	1/30/1998	A61N5
Company A	EP	1/30/1998	A61N5
Company A	EP	1/30/1998	A61N5
Company A	JP	8/28/1997	A61N5
Company A	JP	10/4/2002	A61N5
Company A	JP	1/27/2003	A61N5
Company A	JP	4/14/2003	A61N5
Company A	JP	5/13/2011	A61N5
Company B	JP	4/2/1998	G12B13
Company B	JP	4/2/1998	G12B13
Company B	JP	5/28/1997	A61N5
Company B	JP	11/12/1997	A61N5
Company B	JP	2/29/2000	A61N5
Company B	JP	4/30/2002	A61N5



① ② ③ ④ ⑤ ⑥

Figure 1-6. Classic and new approaches

Discovery

Training Data classically has been about discovery of new insights and useful heuristics. The starting point is often text, tabular and time series data. This data is usually used as a form of discovery, such as for recommender systems (Netflix suggested movies), anomaly detection, and car reconditioning costs.

Crucially there is no form of human “supervision”. In the modern deep learning context, there may not even be feature engineering. To slightly oversimplify, the data is fixed, a very high level goal is defined, and the algorithm goes to work.

Feature engineering

A practice of selecting specific columns (or slices) that are more relevant, for example the odometer column for a vehicle reconditioning cost predictor

Monkey See, Monkey Do

With Supervised, we already know what the correct answer is, and the goal is to essentially copy that understanding. This direct control makes supervised learning applicable to a new variety of use cases, especially in the realm of “dense” data, such as video, images, and audio. Instead of the data being fixed, we can even control generating new data, such as taking new images.

We will cover a refresher on the Classic context and in-depth comparison of how it relates to this new Supervised context.

Training Data Sample Creation

Let’s explore, from the ground up, how to create a single sample of training data. This will help build understanding of the core mechanics of what the literal supervision looks like.

Introduction

Imagine we are building an autonomous system, such as a traffic light detection system.

The system will have a deep learning model that has been trained on a set of training data.

This training data consists of:

- Raw images (or video)
- Labels

Here we will discuss a few different approaches and the appropriate training data.

Approach One: Binary Classification

As an example, two of the images in the set may look like this:



Figure 2 TK

To Supervise Example One, we need only two things:

1. To capture the relation to the file itself. E.g., that it's a "sensor_front_2020_10_10_01_000." This is the "link" to the raw

pixels. Eventually this file will be read, and the values converted into tensors eg position 0,0 having RGB values.

2. To declare what it is in a meaningful way to us. “Traffic_light” or `1`

And for Example Two: we could declare it as:

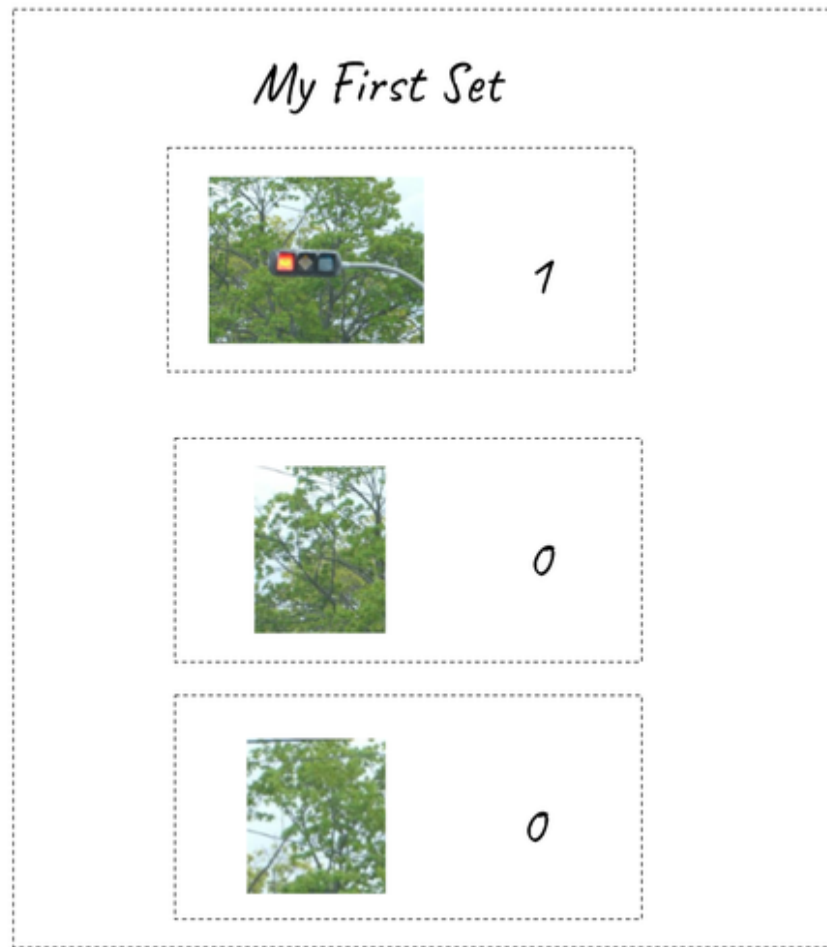
1. “sensor_front_2020_10_10_01_001”
2. “No_Traffic_light_not_present” or `0`

That’s it. While there is research on ‘zero shot’ and ‘one shot’ learning, in general we will use a set. So for example we would have a list of three images, and three3 corresponding arrays detailing A and B

Let’s manually create our first set

You can do this with a pen and paper or white board. First draw a big box and call it “My First Set”.

Then I draw a smaller box, and put a sketch of a traffic light and the number 1 inside it. Repeat that 2 more times drawing an image without a traffic light and a 0.



This is the core mapping idea. It can be done on pen and paper and can also be done in code. Realistically we will need proper tools to create production training data, but from a conceptual standpoint this is equally correct.

For example, consider this python code. Here we create a list and populate it with lists where the 0th index is the file path and the 1st index is the ID. The completed result (assuming the literal bytes of the .jpgs are available in the same folder) is a set of training data.⁷

```
Training_Data = [  
    ['tmp/sensor_front_2020_10_10_01_000.jpg', 1],  
    ['tmp/sensor...001.jpg', 0],
```

```
['tmp/sensor...002.jpg', 0]]
```

This is missing a label map (what does 0 mean?). We can represent this as simple dictionary as:

```
Label_map = {  
0 :  
    "Traffic_light",  
1 : "No Traffic  
light"  
}
```

Congrats! You have just created a Training Data Set, from scratch, with no tooling, and minimal effort! With a little wrangling, this would be a completely valid starting point for a basic classification algorithm.

You can also see how, by simply adding more items to the list, you can increase the training data set.

While “real” sets are typically larger, and typically have more complex annotations, this is a great start!

What are we doing?

Let’s unpack the human algorithm we are using.

Look at picture

Draw on knowledge of traffic lights

Map our knowledge of of traffic lights to sparse value eg “Traffic light is present”

Making it clear to the Machine

While it’s obvious to us - it’s not so obvious to a computer. For example, prior approaches in this space such as Histogram of Oriented Gradients, and other edge detection mechanisms, have no real understanding of “traffic light” and “not traffic light.”

Not so over simplified

It's true that modern self-driving teams use more advanced approaches (that we will describe later).

For the sake of this example, we imagine the Traffic Lights are pre-cropped by some process. We may make assumptions about the angle of the traffic lights. If we have known maps etc. this may be quite reasonable and reliable.

Ultimately, other approaches typically build on classification, or add “spatial” properties to it. At the end of the day, even if some prior process runs, there's still a classification process being done.⁸

Approach Two: Upgraded Classification



Why are we using strings or integers?

We will introduce the concept of the Label Map

Generally speaking, most actual training will use integer values. However, those integer values typically are only meaningful to us if attached to some kind of string label:

```
{ 0 : "None",  
  1 : "Red",  
  2 : "Green"}
```

While mapping of this type is common to all systems, these label maps can take on additional complexity. It's also worth remembering that in general, the 'label' means nothing to the system. It's mapping the ID to the raw data. If these values are wrong it could cause a critical failure. Worse - imagine if the testing relied on the same map!

That's why when possible it's good to "print output" eg that you can visually inspect the label matches the desired ID. (specifically in regard to train/val/test) [Technical] As a test case, could also Assert on a known ID matching String. [TODO move this to a different section maybe. TODO add a more general comment about where the testing concepts are introduced as this isn't really the main point of testing.]

Supervision vs Annotation

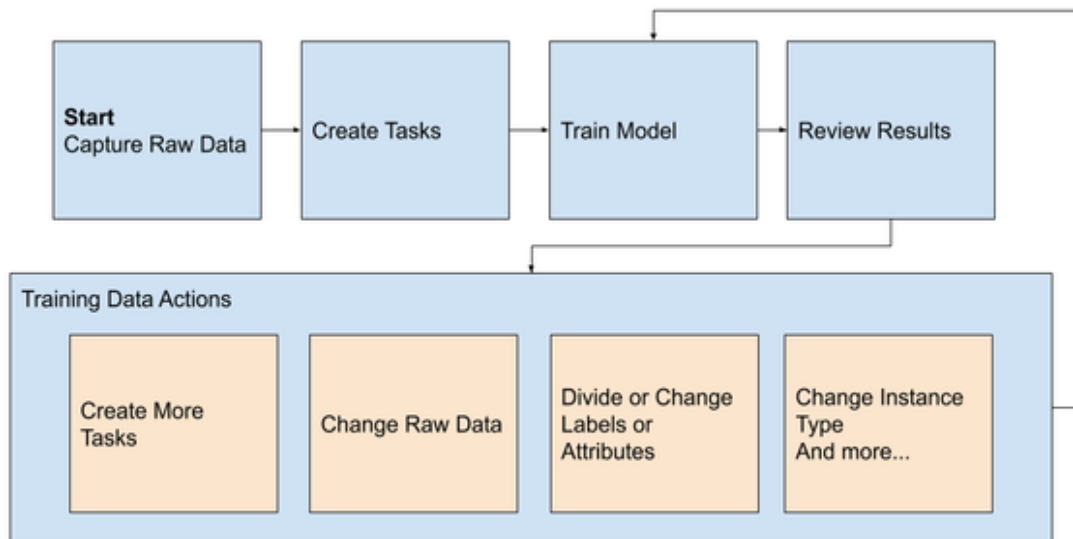
Annotation is a popular phrasing for anything to do with training data. Annotation generally implies adding or drawing information - without regard to any concept of a system. In words to annotate is generally a "secondary" action. This masks the importance and the context of the work being done. Supervision more accurately reflects the overall scope and context of the work. It also better reflects the increasingly common context of correcting (supervising!) an existing model or system.

Where is the Traffic Light?—Objectness score

The problem with the above approach is that we don't know where the traffic light is. There is a common concept called an objectness score, and other more complex ways to identify location. We will cover location concepts in more detail.

Training Data Process Introduction

Now that we have covered the basics of how a single sample is created, and introduced some key terms, let's take a high-level look at the process.



Getting Started

This process will require several stages.

Raw Data and Tasks

Training data starts with identifying and capturing raw data. The next step is to design the Tasks, chiefly the Labels and Attributes.

Train Model and Review Results

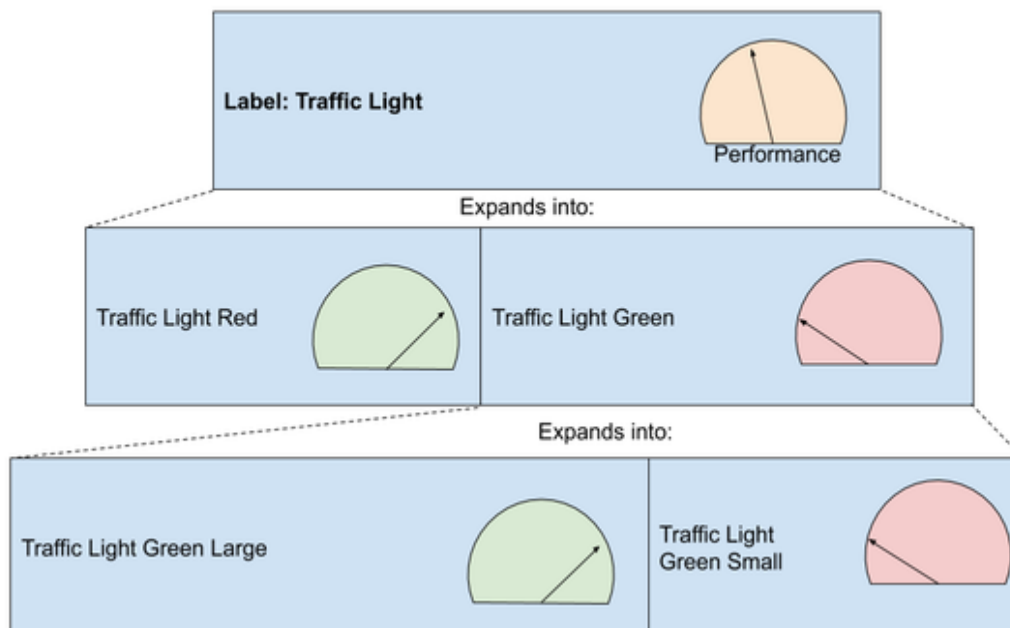
As soon as a minimal dataset is constructed it's good to start training a model. This will give us clues to help better design the Tasks.

Training Data Actions

These are actions that can be taken generally after some form of information from the model training process.

Change the Labels and Attributes

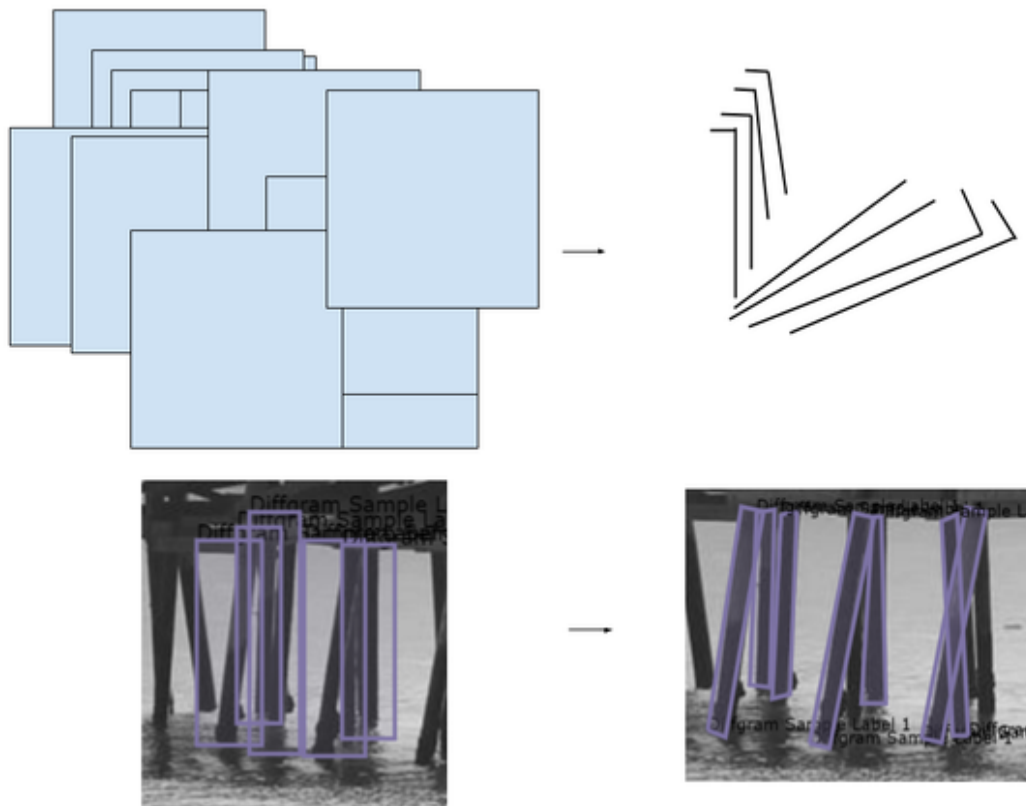
This is one of the most common approaches. One example is to divide and conquer the label classes, especially poorly performing classes. Essentially this is to both identify and improve the specific classes that are the weakest. Here we have say Traffic Light. Performance is mixed. It's unclear which examples are needed to improve performance. When reviewing the results, we notice that Green seems to show up more often in the failure cases. One option is to try to add more green ones to the general traffic light set. A better option is to “split” the class Traffic Light into “Red” and “Green”. That way we can very clearly see which is performing better. We can repeat this until the desired performance is reached. For example, again splitting between large and small. There are a few intricacies and approaches to implementation of this but they generally revolve around this idea.



Change the spatial type of instance

Imagine you started with choosing segmentation. Then realize the model is not training as desired. You may be able to simply switch to an “easier” task like object detection, or even full image classification. Alternatively perhaps object detection is yielding a bunch of overlapping boxes which aren’t helpful and you need to switch to segmentation to accurately capture the meaning.

At the time of writing there are nearly a dozen popular spatial methods. While it may appear clear which methods are less ideal for a certain case the optimal method is often less clear. This is also a bit of a moving target as annotation tooling and model training methods change.

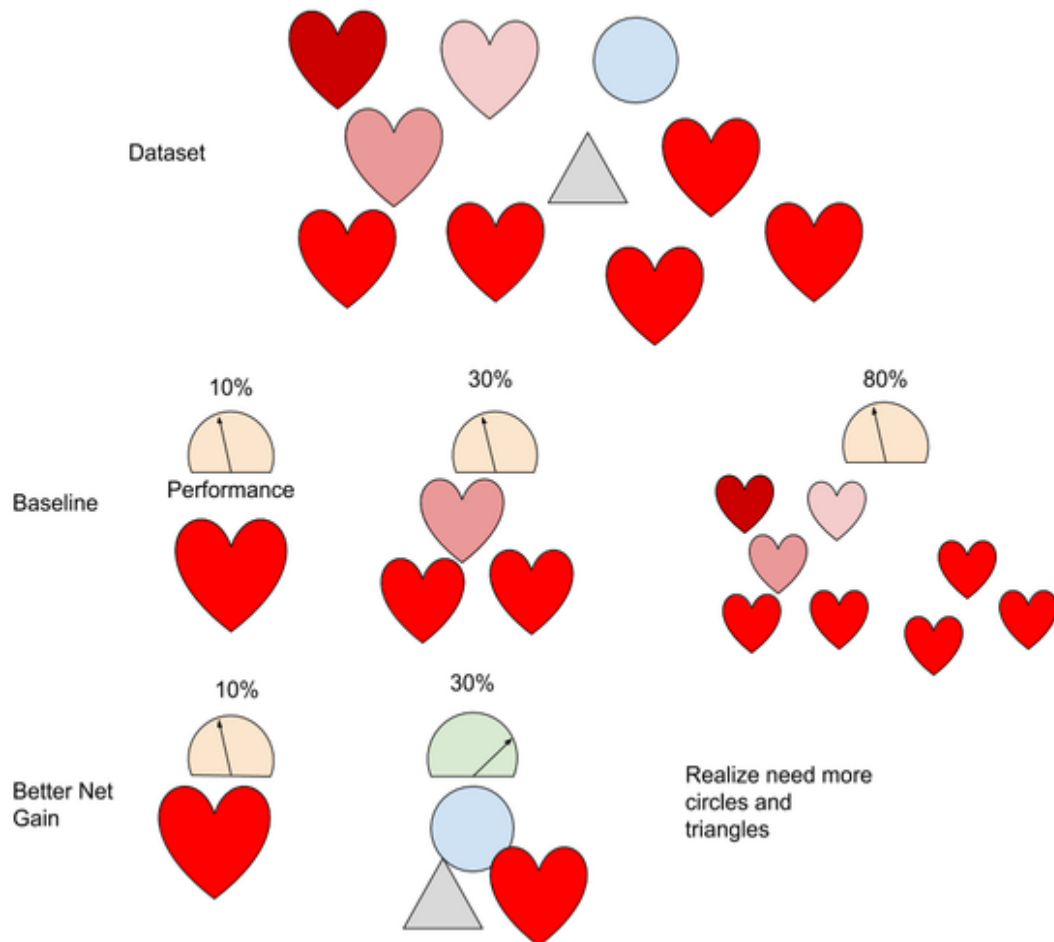


Create More Tasks

Annotating more data for better performance has almost become a cliché already. This is often combined with the other approaches. For example,

dividing the labels, or changing the spatial type, and then supervising more. The primary consideration here is if more annotation will provide net lift.

Net Lift Introduction



For those with technical knowledge let's first dispel a notion - this is not about balancing the dataset. Try to forget the concept of balancing while considering this.

To illustrate the need for net lift consider a raw, unlabeled dataset, in which 10% is labeled. [Shown Fig as Dataset]. As a baseline approach we will random sample data 3 points (10/30/80%). At each point we will look at model performance. If the performance is unchanged we will stop.

By chance we draw all hearts each time. Each additional heart we supervise provides minimal value - since we already have seen many hearts. Further,

we don't really understand the complete production picture because we did not encounter circles or triangles.

Two different things here:

1. The idea of identifying previously unknown cases
2. The idea of wanting to maximize the value of each net annotation.

Change the Raw Data

Change the sensor angle

Change what part of the screen is being capture

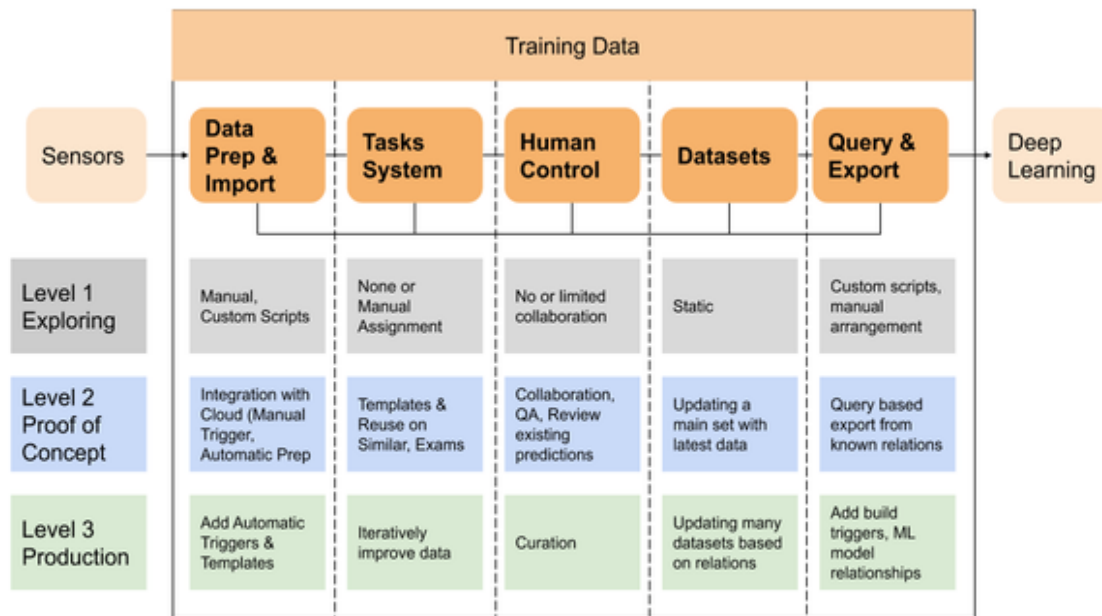
Split the models / heads

A model may need less data for one class then another - sometimes by multiple orders of magnitude

Levels of System Maturity of Training Data Operations

The above described process is the overarching strategy. We will also zoom into the more tactical concerns of operating and executing that process. This is Machine Learning Operations (MLOps) for supervised data.

You will learn about the 5 major stages of operations, Data Prep, Tasks, Literal Human Control, Datasets, and Export. Both the specific operational concerns of those stages and the 3 major levels of system maturity for each. From early exploring, to proof of concept, through to production.



Training Data in the Ecosystem

Training Data sits in between Raw Data, such as sensors, and Modeling (Training and Prediction).

For example, you may create Training Data with one tool, and Train with a different tool. I will touch on a high level map of adjacent areas, tools, and popular integration points.

Tooling

There are tools designed specifically for Training Data, such as Diffgram. I will talk about open source and commercial options here. Expressly highlighting trade-offs of popular tools. While generally I will aim to stay tooling agnostic I will ground some of the examples in specific tools.

Applied vs Research Sets

The modern needs and form of training data continue to rapidly evolve. When people think of datasets popular research datasets, like MS Common Objects in Context (COCO) 2014⁹, Imagenet 2009¹⁰, (both vision) and the General Language Understanding and Evaluation (GLUE) 2018¹¹¹² sets come to mind.

These sets are designed for research purposes, and by design, they evolve relatively slowly. In general, it's the research that's designed to change around these sets, not the sets themselves. The sets are used as benchmarks for performance. For example here, the same set is used in 5 different years. This is useful for research comparisons. The core assumption is the sets are static.

In the context of a practical commercial product, things change. In general the rule of thumb is that data more than 12 months old is unlikely to accurately represent the current state. Generally the assumption for practical sets is that they are only static for the moment of literal training, but are otherwise always changing. Further commercial needs are very different from research. Time to create the sets, time to update, costs to create the sets, implications of mistakes, etc. are very different from a research context.

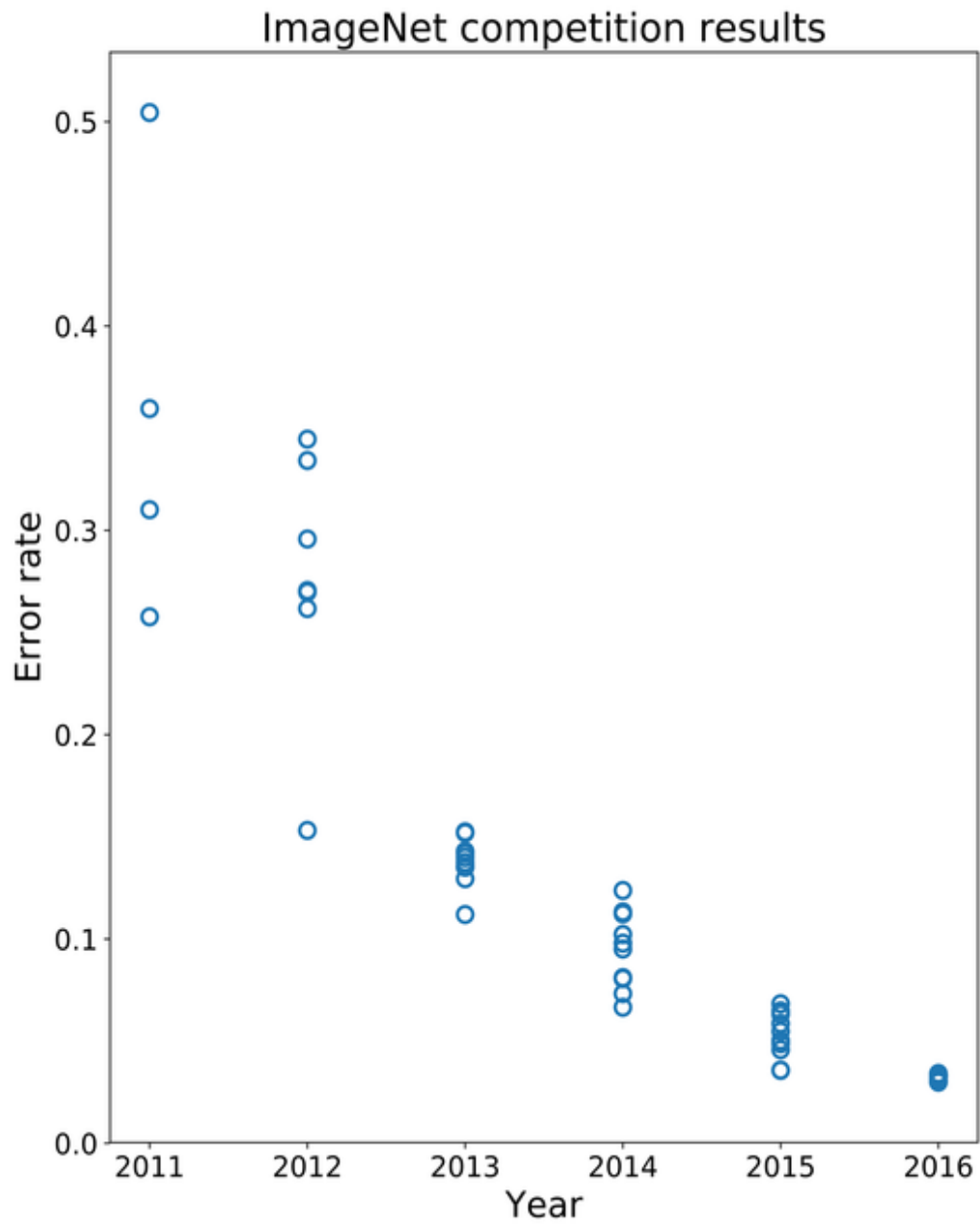


Figure 1-19

Training Data Management

I have introduced the strategic level process of Create-Predict-Update. And we have touched on the operational concerns of the process. Cross cutting both of those concerns is the ideas around Training Data Management. Generally this is concerned with: Organization of humans doing the literal tasks and organization of data throughout the entire operations cycle.

Introduction

One of the central ideas behind Training Data Management is maintaining the “meta” information around the Training Data, such as what assumptions were present during the creation of the data. In the context of continually improving, and reusing data, for a large organization this is especially important. Imagine spending a significant budget on setting up pipelines, training people, creating literal sets, only to “lose” that information because of improper handling.

Completed vs Not Completed

Knowing which samples are completed at any given moment in time is surprisingly challenging. At a high level this is partly because we are trusting other humans to choose what is completed and what is not. Second, because the schema often changes, the definition of complete also changes. For example a file may be complete relative to a schema that is no longer relevant.

The most minimal management needed is to separate “complete” samples from “incomplete” samples. It matters because any sample trained on is considered to be valid to the network. So for example if this sample was included with no further labels, it would be considered background:

This can cause severe problems:

- It makes it hard to create a performant network / debug it.

- If this class of error makes it to a validation test set - the test set will be equally affected.

While this may seem trivial in small set sizes, for most significant sets:

- The data scientist(s) will never observe 100% of the samples, and sometimes will never observe a significant set of the samples, or even any! In the case of using transfer learning, eg pretrain on image net, anyone who has ever used image net has done this!
- In large set sizes, it is unreasonable to expect any single person to be able to review all of it.

This is the “test case pass but checkout button is invisible to the user” case for training data.

In general, one should use software to manage this process and ensure that completed/not completed is well tracked. It’s also worth considering that in many externally created datasets / exported datasets this “completed” tracking gets lost, and all of the samples are assumed to be completed.

When Completed Is More Complicated

A few notes:

- Consider that with multiple people looking at the same file, each Task may be “completed”, but the “File” is not. (or is grouped with different sets).
- In multi-modal cases, there may be a set of files to complete.
- It’s common for Label Definitions (and other things to change) to change. This may in effect ‘reset’ the completed status.

For more complex file types there may be confusion on what ‘complete’ means, e.g. for long videos

Freshness

Certain aspects of training data “age well”. What does age well:

- Transfer learning type concepts / “lower level” representations (likely order of decades (unproven yet))
- Tightly controlled scenarios (eg some types of document reading, medical scanning) (likely order of years)

What doesn’t age as well:

- Novel scenarios / Unconstrained real world scenarios (self driving) —anything that involves a sensor that’s not in tightly controlled space, e.g. anything outside.
- Specific “heads/endpoint labels”
- Step function changes in the distribution, eg sensor type change, scenario change (day/night), etc.

Different applications naturally have different freshness requirements. However, some models that appear to “last a long time” are doing so because they were relatively over built to begin with.

Part of the “trick” with the whole freshness aspect is which samples to keep. If possible, it’s good to test and compare a rolling window vs continuous aggregation.

Maintaining Set Metadata

In the rush to create training data we often lose sight of key contextual information. This can make it very difficult to “go back”, especially as one develops many different sets. A training data set without context to a real distribution, eg the real world, is of little value.

Task Management

The reality is that with humans involved there is inevitably some form of organization or “task” management system needed - even if it’s ‘hidden’

behind the scenes. While often this ends up involving some form of people management skills that is beyond the scope of this book. When we talk about tasks we will generally remain focused on Training Data specific concerns, such as tooling and common performance metrics.

Challenges Introduction

Let's examine an example.

Failures caused by Training Data

In April 2020, Google deployed a medical AI to help with COVID-19.¹⁴ “the model had been trained on high-quality scans; to ensure accuracy, it was designed to reject images that fell below a certain threshold of quality.” “They sometimes wasted time trying to retake or edit an image that the AI had rejected.” They ended up rejecting about 20%, if including the “retry” attempts this likely is above 25%. This does not even account for the accuracy of the model. Consider an email service that failed (and refused to send even after retrying) every fourth email you tried to send. This shows how important it is to align the training data with what will be used in the field.

Failing to Achieve the Desired Bias

When we think of classic programs, any given program is “Biased” towards certain states of operation. For example, an application designed for a smartphone has a certain context, and may be better or worse than a desktop application at certain things. E.g. A spreadsheet app may be better suited for Desktop.

When we write programs, we bias them towards certain goals. For example, some applications may be very easy to edit/update, whereas others (such as say sending money) make it difficult to edit/undo an operation. Once a program like that has been written, it becomes hard to “unbias it”. The edit focused program was built assuming the user would be allowed to

(generally) - edit stuff. Whereas the money sending app has many assumptions built around an end user not being able to “undo” a transaction.

There’s a similar concept in Training Data. Let’s imagine a crop inspection application. Let’s say it’s mostly designed around diseases that affect potato crops. There are assumptions made regarding everything from the “raw” data (e.g. that the leaves are certain heights), to the types of diseases, to the volume of samples. It’s unlikely it will work well for other types of crops.

I will cover Bias from many angles and provide practical tips on how to work with Training Data to achieve your desired Bias.

Summary

I have introduced high level ideas around Training Data for Machine Learning. These concepts will ground us for the rest of the book where we explore more breadth and depth.

Training Data is control of the system, the goal for the system to learn. Training Data is not an algorithm or a single dataset. It’s a paradigm that spans people from Subject Matter Experts, to Data Scientists, to Engineering and more. It’s a way to think about systems that opens up new use cases and opportunities.

I introduced core concepts, such as literal representations, assumptions, randomness, automation processes, tooling, and more. You learnt two tasks, how to create a single sample and how to expand that case into an upgrade classification. We will cover all of these concepts and popular modern approaches to tasks in more detail.

The process of getting started, from raw data to defining tasks for human supervisors was introduced. I showed the core loop of Create-Predict-Update. You learnt about how you change labels, raw data, net-lift, spatial types and more to effectively and efficiently control the results. I introduce the major Training Data parts of MLOps and a path for planning system maturity. I introduced the concept of Training Data Management -

organization of Training Data and People. Lastly, introducing hard challenges faced by even the most competent teams.

In the next few chapters we will dive deeper into understanding Training Data - concepts, mindsets around it, and technical representations.

-
- 1 Assembly, ‘Any low-level programming language in which there is a very strong correspondence between the instructions in the language and the architecture’s machine code instructions’ https://en.wikipedia.org/wiki/Assembly_language
 - 2 .
 - 3 Usually by reference id only
 - 4 In one reference frame. Multi-model can be represented as a set of related instances, or spatial locations for a given reference frame such as camera id x.
 - 5 To further understand why this is the case, consider that in many use cases the loss function is predefined by the task. For example, research in object detection yields a State of the Art approach with associated specifics including the loss function. That said, in the case of “unsupervised” learning, the loss function is more closely related to the goal. While this may seem like a contradiction at first blush, for practical purposes it’s generally not relevant to supervised cases.
 - 6 There are statistical methods to coordinate experts’ opinions but these are always “additional”, there still has to be an existing opinion.
 - 7 Sharped eyed readers may notice this becomes a matrix once completed. The matrix shape has no significance here and it’s generally best to think of these as a set. However python sets introduce quirks also not relevant here—so I use a list.
 - 8 Algorithms usually predict a continuous range, and then do a function (eg softmax) to convert to these category values. (TODO double check exact wording here)
 - 9 <https://cocodataset.org/#home>
 - 10 <http://image-net.org/about-publication>
 - 11 <https://arxiv.org/abs/1804.07461>
 - 12 These sets all have taken an incredible amount of work, and some have been substantially updated over time. Nothing in here is meant to take away from the contributions of them.
 - 13 [https://en.wikipedia.org/wiki/ImageNet#/media/File:ImageNet_error_rate_history_\(just_systems\).svg](https://en.wikipedia.org/wiki/ImageNet#/media/File:ImageNet_error_rate_history_(just_systems).svg)
 - 14 <https://www.technologyreview.com/2020/04/27/1000658/google-medical-ai-accurate-lab-real-life-clinic-covid-diabetes-retina-disease/>

Chapter 2. Training Data Concepts

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 2nd chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at jleonard@oreilly.com.

Schema Deep Dive Introduction

What do you want your AI system to do? How will it accomplish it? What methods are you going to use? In this chapter I dive into some of the foundational concepts around supervised training data.

The real world is messy. And often commercial applications require a level of detail that’s far more specific than high level labels. There are many ways to structure this. In general these concepts exist as “pivot points” in the application.

I will introduce the core concepts around Training Data Schema - a paradigm for encoding Who, What, Where, How & Why. The Schema is the overall representation of Labels, Attributes, their Relation to each other. And more. It’s how we represent the meaning of what something is, where it is and more.

Info box?: Schema is also known as Ontology or Label Setup.

This builds on the high level concepts of Labels and Attributes introduced in Chapter 1. After I will map these training data concepts back to Machine Learning tasks.

In this chapter you will learn:

- Tools to get started in reasoning about your system and building your Schema
- Trade offs of popular methods and tasks
- Specifics of some high level ideas from chapter 1.
- Let's dive into ways to represent Training Data.

What is it? Labels & Attributes

Let's examine labels and attributes.

What do we care about?

Generally, we care where something is, what it is, and how it relates to other things.

Labels and Attributes are the tools we use to express what something is. In the next section I will introduce Spatial types to discuss Where something is.

Why "What" is important.

While the spatial location aspects are needed, and challenging in some cases, in general there are stronger limits to how spatial concerns can expand. Whereas the concept of representing What something is, can expand with near infinite complexity.

Label Introduction

Labels are typically “top level” semantic meaning. In the base cases, they may represent only themselves. Eg a label “car” may map to literally “car”.

Labels are the top level id for a set of attributes of interest. In the most generic case, a top level label could just be “object”. While in theory it’s possible to do a structure purely off of Attributes in practice Labels are a very useful distinction.

For technical readers, to help ground this idea, I like to compare it to SQL design.

T
a
b
l
e
2
-
l
.
C
o
m
p
a
r
i
s
o
n
t
o
S
Q
L

Intuitive Concept	Training Data	SQL
-------------------	---------------	-----

Set	Label	Table
-----	-------	-------

Attribute	Attribute	Column
-----------	-----------	--------

Of particular note here, tables don't usually have a "Type" whereas Columns of course do. In this same sense, a Label doesn't have a type - if an instance exists it's there, whereas Attributes do have types. Interesting, in E.F. Codd's "The Relational model for Database Management", he mentions that Columns were originally thought of as Attributes.¹ While far from a perfect analogy it helps convey the general idea.

- For videos, labels help constraint relationships and organize sequences
- For organizing what to show, it can feel better to relate Attributes to a Label. For example, an Attribute like Occluded may be common to 100s of labels.
- When doing literal labelling, it's a smoother transition to go from a top level concept (especially in cases where it's machine made).

Attributes Introduction

Attributes usually represent the bulk of the "What is it" encoded meaning. Attributes are usually represented in a constrained manner. Generally the constraint will include a Type, such as Multiple Select, Select, Text, A child Group, Date etc.

Type, in this context straddles a strange line between User Interface Form types and Data Types. For example a Slider UI control could collect Float or Integer data. In the most general case Attributes are often treated as Strings. Where needed both a UI type and a Data type can be stated.

Furthermore, there may be Constraints on the form collection, for example how many are allowed to be selected. Continuing the slider example, it would have a lower and upper bound, a predefined selection. Generally an administrator defines valid values.

In general, a human is selecting these values or at the very least reviewing them. In that context usually each group will have additional information, such as

:

- Prompt
- Display order

Usually, Attributes are defined as some kind of Template. Values are unique to each Instance and may be concrete or a reference. Typically, any kind of number collection, free text, date, etc. would be concrete, eg “3.14”. whereas when possible, if a known set is provided for say a selection from a list of 6 elements, a reference ID can be used.

- Common Attributes Include:
- Occlusion (Blocked/Out of view) & Truncation (Out of frame)
- Depth / Hierarchy of Labels/ meaning. Eg What type of Action (Moving, Jumping, Running)
- Directional Vector (eg front/back/side)



Figure 2-1. Example of labeled attributes

To make this easier we use Constraints. We may constrain for example that a car can have a directional vector but a sidewalk does not. A disease may have multiple types, or we may constrain to only choose a single type. In the simplest case if we imagine Labels of “cat” and “dog” being the 2 choices, we are constrained to limit to only those two choices.

Label’s Relationship to Attributes

Earlier, I introduced a Label as the highest level of semantic meaning defining “What” something is. For example a “Strawberry” or “Leaf.” Attributes were introduced as a way to expand the What.

So how do these work together?

Let’s imagine we are building a system to automatically detect what percent of fans at a sports match are cheering for one team over another. And perhaps it also needed to identify “Offensive” content. We may want to identify articles of clothing, such as t-shirts, pants, and ballcaps. There are “What” representations like color, team logos, is it offensive, etc. that are common to all of those items. There may also be certain things that are only relevant to say t-shirts or ballcaps

One way to structure this is to have t-shirts, pants, and ballcaps as the Labels. Then to create an Attribute called “Color”, with the various properties such as “Red”, “Blue”, “Green”. All of the labels can have access to this attribute.

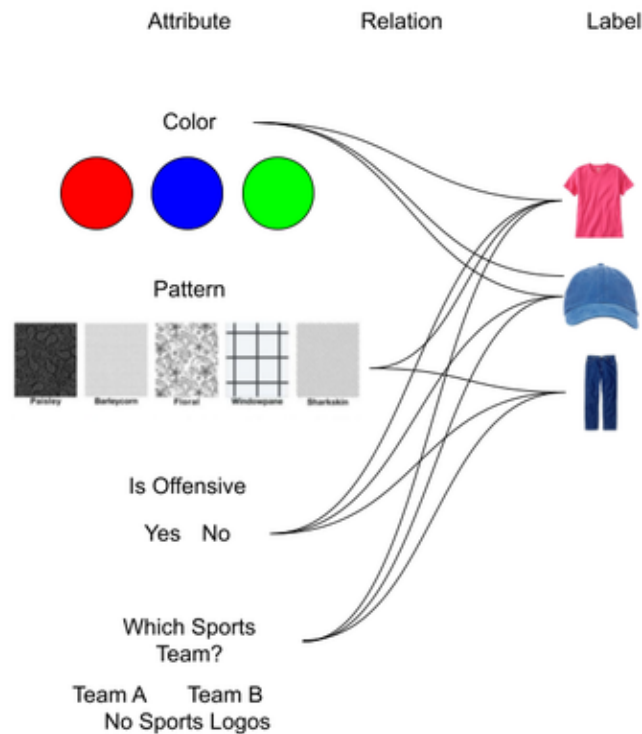


Figure 2-2. Relationship between Attributes and Labels

A high-level example of trade-offs to think about here. Is it worth it to identify color? Perhaps it's common to wear a team's colors, but a logo may not be visible. So perhaps having a good idea of color may be used for further downstream processing to determine which team the person is cheering for.

Schema Complexity Trade-Off

More labels and attributes means more labeling time. This is multiplied if there are many objects present. In a Video context if the attributes are different in each frame then this further multiplies it. It can also create more complexity for model training.

The benefit of more here is often a “smarter” system. For example, if we don't have labeled data on “is offensive?” then we wouldn't be able to train a model. More labels also offer more insight into performance.

Generally speaking, systems expand how many labels and attributes they have

over time. Schema complexity is not something you need to worry about right off the bat. If you aren't sure where to start here a good default is to start with 2-10 Labels and 0-2 Attributes.

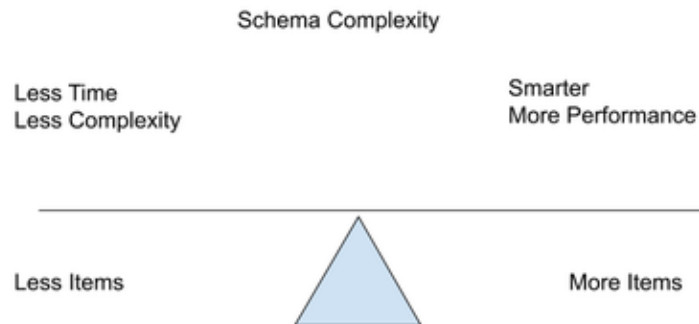


Figure 2-3. Schema Complexity

One approach is to multiple the Labels by the number of Attributes. For example, a network with 20 labels and 5 attributes (with say 10 options each), is 1000 ($20 \times 5 \times 10$) labels. This is a good “back of the napkin” way to understand how complex it is. In general, this is actually more useful than say a total count of images. A fully labeled dataset with 1000 total labels on 500 images is “more complex” than 3 labels on 1,000s of images.

Attribute Item Depth

Imagine a grocery store checkout system. There may be many sizes of a specific brand of cereal box and we aim to identify the specific stock number (SKU). In this context, we may wish to “database back” our attributes. Such that as inventory changes, the options presented change. Further that we load thousands or even millions of options. Putting aside UI challenges regarding searching and selecting each attribute - this is a perfectly reasonable solution. At the time of writing there is no “right” answer here. A system may easily have a handful of attributes, or thousands.

Attribute Schema Depth

Schema depth is defined by conditionals and complex hierarchies. At any point we can expand an attribute into a child node or a list of nodes. For example, selecting “No” could expand a “No - Options” node. An option for any given

selection is to expand into a child node. Generally the same principles discussed here apply.

You can get far without doing this. At the time of writing some AutoML systems will struggle to even effectively train 5 labels, so doing a complex hierarchy here is as the name would imply an advanced use case. It's worth being aware of the existence of this if it becomes relevant to handle edge cases.

Relationship to Spatial Types

Imagine for our sports fan detector, we have a “Person” as the top level object. And then for detecting if they are wearing an offensive shirt or doing an offensive action, we have an “is_offensive” attribute.

One consideration here is that if an ML Task like semantic segmentation is chosen, and a polygon used, do the attributes chosen correspond to every pixel in the outline? Each instance generally has one spatial type. So while we are thinking about the shirt, the actual data is for the whole person.

This risk is because while we as humans know the offensive part is the shirt, what we have actually encoded into the machine is that the whole area is offensive. To help visualize this, consider that to the machine, these things are functionally equivalent.

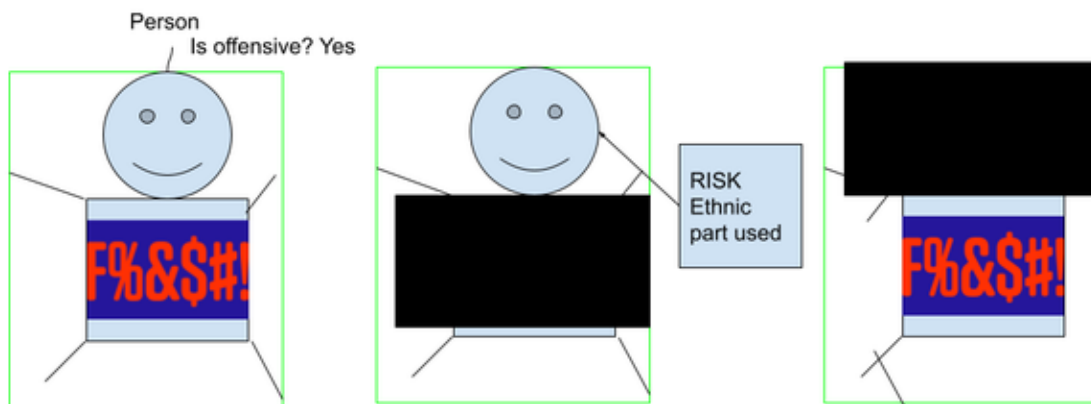


Figure 2-4. Functionally equivalent data—example of risky schema for unwanted bias

The model may just as easily use a feature from the racial features of the face instead of the t-shirt, making the risk of it classifying someone who has some facial or ethnic properties as “offensive”. This relationship is slightly more

obvious with labels because they are “top level”, but can be more subtle in “buried” groups of attributes.

Consider that some systems (or even people) will use this type of training data without looking at the samples, or critically analyzing it’s Schema construction. To avoid this, we want to make sure the Schema encodes what’s actually present, as accurately as reasonably possible, and when possible that we record assumptions made around the system.

One way to avoid spatial bias

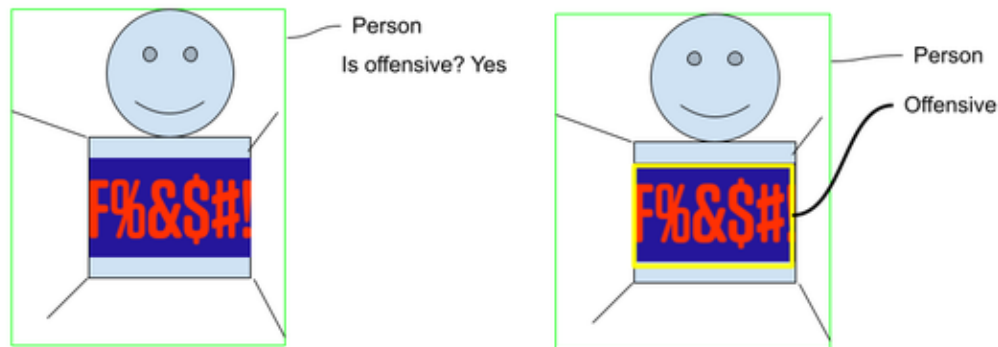


Figure 2-5. One solution to this is to break out the label into two

One solution to this is to break out the label into two. Then the spatial location (eg here of the Yellow box) can better represent the label. The key point here is that we are trying to get the spatial location to be representative of the item of interest - and only the item of interest.

Now perhaps in the future certain training methods will allow for this. However we can relatively easily mitigate this risk today at the training data level simply by being aware of this and being more precise in how we lay out our classes.

To further appreciate the importance, imagine an automatic alarm system that is monitoring for a {gun, knife, bomb} or some other real time threat (such as airport security). If the system is trained on images of people with guns, but doesn’t separate the person from the gun, there is a risk of it falsely triggering when a person of a certain ethnic background is present (when no gun is!).

Joint responsibility

If you are a data specialist you may think this doesn't apply to you. Because of course this is a "Schema level" concern. Equally, if you are designing the schema, you may assume that the data specialist will notice this and raise an alarm. Clearly this is a joint responsibility to identify when the Schema is at risk of creating bad data.

If this is sounding too namby pamby consider there is another reason to be aware of this: it creates better quality data, which results in faster and better models.

For example here, you may need a lot more data to identify offensive items if you are doing it at the person level. By scoping the spatial location to better match the object in question we improve the overall quality of the data.

Many more trade offs

As mentioned earlier, this is not an ethics book. Here I outlined the direct technical effects of choosing attributes and their related spatial locations.

Importance of What it is

Earlier I mentioned briefly why "What" is important. To expand on this consider that there oftentimes there are severe limits to the definition of where. In complex cases, multiple sensors can be combined to represent a complex 3D view of where. In Segmentation cases, per pixel is significant.

In each of these cases though, there's a relatively finite ability to declare spatial locations.

Either there's an object there or not. Or is it?

What about partial objects? Often what we see for complex cases, is that a general localization is provided, such a sports player:

[TK: drawing of player with bounding box].

In this case, a player can be said to be "running", or maybe "attacking"? What about identifying who the player is? Sure we can say the "limb" is in a certain position, but what does that really mean?

Further, through a variety of assist and pixel based methods, localization is relatively speaking easier to solve. whereas the “meaning” of some concepts in life are the subject of philosophy books.

To be clear there can still be significant challenges localizing novel objects, but generally for ongoing concerns, the “meaning” of what it is, becomes more central.

The Hidden Background Case

When we think about where something is, it’s sometimes more useful to start with where something is not. For example, when we say it’s a traffic light at some point, what we are actually saying is 96% of the image is not a traffic light. This is Where conflated with What.

In the case of object detection, this is often handled with an “objectness” score, and integrates with region proposal methods. The implementation of that is beyond the scope of this book. Where it relates to training data is that often training data is created without a 1:1 correlation to how it will be used.

For example, it may be visually easier for a hierarchical “nested” list to be displayed to a human. But in actual practice that may be implemented in the network in many ways. Such as “flattening” the labels (red_occluded_20, red_occluded_40), but combining multiple networks, or by using an architecture that supports nesting.

Depending on the implementation, it’s possible to predict:

- An object here
- Object classification is x

These methods may change in the future, and there is already significant breadth of approaches.

Technical Specifications

There are a few things to keep in mind.

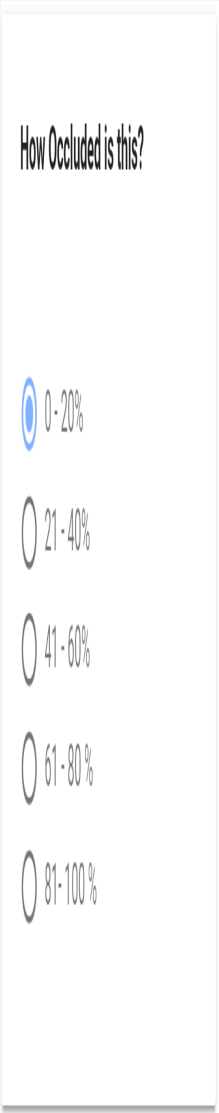
Representation, by reference vs by value

This matters because your Schema will likely change over time.

As shown below, some things in a Schema can be defined as references. For example, here, each option could be represented as a single radio select. However for other types of attributes, like free text, the answer may be unique for each instance.

Sometimes, for ease of change management, it's useful to "lock" either the schema or pass by value even when it's possible to pass by reference. At a very high level, generally it's easier to think of labels as "slow moving" concepts and attributes as "fast moving". Because it's relatively easy to add and remove attributes from labels, but changing labels may introduce breaking changes to other attributes. For technical folks the analogy of tables (labels) and columns (attributes) in a database works well here.

Technical specs:

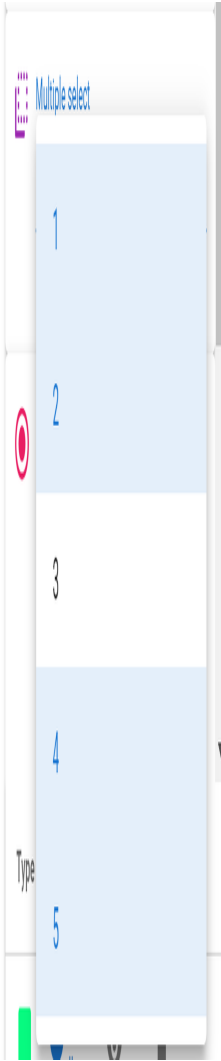
Attribute	Example	Technical Output Types	Reference or Value	Constraints
Select (Drop down) or Radio Buttons (Meaning all options shown)		String	Reference	Allow list

Multiple Select

List

References

Allow list



Free Entry

String, Integer

Value

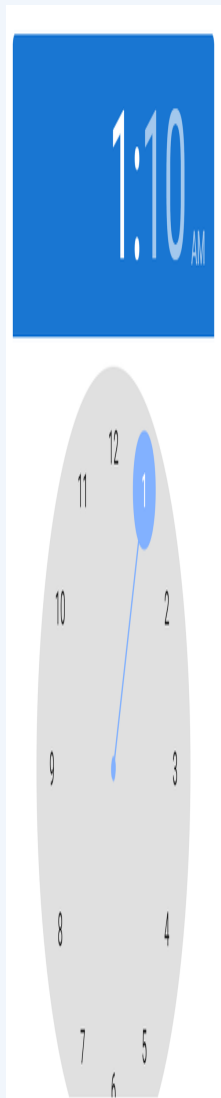
Blocklist Force
type (String,
Integer, Float etc)
Force Range

Describe this

It's so wonderful I can type any...

Slider

How Ripe is it?



Date

Date string
or ISO 8601

Value

Is date



< December 2020 >

S M T W T F S

1 2 3 4 5

6 7 8 9 10 11 12

13 14 15 16 17 18 19

20 21 22 23 24 25 26

27 28 29 30 31

Child Node	Expands another Attribute Group	String eg unique hash, Integer, eg a Integer Primary Key	Reference
------------	---------------------------------	---	-----------

ATTRIBUTE VS ATTRIBUTE GROUP

For ease of speaking I sometimes refer to an Attribute like Color as simple “Attribute.” More technically this is a set of attributes.

Future attributes

At the time of writing this represents a relatively complete list of popular attribute types. However this is a rapidly evolving area and I expect that new types, or expansions of existing types will come.

Technical Example of Attribute Relation to an Instance

For example, a minimal group may be represented in the form of:

```
"instance_list": [
  {
    "type": "box",
    ...
    "attribute_groups": {
      id_of_group : {
        "id": id_of_attribute_selected
        "kind": "select"
      }
    },
    {... another instance }
  }
  ...
]
```

Technical Example of Attribute

This assumes some kind of map of `id_of_group` to information about the group is available, such as:

```
"attribute_groups_reference": [
  {
    "id": id_of_group ,
    "options": [
      {
        "id": id_of_attribute_selected
        "name": "option_name"
      }
    ],
    "prompt": "a prompt written is a prompt earned",
  }
]
```

Where is it? - Spatial Representation

Here I will introduce the common spatial types and concepts. Later we will see how these relate to Machine Learning tasks and some common trades offs between them.

Also known as Localization, or Spatial Locations, Spatial Types, Shapes, Drawing Tools.

Computer Vision Spatial Types

The 3 most popular are Segmentation, Boundingbox, and Full Image. Generally full image tags are not terribly useful for most “modern” use cases because they lack spatial location. While one can get impressive results, often sets used to these cases are artificially constrained. And usually, if you could set up the incoming data in such a way that the “item” of interest already fills the frame, you already perhaps know what it is!

Box

The bounding box is one of the most “tried and true” methods of object detection.

A box is defined by exactly 2 points.

[Illustration of box example]

Storing a box

It requires only 2 pieces of information. For example it can be defined as either as the (top left point x,y) and (width, height), or as an (x_min, y_min) and (x_max, y_max) pair of points.²

Using a Box in Machine Learning:

The internal representation of the location is often quite complex. Eg “State of the art algorithms

enumerate a near-exhaustive list of object locations and classify each into: object or not.”³

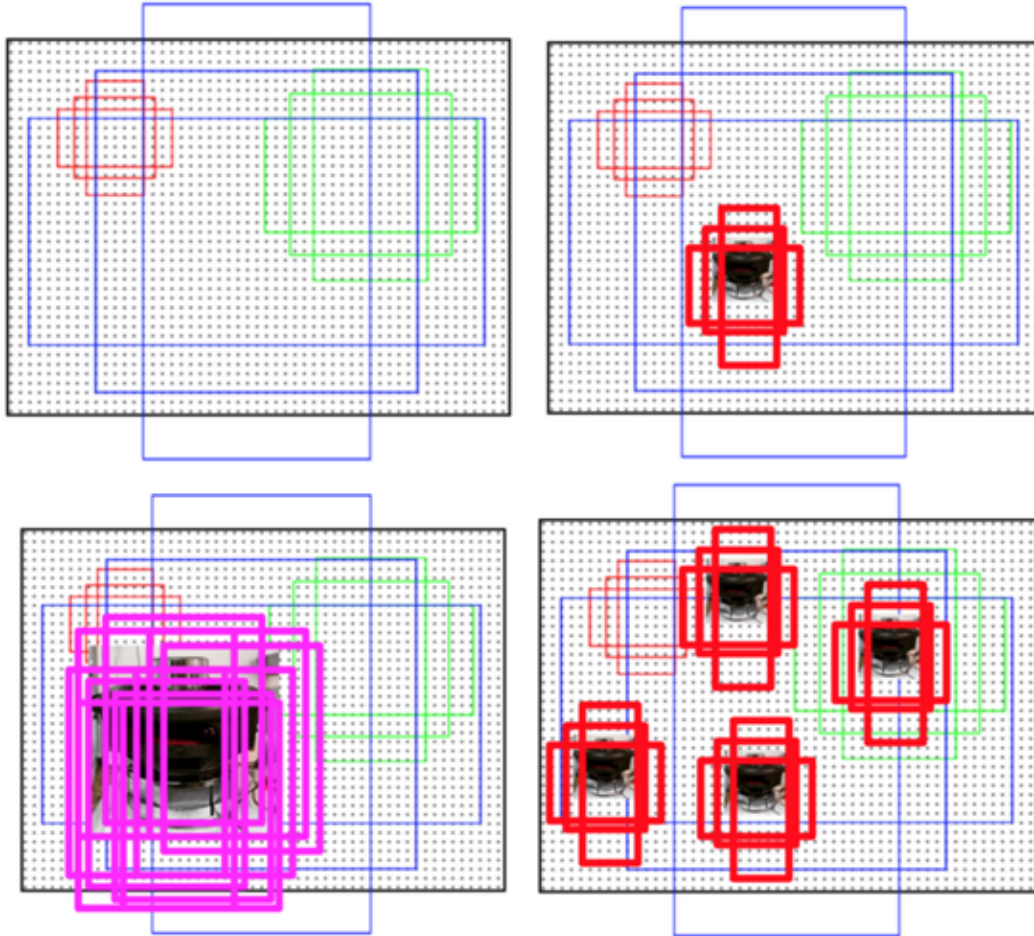


Fig. 4: Schematic illustration of anchors of different scales matching the ground truth objects. Small objects have much less anchors matched. To overcome the problem, we propose to artificially augment the images by copy-pasting small objects, so that there is more anchors positively matched with small objects during training.

For example ExtremeNets process is to
:

1. Build a heatmap
2. Extract peaks
3. Enumerate combinations
4. Compare scores of geometric centers

5. Accept/Reject based on high center score.

5

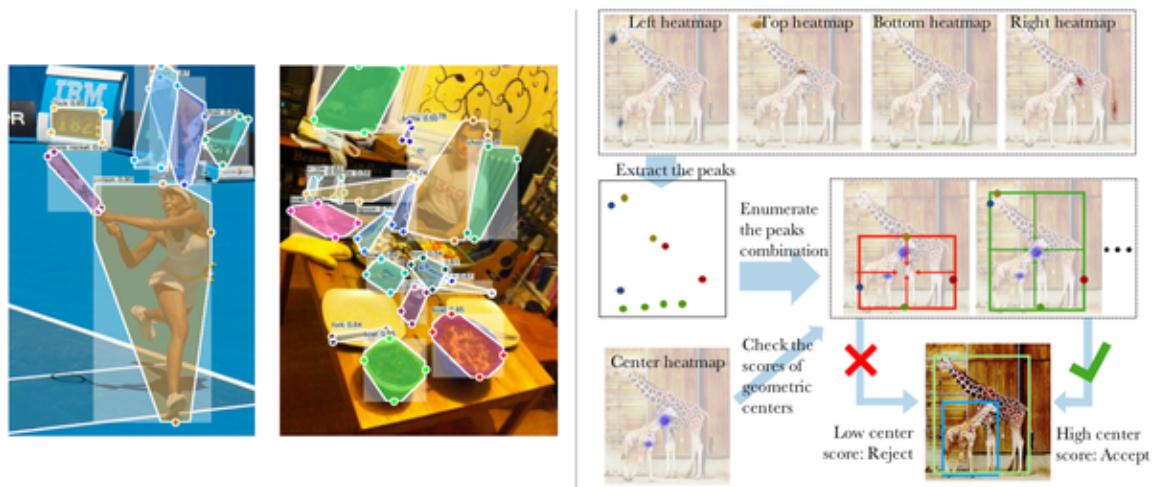


Figure 2-6. Extreme net illustration

Box Variants and uses

Rotated box... (Content TK)

Polygon

It would be quite painful to literally label every pixel. So to get around this various higher level types are used. In this case, a Polygon may be drawn around the border. The Polygon is the Where (Spatial) and the Label “liver” is the What.

A polygon is defined by at least 3 points. There is no limit to how many points a polygon may have. Practically this will often max out at ~1,000 points. Often points are created via an assisted process, such as dragging the mouse or holding the shift key.

Continuing the segmentation example above, in the context of an image, “per pixel” labels can be used, here each green pixel is predicted as “liver” on this CT scan:

An example of spatial locations is “per pixel” or more commonly “pixelwise”.

Is the “highest” level of 2D localization.

Keypoint

(Content TK.)

Ellipse and Circle

Is defined by a center point and a radius (x,y) . These are used to represent circle and oval objects. It functions similar to a box. It can also be rotated.



Cuboid

A cuboid has two “faces”. This is a projection of a 3D cuboid onto 2D space. Each face is essentially a “box.”



Lines & Curves

(Content TK)

Straight line

2 points... (Content TK)

Quadratic curves

- 2 points and a control point
- Tags
- Others and derivatives of those.

Types with multiple uses

Some types have multiple uses. A polygon can be easily used as a box by finding the extreme most points. There is not exactly a “hierarchy.”

Complex Spatial Types

A Complex type refers to a spatial location defined by a set of multiple “primitive” types, such as Box and Polygon as defined above. This is similar, but different, to the concept called “Multi-Modal.” Generally Multi-Modal refers to multiple items of raw data.

A common use case for complex types is complex polygons. Such as a polygon that is partially defined by segments of quadratic curves and partially by segments of “straight” points.

Trade offs with types for architecture and creation

1. Relevancy to Concept. In general the type must “match” the concept to some extent. For example, forcing an octagon shape for a car is unlikely to yield a benefit.
2. Annotation time. Of course the more complex the more time. But often it’s in a step function type fashion, eg segmentation may take 10x longer to annotate than a box.

Trade offs with types for usage

- Compute cost
- Research completeness
- Complexity of output

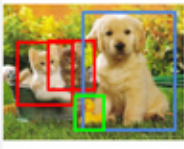
	Classification	Object Detection	Instance Segmentation
	 CAT	 CAT, DOG, DUCK	 CAT, DOG, DUCK
Compute Cost	🔥	🔥🔥	🔥🔥🔥🔥
Annotation Effort	✍️	✍️✍️	✍️✍️✍️✍️✍️
Location	✗	✓	✓✓
Research completeness	📖📖📖	📖📖	📖
Complexity of output	💎	💎💎	💎💎💎💎

Figure 2-7. Designing a visual deep learning system

When is it? - Relationships, Sequences, Time Series

What follows is a discussion and expansion on relationships and sequences.

Sequences and Relationships

Many of the most interesting cases have some form of relationship between Instances.

Imagine a football game. A soccer ball is touched by a player. This could be considered an “event”. At a given frame, eg frame 5. (TODO introduce this more gradually)

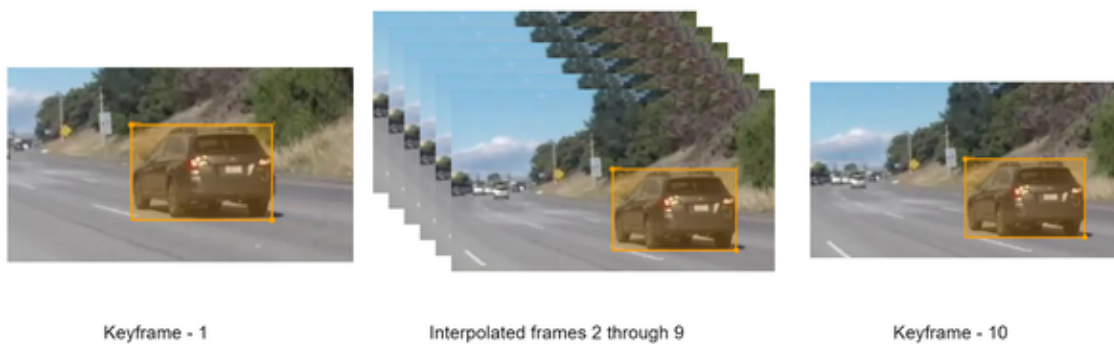
In this case we have an Instance of say Ball and Player.

What matters is that both of those instances are in the same Frame.

When

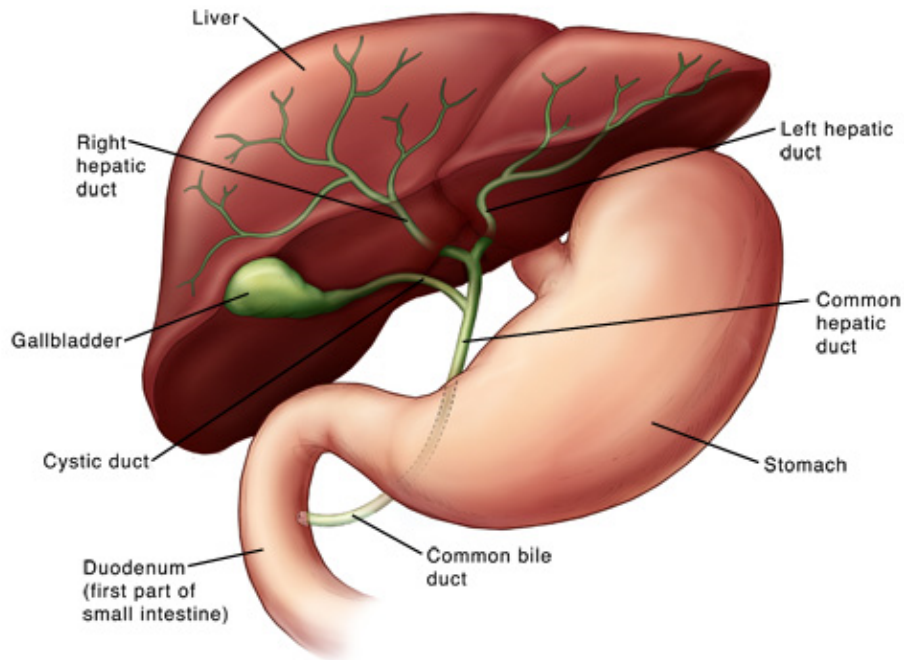
Another big concept at play here is Relationships. Consider upgrading our still image example to a video context. From the human perspective we know the “liver” in frame 0 is the same organ as the one in frame 5, 10 etc. It’s a persistent object in our minds. This needs to be represented in some form, often called a Sequence or Series. Or it can be used to “ReID” an object in a more global context, such as the same person appearing in different contexts.

[Visual: Not exactly this, but something showing the same object. (this text has interpolation which is separate concept)]



Guides, Instructions

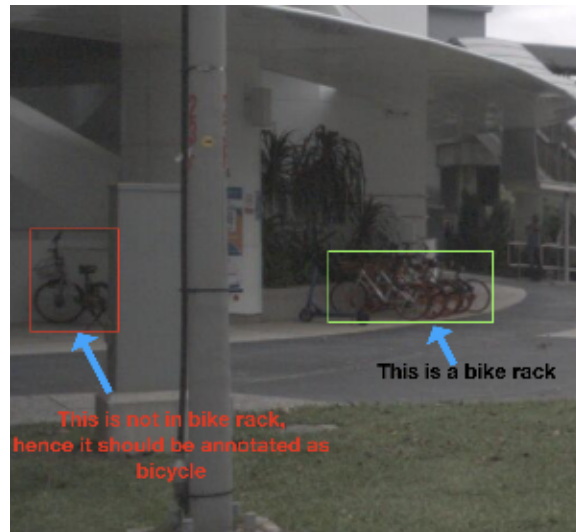
Of course, how do we know to label it “liver” at all? I sure wouldn’t be able to figure that out with some form of guide. What about sections of the liver that are occluded by the Gallbladder or stomach?



In a Training Data context we formalize the concept of a Guide. In a strict sense, the Guide should be maintained with as much respect as the actual training data, because the “meaning” of the training data is defined by the Guide. In a sense Guides address the “How” and “Why”.

We have covered some of the high level mechanics of representing Training Data. While complex scenarios and constraints can be a challenge, often the “real” challenge is in defining useful instructions. For example the NuScenes data set, has about a paragraph of text, bullet points, and 5+ examples for each top level class.

For example, here showing the difference between a “bike rack” and a “bicycle”.



Judgement calls

To see how some of this can quickly get dicy, consider “drivable surface” vs “debris”. NuScenes defines it as “Debris or movable object that is left on the driveable surface that is too large to be driven over safely, e.g tree branch, full trash bag etc.”⁷

Of course what’s safe to drive over in a semi-trailer vs a car is different. And this isn’t even getting into choice semantics like “should you drive over debris to avoid rear ending someone”.

Choosing good names

There’s a classic joke about there being 2 hard things in computer science, and naming being one of them. In Chapter 3 we cover good bad training data concepts in more detail, but for now let’s just consider the “architecture” of it.

We will introduce the key concepts and then dive deeper into the technical specifics of what that means.

Historically Vision and natural language processing (NLP) were treated as very different tasks. In the new deep learning context there is much more overlap.

Going back to our cats example, it’s tempting to thinking of training data as straight forward - especially since in early history it was generally an outsourced task. Yet, the complexity is expanding rapidly.

Relation of Machine Learning Tasks to Training Data

Training Data is used in a Machine Learning system. Therefore, it's natural to want to understand what ML Tasks are common and how they relate to Training Data.

Tasks

There is general community consensus on some of these groupings of tasks. There are many other resources that provide a deeper look at these tasks from the machine learning perspective. Here I will provide a brief introduction to each task from the Training Data perspective.

Semantic Segmentation



Every pixel is assigned a Label.

With Training Data, This can be done through “vector” methods (e.g. polygons) or “raster” methods (picture a paint brush). At the time of writing the trend seems to favor vector methods but it is still an open question. From a technical perspective vector is much more space efficient. Keep in mind that the user interface representation here may differ from how it's stored. For example a UI may present a “bucket” type cursor to select a region, but still represent the region as a vector.

Another option question at the time of writing is how to actually use this data. Some new approaches predict polygon points, whereas the “classic” approach here is per pixel. If a polygon is used for training data and the ML approach is classic, then the polygon must go through a process to be converted into a “dense” mask. (This is just another way of saying a class for each pixel.) The inverse is also true if a model predicts a dense mask but the UI requires polygons for a user to more easily edit.

While there are many great resources available, one of the most prolific is the site Papers With Code lists over 885 computer vision tasks, 312 NLP tasks, and 111 in other categories.⁸

Let’s walk through an example of a Medical use case. If one has a goal of doing Automatic Tumor Segmentation (eg of CT scans), Segmentation Training Data is needed.⁹ Researchers routinely note the importance of training data. “The success of semantic segmentation of medical images is contingent on the availability of high-quality labeled medical image data.”¹⁰

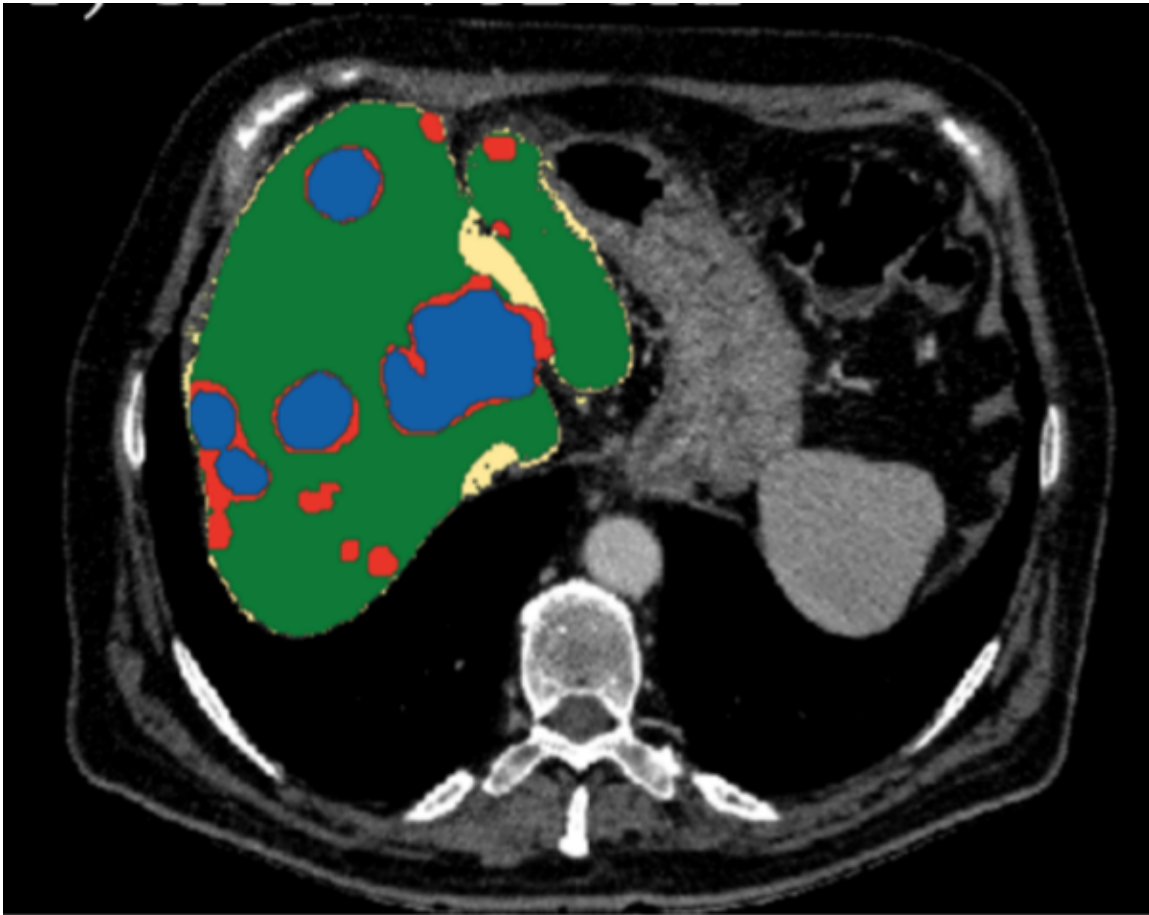


Figure 2-8. Example of Tumor Segmentation a sub category of Segmentation.

Training Data is created in conjunction with the Task goal, in this case, automatic tumor segmentation. In the above example, each pixel corresponds to a classification, this is known as “pixel-wise”. Each green pixel is predicted as “liver” on this CT scan.

Image Classification (Tags)

Whole image classification. An image may have many tags. While this is the most generic, consider that all the other methods are essentially built on top of this. From an annotation perspective this is one of the most straightforward.

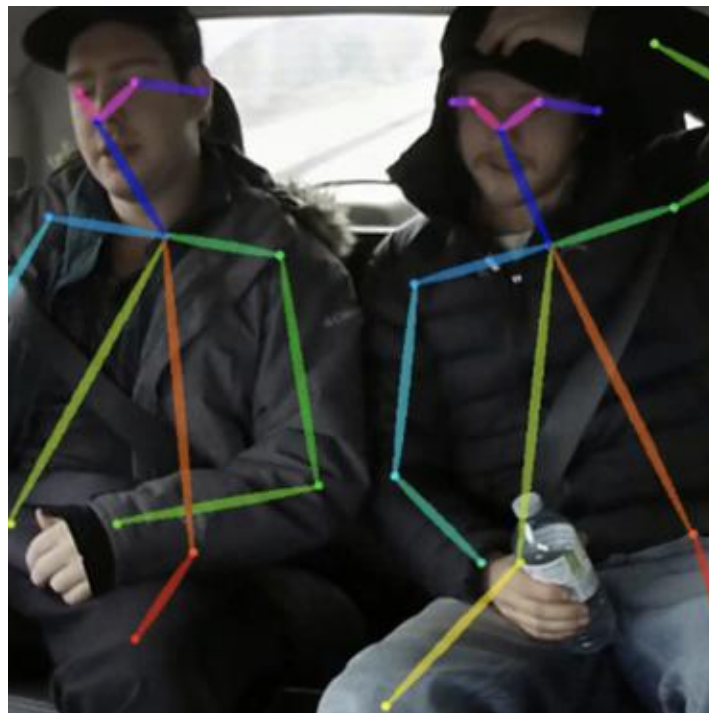
Object Detection

Detect spatial location of multiple objects and classify them. This is the classic “box drawing”.

At the time of writing, this is in close competition with semantic segmentation for “best bang for your buck” awareness. It’s generally the fastest annotation method and provides the big leap of getting spatial location. It’s a great “default choice” if you aren’t quite sure where to start.

While most of the research has centered around “boxes”, there is no requirement to do this. There can be many other shapes like ellipses etc.

Pose Estimation



At a high level this is “complex object detection”. Instead of a general shape like a box, we try to get “keypoints”. Typically there’s a relationship between the points. Such that say the left eye is within some bounds of the right eye etc.

From a training data perspective, this is handled via the Keypoint template. For example, creating an 18 point representation of a human skeleton to indicate pose and orientation. This is different from segmentation/polygons because instead of drawing the outline, we are actually drawing “inside” the shape.

Chart - Relationship of Tasks to Training Data Types

Spatial Type	Tasks
Polygon, Brush	Segmentation Object Detection ^a
Box	Object Detection
Cuboid	Object Detection 3D Projection Pose Estimation
Tag	
Keypoints	Pose Estimation

^a Well technically it can be used for object detection, usually that's a secondary goal because it's faster to do a box for the generic detection base.

All of these methods generally do some form of “Classification” too.

General Concepts

These concepts apply to, or interact with, both Spatial and What

Instance Concept Refresher

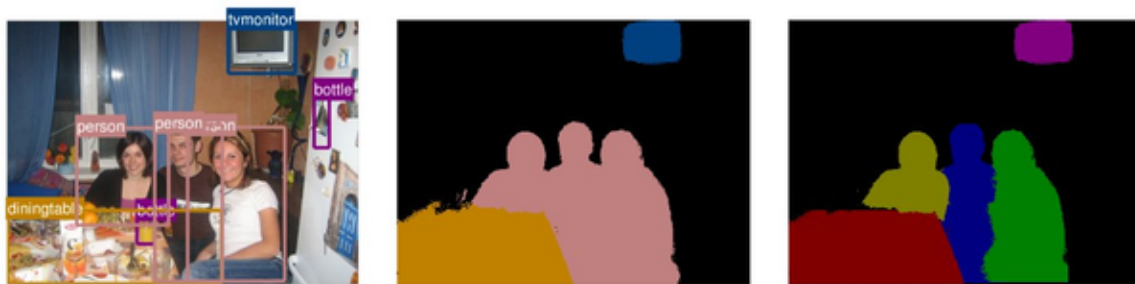
Virtually everything discussed here relates to an Instance. An Instance represents a single example of an annotation. For example, here each person is

an Instance.

An Instance has references to Labels and Attributes, and may also store concrete values, such as free text that's specific to that Instance. In a video or multiple frame context, an instance uses an additional ID to relate to other Instances in different frames. Each frame still has a unique Instance, because the data may be different, for example a person may be standing in one frame and sitting in another, but they are the same person.

To illustrate a subtle difference, consider that here all 3 people have the same "class", but are different instances. If we didn't have this Instance concept then we would get a result like the middle image.

12



Consider that for every Instance, there can be n number of Attributes, which may in turn have n number of children, and the children may have arbitrary types/constraints (such as selection, free text, slider, date) of their own? Yes - in fact each instance is almost like a mini graph of information. And if that wasn't enough, the spatial location may actually be 3D. And there may be a series of frames.

Modern software tools handle the relationships between these concepts well. The challenge is the data supervisor must, at least to some degree, understand the goal, so as to do a reasonable job.

Further, the types must align with the use case in the neural network. Common network architectures have assumptions to be aware of when constructing training data.

Upgrading data over time

In some cases it can be quite reasonable to “upgrade” data over time. For example a common pattern:

1. Run a “weak” classifier to tag images, purely with intent of identifying “good” training data.
2. Humans create bounding boxes.
3. At a later point in time, a specific class is identified as needing segmentation. The bounding box can then be used for algorithm designed to “generate” segmentation data once a rough localization has been done (the existing bounding box).

Advanced concepts

Let’s expand on a few points here.

Boundary between Modeling and Training Data

The general point here is that there is always some degree of disconnect between the representation of the training data and how the model ‘actually’ uses it:

- The classic example that the model uses integers while humans see string labels
- The ‘where vs what’ problem
- Model ‘sees’ a dense pixel mask (for segmentation) but humans rarely if ever consider single pixels. Often segmentation masks are actually drawn through polygon tools
- The transformation of human relevant Attributes to use in the model (eg flattening example).
- Localization, for example, the human may draw the box using 2 points, while the machine may predict from a center point¹³, and other methods, that really don’t have anything to do with those 2 points drawn¹⁴.

Basically what this means is:

- There's little value with, in relation to the planned model, in being overly precise with how the training data is defined here. **I want to update this point
- It is important to capture as many assumptions about how the training set was constructed as possible. This is especially vital if any type of Assist Methods is used to construct the data.
- Ideally, whomever manages the training set, should have some idea of how the model actually uses the data. And whoever manages the model, should have some idea of how the set is managed.

Sometimes - these distinctions can be trivial. For example, in a single label case.

Raw Data Concepts

Raw data means the literal bytes being supervised. It's generally considered to be "immutable" in that the data is what it is - whereas we can choose a different schema, or choose how to label it etc the data is what it is.

Images

Photographs, PDFs, Scans, Screenshots, etc. Anything that can be represented as an "image" can be used.

Raw Data Constraints

Content TK.

Resolution constraints

- Most NLP models have severe constraints on the "length" of concepts
- Most vision models have severe resolution constraints, with the 'defaults' of popular methods common evaluating images at low (eg 300x300 resolutions). While there are workarounds (todo list them),

this is often glossed over! A human may see at 3840x2160 (8,294,400 pixels) (4k) resolution image, and be zooming into it, whereas in the naive case, the model may only see 300x300 (90k pixels)! 10 Orders of Magnitude Difference!

- Further, many benchmarks are done at these relatively low resolutions. While higher resolution benchmarks will surely come in the future, it's worth being aware that many research concepts have really only been vetted at these lower resolutions.
- And circling back to the point of humans having access to different level of detail here.

Video

- Preprocessing...
- Frame/Time
- Uses...
- Sequences...

(Content TK)

3D

(Content TK)

3D Point Clouds

Point clouds are scary right? Well, yes and no. First, point clouds often contain less data than images. Yes less. This is possible because while a point cloud stores a triplets for coordinates (x, y, z), there are often only “millions” of these points. Consider that a single 4k image contains 8,294,400 RGB (Red Green Blue) triplets.

Generally the “hard part” with this is often the projections to 2d.

TODO double check about reflectivity point

Text

(Content TK)

Raw Data Combinations

Sometimes examples can be combined (or split) in unique ways. For example, 3 images can be trained on as a single image. In theory this is to convey a relationship between those images. Conversely a single image can be split and trained on. “Sliding window” approach.

At the time of writing these methods are generally either fairly niche or part of a pre/post processing concern. This type of editing is usually done at the algorithm / code level and is not a primary concern of Training Data.

Multimodal Data

- Multiple “overlays” of same thing
- Which sensor type was it annotated in? Does it apply to both?

Transformations - What view is the data being annotated in? Where is it getting predicted on?

(Content TK.)

2d to 3d to 2d

High level thing here is that the data may be labelled in different dimensional space then what it gets trained on. For example, it may be labelled on a 2d image, and trained in 3d space, or vice versa.

Summary

Next we will move from this high level viewpoint to that of a data specialist. How do we literally supervise and annotate this?

- 1 <https://dl.acm.org/doi/pdf/10.5555/77708> Pg. 43 Actual Pg.68 in PDF “In subsequent papers (e.g., Codd 1971a, 1971b, and 1974a), I realized the need to make this distinction, and introduced domains as declared data types, and attributes (now often called columns) as declared specific uses of domains.”
- 2 <https://medium.com/diffgram/how-do-i-design-a-visual-deep-learning-system-in-2019-8597aaa35d03>
- 3 <https://arxiv.org/abs/1901.08043>
- 4 Kisantal et al <https://arxiv.org/pdf/1902.07296v1.pdf>
- 5 <https://github.com/xingyizhou/ExtremeNet>
- 6 (own work) <https://medium.com/diffgram/how-do-i-design-a-visual-deep-learning-system-in-2019-8597aaa35d03>
- 7 https://github.com/nutonomy/nuscenes-devkit/blob/master/docs/instructions_nuscenes.md
- 8 <https://paperswithcode.com/sota> Accessed Oct 4 2020
- 9 arXiv:1702.05970 [cs.CV] Automatic Liver and Tumor Segmentation of CT and MRI Volumes using Cascaded Fully Convolutional Neural Networks - Patrick Ferdinand Christ et al
- 10 <https://arxiv.org/pdf/1902.09063v1.pdf>
- 11 arXiv:1702.05970 figure 7
- 12 <https://davidstutz.de/bottom-instance-segmentation-using-deep-higher-order-crfs-arnab-torr/>
- 13 TODO I think there's a specific center point paper we can reference
- 14 TODO be more specific, I'm thinking of SSD and how some of the regression points work here

Chapter 3. Annotation Literal Concepts

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 3rd chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at jleonard@oreilly.com.

Now that you know all about the high-level concerns of Training Data, how do we start to put some of these concepts into practice?

In this chapter we start to dive into the heart of actually getting data into your system, configuring it, and getting data out to your models. And I’ll cover the nitty gritty of actually doing annotations. Let’s dive in!

First a quick refresher on Schema. Schema may be as simple as a set of labels, like “valid”, “invalid”, “unsure”, or as complex as many nested attributes and relations. More generally, it’s a paradigm for encoding Who, What, Where, How & Why. A representation of meaning structured by Labels, Attributes, and their Relation to each other.

Chapter Organization: Administrators and Annotators

This chapter is made of two parts. The first is focused on the Administrator's view and the second for the Annotator. Like any software system, often there is a need for at least one person to set up the system, while the majority of users are focused on purely using it. For example, an email system administrator vs an email user.

It's absolutely valuable for Administrators to know what an Annotator's day to day usage looks like. An annotator looking to lead a team, move into an administrator role, or simply curious about the bigger picture can benefit from the Administrator section.

Now you may be wondering, "I'm a (data scientist, subject matter expert, etc.), does that make me an administrator?"

Interestingly enough, increasingly an Administrator may be neither a data scientist nor a data engineer. Often an Administrator may simply be a person who's been charged with overseeing a system - a new role. As an analogy, an "average manager" is probably not in charge of the IT systems for an email system. However, the manager may be the administrator who logs into the web UI and adds or removes users.

This is in part because as the supporting systems have been improved and sometimes the need for software knowledge has become less. This is similar to how we used to think of operating a computer as a special skill, and now we generally assume someone knows the basics of operating a computer that we don't even mention.

Why does this distinction matter so much? Well, unlike something like email where the manager can make the call without involving the user, in this case, the users (subject matter experts, annotators) often have a lot more value to add in how it's structured.

Partnering with non-software users in new ways

As mentioned earlier, annotating data is like writing code. In this context administrators face some big challenges.

If, as an Admin, you are also a subject matter expert, then you may be able to create and update the Schema mostly on your own. However, there may be a gap in understanding how to match the Schema to a desired data science modeling need, requiring more interfacing with the data science.

Conversely, if you are a data scientist, you may need to lean heavily on your subject matter experts to build an accurate Schema.

Either way - you will be partnering with your annotators to jointly create a codebase. In this chapter I will cover the “how to” of the process in depth. Starting with the Admins point of view, and shifting to some specifics for Annotators.

Administrators Process Overview

Here I will introduce the standard process. There are many additional processes such as prelabel, that I will introduce later. In the earlier high level processes we had two boxes: Capture Data and Create Tasks. Here I assume that the raw data has already been captured somewhere and is available to use. In practice this could be a cloud storage bucket or a local storage drive. The following is a detailed look at the “Create Tasks” box:

Create a Training Dataset Process

1. Import and prep data into an annotation system.
2. Define your Schema - what you want to label.
3. Create tasks for your annotators.
4. Your annotators view the images and do the annotation.

5. Export data to load into your machine learning model training system.

The output of the 5 steps is the Training Dataset.

Introduction to Annotation Tools

For any task there are often software tools that can help. For word processing there's Word, Google Docs, etc. There is a new batch of software tools, such as Diffgram, designed specifically for the needs of Training Data. These tools vary in complexity. Some act as entire platforms, merging a suite of distinct tools into one integrated application. There are others focused on narrow niches. I often use the words “tool” and “system” somewhat loosely here. We will cover Annotation Tools in more depth in Chapter 4. The main point is that we assume there is some kind of software system that will facilitate the literal annotation, and the goal for now is to get the data to it!

Importing Data & Data Prep

At a high level the goal is to get the raw data, such as images, video, and text, into usable form for Annotation. From the admin view, the import of data is often one of the biggest hold ups in getting projects started.

Wizards

Recently new technology like “Import Wizards” - step by step forms - have come up that help make some of the data import process easier. While I fully expect these processes to continue to become even easier over time, the more you know about the behind the scenes aspects, the more you understand how these new wonderful wizards are actually working.

TK: much more detail on Wizards

Bucket

A remote technology service that stores raw data such as images and video. Also called: Object Store

Binary Large Object - BLOB

Raw data such as images, video, text. Also known as: Object

Organizing Data and Useful Storage

One of the first challenges is often how to organize the data you have already captured (or will capture). One reason this is more challenging than it may at first appear is often these raw datasets are stored remotely. At the time of writing cloud data storage browsers are generally less mature than local file browsers. So even the most simple operations, for example myself sitting at a screen and dragging files, can take on a new challenge.

Some practical prescriptions here:

1. Try to get the data into your annotation tool sooner in the process than later. For example, at the same time new data comes in, if I write the data to the annotation tool at a similar time I'm writing to a generic object store I can "automatically" organize it to a degree, and or more smoothly enlist team members to help with organization level tasks.
2. Consider using tools, such as open source Lightly, that attempt to automatically surface the "most interesting" data. This is an emerging area - but it's already clear that these methods, while not without their challenges, have merit and appear to be getting better.

Remote Data Storage

Generally, the data is always stored remotely - relative to the end user - because of the size of the data, security requirements, automation (eg connecting from an integrated program, practicalities of running model inference, aggregating data from nodes/system), etc. And teammates, the person who administers the training data, may not be the person who

collected the data (consider use cases in medical, military, on-site construction etc).

This is relevant even for solutions with no external internet connection, also commonly referred to as “air gapped” secret level solutions. In these scenarios, it’s still likely the physical system that houses the data will be a different box then the end user even if sitting 2 feet from each other.

The implication of this is we need to somehow access the data. At the very least by whomever is annotating the data, and most likely also by some kind of data prep process.

Streaming point and automation definitions expanded

One thing to keep top of mind throughout this is that we aren’t really wanting to statically select folders and files. We are really setting up a process in which we stream new data, in an event driven way. To do this, we need to think of it more like assembling a pipeline, then the mechanics of getting a single existing known set.

Access Control

Two of the big questions that can help us differentiate this here is:

- What system is processing the data? What are the Identify Access Management (IAM) permission concerns around system level processing and storage?
- What user access concerns are there?

Cloud connections & Signed URLs

Ultimately whomever is going to supervise the data needs to view it. This is the minimum level of access and essentially unavoidable. Prep systems, such as the system that removes Personally Identifiable Information (PII), generates thumbnails, pre-labels, etc. also needs to see it. Also - for practical system to system communication it’s often easier to transmit only a URL / file path and then have the system directly download the data. This

is especially true because many end user systems have much slower upload rates than download rates. For example imagine saying “use the 48 GB video at this path” (KBs of data) vs trying to transmit 48 GB from your home machine.

There are many ways to achieve this but signed URLs - a per resource password system - are currently the commonly accepted method. They can be “behind the scenes” - but generally always end up being used in some form.

For both good and bad reasons this can sometimes be an area of controversy. I’ll highlight some trade-offs here to help you decide what’s relevant for your system.

SignedURLs

A URL that contains both the location of a resource (like an image) and an integrated password. Similar to a “share this link” in Google Docs. Signed URLs may also contain other information and commonly time decay, meaning the password expires. For example, a signed URL has the general form of:

sample.com/123/?password=secure_password

(Actual signed URLs are usually quite long, about the length of this paragraph or more.)

The reason that we make a bit of a distinction here and get so specific about IAM is because training data presents some unusual data processing concerns:

1. Humans see the “raw” data in ways that are uncommon in other systems
2. Admins usually need fairly sweeping permissions to work with data - again in ways uncommon in classic systems
3. There are data size and processing concerns that while perhaps not really different from classically, are new and there are less

established norms for what's reasonable.

Manual Creation of Signed URLs

One of the most secure ways is to have your own system do all of the data - that for the sake of this discussion is assumed to be perfectly secure - and magically generate a signed URL exclusively for the annotator - that decays almost immediately after successfully downloading it.

The main benefit of this of course is security and control. The downside is it's extra work to do, is difficult to implement well - for example it can require further unusually tight integration with Identity Access Management (IAM) systems.

In practice, this is really not recommended because essentially it either severely restricts the functionality, or you end up recreating the wheel by essentially rebuilding the training data tooling functions (like data prep, previews etc) in your own application logic.

One of the benefits of Training Data tooling is organizing files at the Training Data level of abstraction. While this is technically possible with the "raw" file access separate, it's asking for trouble because the tool cannot fully verify the authenticity of the data. If the tool can access the data it can verify the video loads, the image exists, etc.

As a rough analogy, think of storing part of your records on the computer and part in a filing cabinet. Sure - the filing cabinet is air gapped from the system, but it also means those records aren't part of the system - they aren't searchable, they can't be analyzed directly by the system etc.

Off the shelf dedicated Training Data tooling: Your own hardware

In this context, we trust the training data tooling to manage the signed URL process. We trust the Training Data tooling to handle the IAM concerns. With this, assuming we trust the tooling (perhaps because we can inspect the source code), the only real concern is what bucket it uses - and that generally becomes a one time concern, because the tool manages the IAM.

For advanced cases the tool can still link up with a single sign on (SSO) or more complex IAM scheme.

The tooling doesn't have to run on the hardware. The next level up is to trust the training data tooling and trust the service provider to host/process the data. Although at that point most of this discussion is less relevant because really just following instructions, however, it's the least level of control.

Where does the data rest?

Generally speaking, any form of tooling will generate some form of data that is additional to the “original” data.

This means if I have data in bucket A and I use a tool to process it there either be additional data in bucket A or I will need a new bucket B to be used by the tool. This is true for Diffgram, Sagemaker, and as far as I'm aware most major tools.

Depending on your cost and performance goals this may be a critical issue or of least concern. On a practical level for most use cases:

1. Expect that additional data will be generated.
2. Know where the data is being stored - but don't overthink it.

In the same way we don't really question how much storage say a Postgres Write Ahead Log (WAL) generates, my personal opinion is that it's best to trust the training data tool in this regard. If there's an issue - address it within the training data tools realm of abstraction.

Bucket connection

Saying all this above - the best abstraction is to create a connection from the training data tooling to the bucket. There are various opinions on how to do this and the specifics vary by hardware and cloud provider.

[insert example from diffgram UI of file browser]

[insert example of solution schema with training data tool and bucket]

For the non-technical user, this is essentially “logging in” to a bucket. In other words, I create a new bucket A and it gets a user id and password (client ID / client secret) from the IAM system. I pass those credentials to the Training Data tool and it stores them securely. Then when needed the Training Data tool uses those credentials to interact with the bucket.

Physical Data Prep

Technical specifics relative to the type of data and use case.

Personally Identifiable Information (PII)

Option 1: PII Removal

For images, for example, this may involve blurring faces and identifying marks such as house numbers. This will vary dramatically based on your local laws and use case.

Option 2: PII Identification and Training

The dataset contains PII and cannot or will not be changed. Perhaps the PII is desired or useful for training. This requires having PII complaint data chain, PII training for staff, and generally appropriate tagging identifying the elements as containing PII.

Images

Images usually don’t require any technical data prep because they are individually small enough files.

Video

It's common to split video files into smaller clips for easier annotation and processing. For example, splitting a 10 minute video into 60 second clips.

One approach for dealing with video is to sample frames. This can be done by reducing the frame rate and extracting frames. For example converting a 30 frames per second (FPS) to 5 or 10 and then extracting frames. The benefit of this is reduced processing overhead. The downside is that it makes it more difficult to annotate or use other relevant video features like interpolation or tracking.

Usually it's best to keep the video as a playable video and also extract all the frames required. This improves the end user annotation experience and maximizes annotation ability. Event-focused analytics need the exact frame when something happens which effectively becomes lost if many frames are removed.

Counterintuitively, keeping all the frames often makes it faster to annotate and delivers higher quality results. The full data is available to be sampled by "find interesting highlights" algorithms. This leads Annotators to see more "interesting" things happening and higher quality data. Object tracking and interpolation further drives this point home, an annotator may only need to label a handful of frames and often get many back "for free" through those algorithms. And while nearby frames are generally similar in practice it often still helps to have the extra data.

An exception to this is that sometimes very high FPS video (e.g. 240-480+) may still need to be sampled down to 120 FPS or similar. Note that just because many frames are available to be annotated, we can still choose to only train models on completed videos, completed frames, etc.

One way that modern tooling has made this easier is with end user selected prep options and maintaining the frame relationship with a global reference frame.

Global Reference Frame for Transformed Videos: What Frame is it? -
Splitting the video into multiple parts leads to the problem, if we have a

single video split into 10 parts, what does frame 5 in clip 8 mean in relation to the original video? There are two ways around this:

1. Use a `global_frame_number`. The frame in the context of the original video. This is defined as:

```
global_frame_number = (Starting offset in  
seconds * original FPS) + ( frame_number *  
FPS conversion rate)
```

2. Use the provided split clips in your ongoing work. Since the clip frame number corresponds to the exact frame being annotated all transformations are already “included”. (This assumes your tool provides this clip).

Pre-Label Prep

It’s common to take existing model predictions and use them as the starting point for annotation. We will discuss the pros and cons of pre-labeling in later chapters. Here is a brief introduction to the technical specifics of adding the data.

The big idea with pre-label is that we take the output from an already run model.

Upfront vs Updating existing

If this is the first time you are importing the raw data, then it’s often best to attach the existing instances at the same time as the raw data. If the data is already in the system, then you will need to refer to the file id, or some other form identification such as the filename to match with the existing file. While this may seem unneeded for small examples, for large volumes of images, frequent updates, video etc, it’s much faster to update an existing known record (or copy of that record) then to re-import and re-process the raw data.

Pre-Label Gotchas

Video formats, especially sequences, can be a little difficult to wrap the head around. This is especially true if you have a complex sequence schema. For these, I suggest making sure the process works with an image, and/or the process works with a single default sequence before trying true multiple sequences.

Many models output predictions in relative coordinates, but annotation tools may expect absolute coordinates. See the section (link?) on this transformation

It can be hard to manage file IDs for updates. Generally it's best to always try and include the file ID from the annotation tool along with the data the model trains on. This ID is generally more reliable than a filename because filenames are often only unique within a directory.

Many predictions methods generate multiple predictions with some threshold for inclusion. Generally, whatever mechanism you have for filtering this data needs to be applied here too. For example, only taking the highest “confidence” prediction. To this same end, in some cases it can be very beneficial to include this “confidence” value or other “entropy” values to help better filter training data.

Pre-Label data prep process

As shown in Figx 3.1 a pre-label process involves the following steps:

1. Map your Data to your Annotation Tool's format.
2. Attach Data to Annotation Tools's Files at Import
3. Verify Data

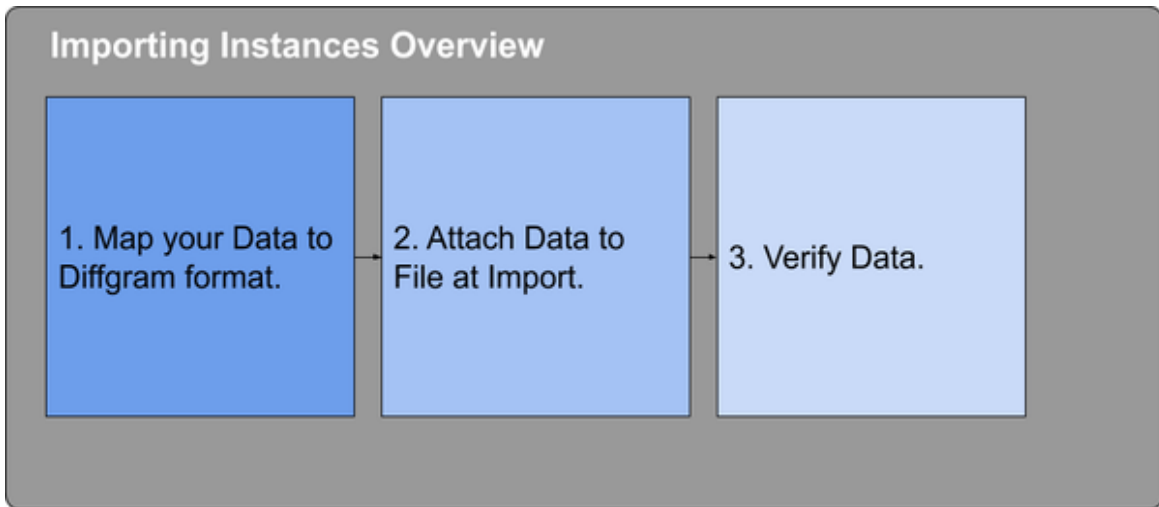


Figure 3-1. Block diagram example

Usually there will be some high level format, such as saying that an image may have many instances associated with it. Or that a video may have many frames, and each frame may have many instances, as shown in Fig 3.2.

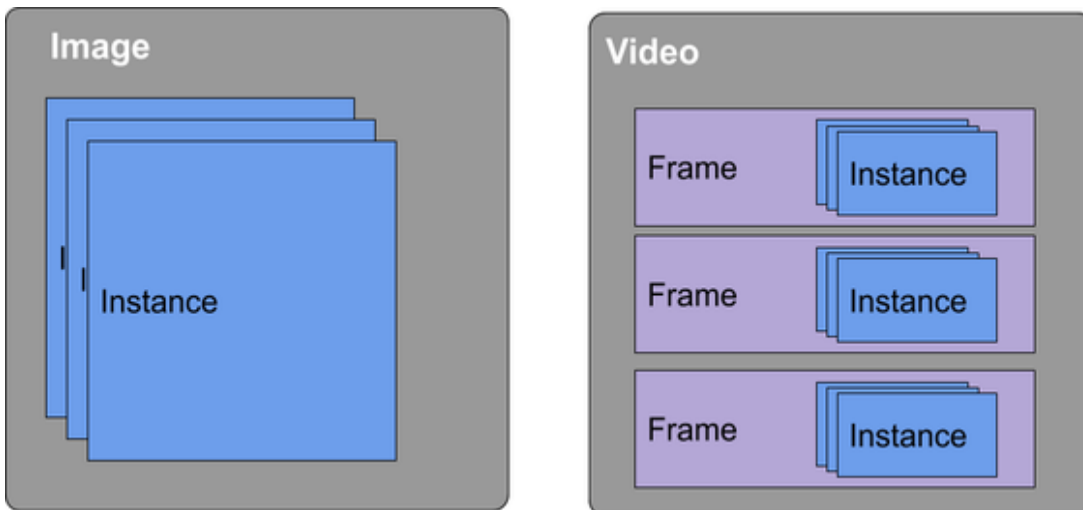


Figure 3-2. Visual overview of relation between raw media and instances

Example python code:

```
def mock_box_from_external_format(
    sequence_number: int = None,
    name : str = None):
    return {
        "name" : name,
        "number": sequence_number,
        "type": "box",
```

```
"x_max": random.randint(500, 800),  
"x_min": random.randint(400, 499),  
"y_max": random.randint(500, 800),  
"y_min": random.randint(400, 499)  
}  
instance = {}  
instance_list = [instance, instance, instance]
```

Summary Of Import

1. There are some data import choices which can generally be left up to Training Data tool to implement
2. Data is stored remotely. This solves many issues but also introduces some access questions
3. It's best to leave the access or IAM within the realm of the training data tooling - even if you control it 100% as a subsystem.
4. It's good to be aware of prelabel considerations prior to running models. For instance, consider how to handle getting the data to the annotation tool and also how to maintain the file ID when loading data.

Leave off notes:

1. Wrapping up / 2nd pass on import section
2. Literal needs for Schema/ tasks (first part was more higher level conceptional)
3. Keep going with process for admins

Define your Schema - what you want to label.

Earlier we covered a lot of the specifics of what Schema and labels are. But how do you actually define it?

Creating labels - Spatial location relation

One of the first choices for label creation is if you want to map a spatial type to a specific label. This can be a highly opinionated area. Essentially the three choices are:

1. Leave it up to the annotators
2. Define it at the “batch” level - eg this batch will be all boxes etc.
3. Attach a spatial location to a specific label (“person maps to polygon”). This can be defined as a default value, or an allow / block list.

Example of Schema definition page:

Label Templates   	
Name	Actions
 test	 
 person	 
 vehicle	 
 rain	 
 snow	 

Figure 3-3. Label templates example

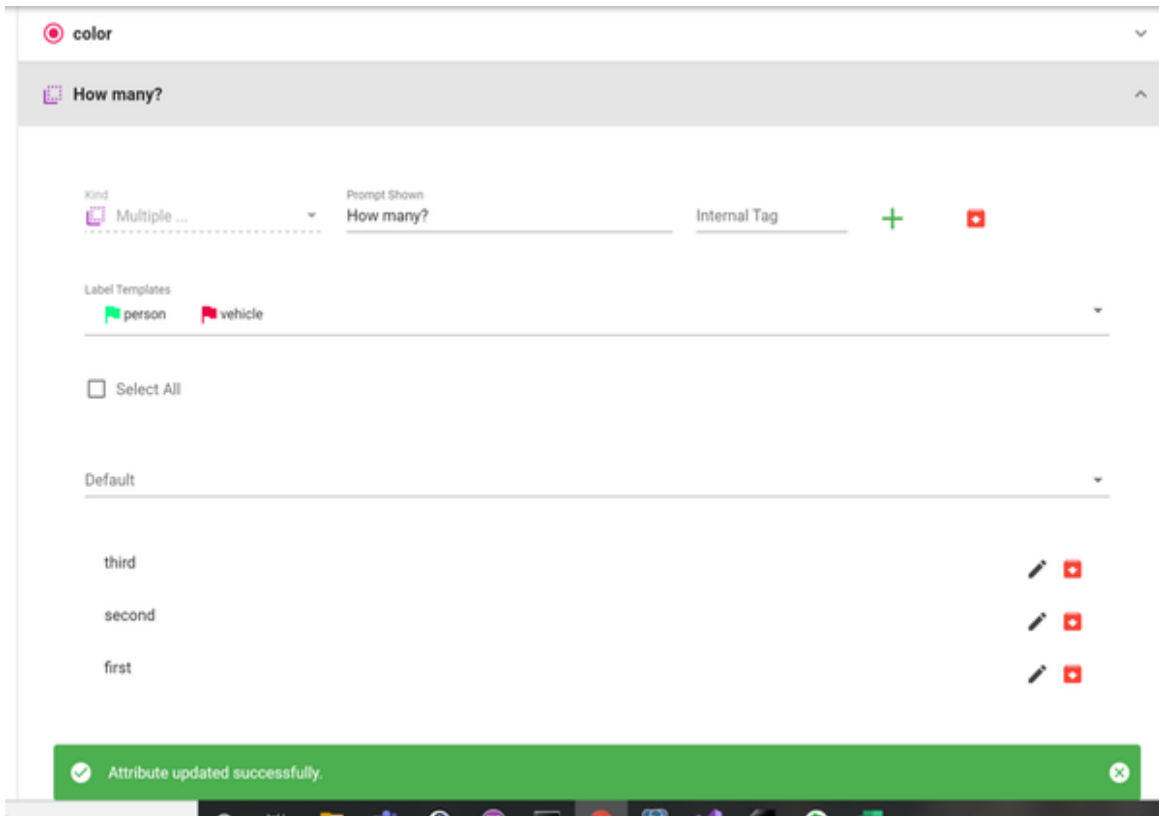


Figure 3-4. Example schema definition page

Keypoint Geometry

One of the spatial types that's not predefined is the “Keypoint” spatial type. In this case you can use an interface to construct the spatial template you desire. Keypoint can be used for pose estimation, known geometry that doesn't conform to typical structures, etc.

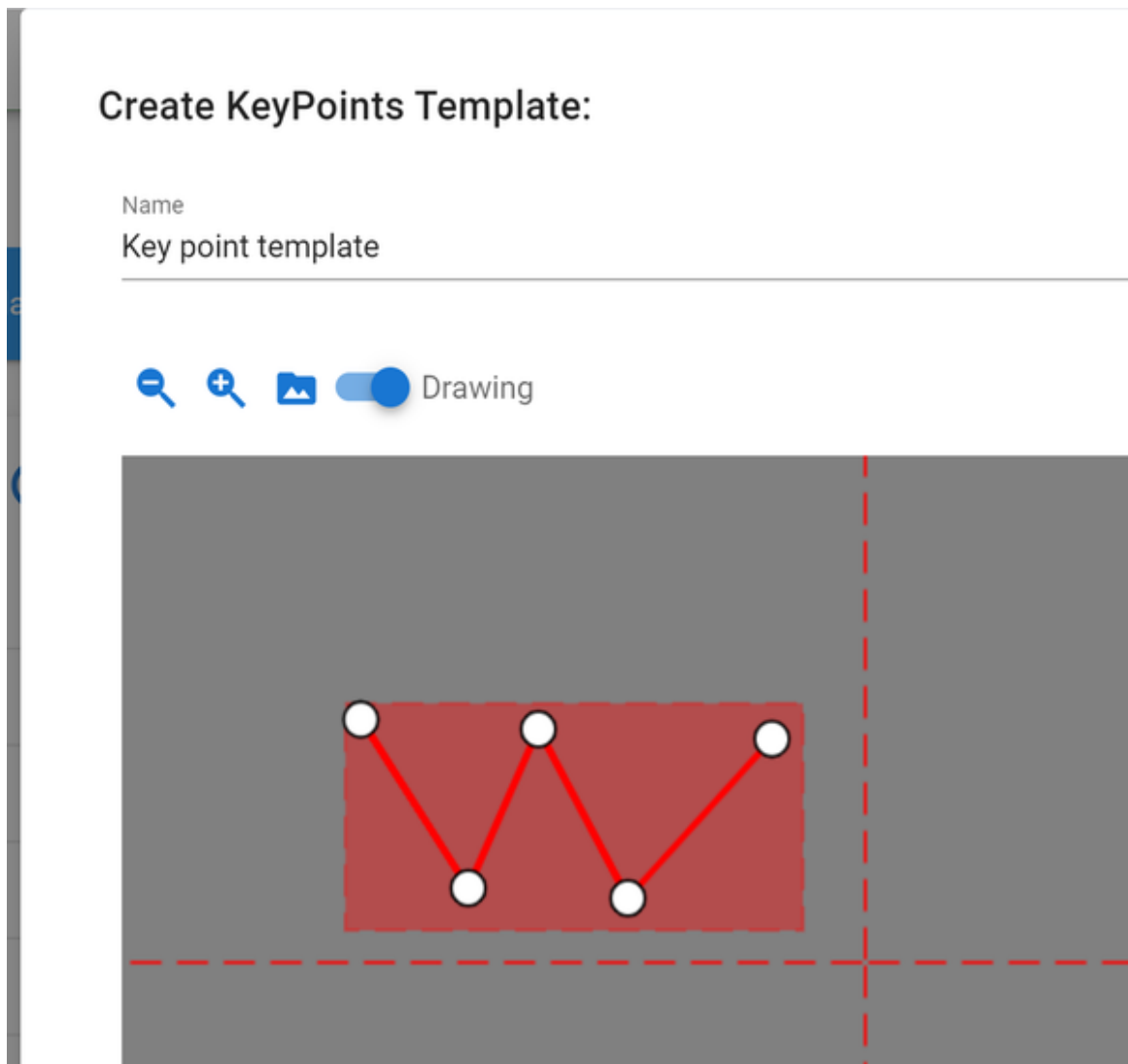


Figure 3-5. Keypoints template example

Other Spatial Templates

You can also create default spatial templates. These can be both for custom shapes, essentially another path to keypoints. The main difference is that keypoints often also include a referenced relation or edge to each point. Whereas a generic polygon template, like the diamond shown in Figure x.x does not specifically enforce this.

Figure 3.6 A visual example of an instance template shaped like a modified diamond. And the UI showing the process of creating a new template is depicted in Figure 3.7.



Figure 3-6. Instance template example

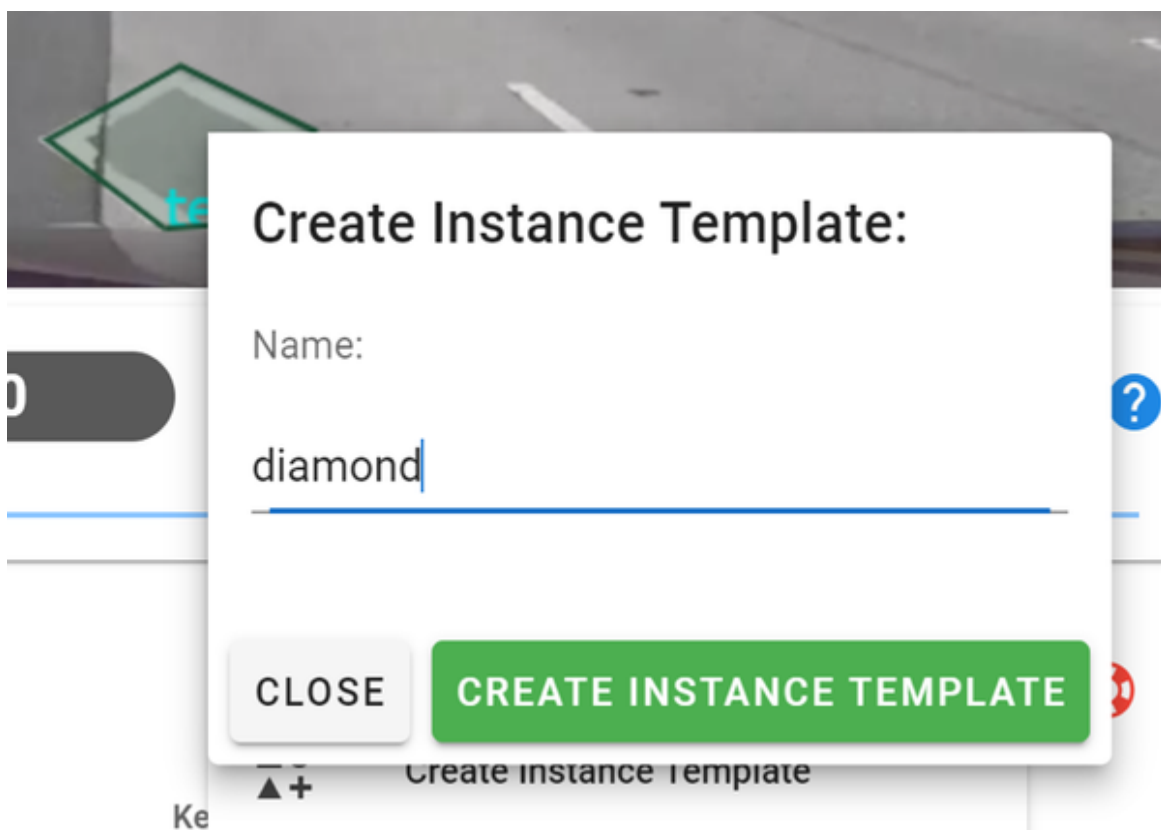


Figure 3-7. New template creation

Timing Concern

A lot of the schema concerns have already been covered or are more abstract concerns. As an administrator probably the biggest “how to” item is to understand the timing concern.

Depending on how strict your project requirements are any substantial changes to a label may invalidate portions of prior work. As an example, consider I label something a “Tree” and then the label gets changed to “Branch”. At the time of labeling, I was labeling “Trees”, not “Branches”! Editing labels after work has started is non trivial, for that reason labels that are edited do not cascade to prior tasks. The labels for those tasks for “frozen”. So in theory the system protects you from this automatically but it’s something to understand and be aware of.

Create Tasks for your Annotators.

The good news is that once you have the data imported into your system, and your schema defined, you are well down the path of having your project setup.

The basic case

In the most basic case, you will define a batch of a work with some descriptive name. Attach a dataset. If your annotators are not already invited to the project, then invite them. If the Guidelines are not already created, then create them. It’s usually best if the task assignment is managed by the system, but if needed, you can manually assign tasks. This most basic case generally requires no other admin work in this section.

Streaming Data

One of the big ideas is that of perpetual improvement of models. This is accomplished by repeatedly updating training data and updating models. In the ideal case, this is done through a “streaming” type mechanism, eg a new model prediction is automatically pushed to a review system if criteria are met. We will talk more about the “MLOps” process later. For now the main

thing to think about is: Where do I want the data to go after that task is completed? What kind of stages do I want to do?

Streaming Stages.

[TK diagram with streaming / templating engine]

TK: Your annotators view the images and do the annotation

From an admin view, a few things to keep in mind.

TK: Sampling work in progress

It's perfectly normal to construct a "sample batch"

TK: Reporting

Timing is not everything.

TK: How long does it take?

[TK: Stats]

TK: Export

(Content TK.)

Quality Assurance

Often, for people new to training data, there is an assumption that quality assurance surely can't be that hard? I mean just look at the data right?

The main issue is that the amount of data a single administrator can see is typically a small fraction of the overall annotated set. In some cases even a small sampling of the annotated data may be more than an admin could

possibly review in their lifetime. This is a strangely hard fact for many people to accept. It seems so easy to look at “a few images”.

As we can see in Fig 3.8, the amount of data a person sees in a reasonable amount of time is often a tiny fraction of the overall dataset size. Often in practice the difference is many orders of magnitude. There’s simply too much data!

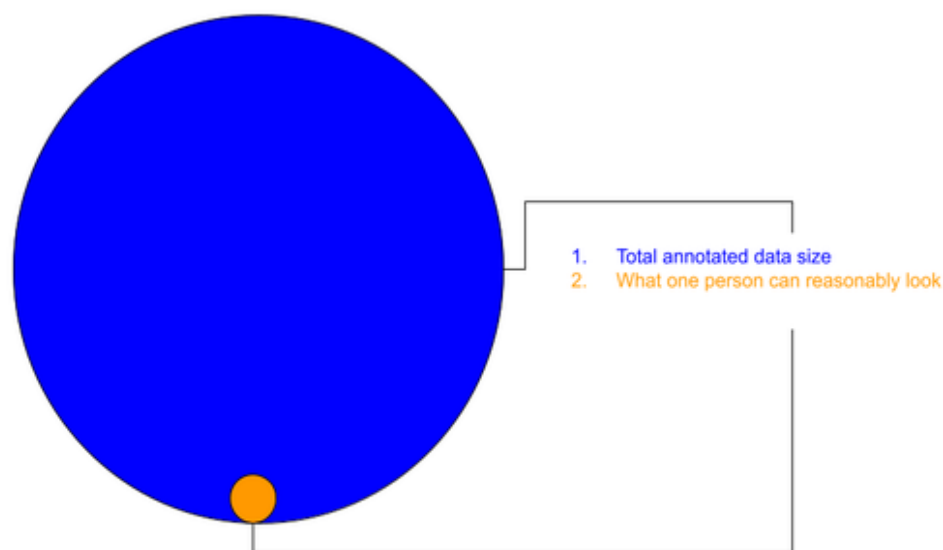


Figure 3-8. Comparison of total dataset size compared to what a person can reasonably look at in a reasonable time frame.

The solutions to this problem generally involve getting more people involved and more automations. We will cover automations in a later chapter. With more people involved the next big question is really “How much do I trust my annotators?”

Annotator Trust

Many early efforts in computer vision generally assumed that annotators were “heathens”. Essentially that they couldn’t be trusted individually. This led to a lot of research in verifying the wisdom of crowds. At a high level this boils down to having multiple people do the same task and then analyzing the aggregated result. But is this the best approach? Let’s take a closer look.

Consensus - The Past

For example, in Fig 3.9 we show 3 people drawing a similar box, and the combined result gets analyzed into one result.

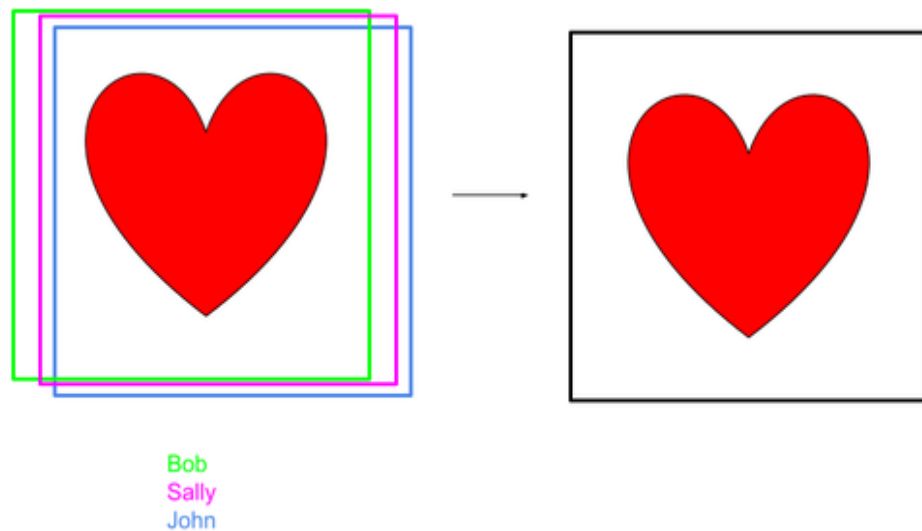


Figure 3-9. Box drawn by 3 people and the combined result after analyzing.

If it sounds a little overwhelming to have multiple people do the same thing and try to combine the results - you’re right! - it is! Generally this at least triples the costs involved. It also introduces a bunch of challenges around analyzing the data. Personally I also think it leads to the wrong type of analysis. It’s generally better to simply get more samples, supervised by different people, then to try and get some grand conclusion on single samples. For example, if there’s wild disagreement - what do you do?

As a rough analogy, imagine asking 3 engineers to write an application, and then trying to automatically figure out which code was “better”. It just doesn’t make sense! Compare that to the 3 engineers working on different sections, then coming together to consciously discuss concerns on specific design trade offs.

The second major problem this introduces is that most real world cases are much more complex than a single instance. Attributes, video, etc. If one person says it’s one frame 12 and another one frame 13 - do you split the difference? While there’s some very real research in this area - generally as the complexity of the challenge has increased, and the “seriousness” of the annotators has increased, the need for consensus has decreased.

Gold Standard - The Current Default

Let’s start from the assumption that Annotators are well meaning and in general will make good choices based on the data presented. In this context, an Administrator is still responsible to provide a definition of what “correct” is. This can be a fluid definition. It can be done by the most senior subject matter experts. This approach can be combined with random sampling of “review” type tasks. It can also be used as an examination. For example, an image is shown to an annotator, they do their attempt, and then the result is compared to the gold standard.

In comparison to consensus this is about a third of the cost. And yet - generally can yield similar or even better results. With consensus there is this sort of forced anonymous aspect. Whereas with this there is a more clear “right/wrong”. There is more accountability - if I create a task, and it gets reviewed and there’s an issue, that’s a clear signal. This mechanism can surface issues in the Schema , data, etc. It also helps encourage individual learning experiences - eg annotators improving.

This is a developing area and there may be other approaches that come up in the future. This area is also somewhat controversial. Generally it’s best to start with the gold standard approach, unless you have a clear reasons or rationale to use consensus.

Annotators are Partners

Annotators are a key part of the QA loop. Often I have seen people jump to an “us vs them” mindset with this. While understanding QA goals and important, and rigor is especially important in larger projects, we are all on the same team here.

I think often concerns regarding annotator agreement and quality are overstated. For many current annotation cases, people generally agree on it's meaning at a high level. Often when there is disagreement, the Schema, resolution of data, hidden assumptions, etc. are more to blame than any single annotator. Working in partnership to define the specs of the project before it begins and building feedback loops into your process will provide the highest quality and most accurate results.

Who supervises the data

There's a human psychological element that's well beyond the scope of this book, but it's worth considering there's *humans* behind the data here.

As an analogy, consider that a programmer is involved in the compilation of her code. At the end of the day, it runs or doesn't. She gets feedback on her efforts. Whereas here, an individual annotator/expert etc. may never get the same degree (or speed) of feedback on their annotation efforts. And may never get feedback on their effect on the ML models.

Think of it almost like doing a “code review” - but with all the effort we go through in a formal code review to protect feelings, emotions, and nascent ideas.

All training data has errors

Training Data quality is of critical importance. The same way code quality is. Similarly all computer programs have bugs and all training data has some form of errors. Annotators are only one source of errors. From what I have seen it's best to focus energy on systemic issues and process over “one-offs”. ML models are surprisingly robust to occasional bad training

data. If there are 100 “truly correct” examples, and 5 “misleads”, in general the model will still work as expected.

For the cases where it really really matters that 100/100 are correct I think either we are kidding ourselves or they are quite rare. Yes - if a machine learning model is landing the Mars mission ok sure the training data needs to be triple checked. But otherwise we are being penny wise and pound foolish trying to over optimize every “error” out of the system.

Annotator Needs

People’s psychology and general work ethic is often at least as important as any specific QA process. People are respectfully treated, compensated, and trained, will naturally perform differently then “lowest bidder” approaches.

- Who is actually annotating the data?
- Are they trained on the tooling? Common expectations (such as zooming in) may not be clear, even to professionals, unless explained. Don’t assume!
- Is their technical setup appropriate? Are they seeing the same thing I’m seeing? (For example, common screen resolutions can differ between countries.)
- What kind of training have they received?
- Do they have an individual account where we can track their specific concerns and performance metrics?
- Are the performance metrics an accurate reflection on the person?

VIDEO MEETINGS EXAMPLE

As a quick aside to consider how knowledge of training data and tooling does not correlate with annotator intelligence consider video meetings. I know when I join a video meeting with a platform I'm unfamiliar with, even simple operations like muting/unmuting can be hard to find or forgotten. Annotation is similar. One tool may do something easily, or even automatically, where as another tool might not. A doctor who has 30 years experience with cardiology may not know a specific tool has a specific feature.

Common Performance Metrics

1. Time per task. Cautions: tasks can vary substantially. This metric can be very dangerous to use by itself. Suggest combining with quality metrics.
2. Number of “accepted” tasks. Accepted could mean passed review, was not reviewed, or similar metric.
3. Count of instances updated or created.
4. Count of instances per task or per frame
5. Count of tasks complete. Also consider the difficulty of tasks and type of tasks.

Common causes of Training Data errors

- Low resolution raw data. If the data is pushing the limit of what a human can reasonably see - keep that context in mind.
- Schema is sub optimal (for this specific section). The annotator cannot reasonably represent their knowledge in the schema.
- Guide is suboptimal or contains errors such as contradictions.
- Highly specific Labels and Attributes

- Free form Labels and Attributes (Annotators enter own novel text / label)
- More complex spatial types. Or overly specific spatial types (pixel specific).
- Any form of “hidden” information, eg occlusion
- Any naturally controversial content (eg politics, free-speech/hate-speech, security type movements, facial recognition)

Annotator

As an annotator you are writing the code that powers ML models. You have a huge responsibility to map noisy real world data onto a Schema defined by others. On your journey we will cover some core concepts of actually doing annotation. Including Video series, images, and mechanics.

A few quick tooling specific notes:

I will present the general goals and themes of each concept. Each application may have specifics, such as buttons, hotkeys, order of steps, etc. that are slightly different. Sometimes for some of these “how to” things it’s nice to jump to the specifics of a current implementation but let’s keep in mind UIs will naturally evolve and change over time too.

Your role in helping setup and maintaining the Schema

As an annotator, you have a ground level view of the data. This means you will often have valuable insights to contribute about the effectiveness of the Schema relative to the actual data. If you are a subject matter expert you may have additional efforts to lend in setting up and maintaining the Schema.

What should you expect to have prepared for you?

Often by the time it gets to actually annotating the initial Schema has been defined. There's still lots of opportunity to help maintain and update it as issues are encountered such as the bike rack vs bicycle example we covered earlier. A few items that are more dependent on your specific case:

1. Spatial tool type
2. Use of automations
3. Use of other UI assist features

Sometimes, a single label may have multiple spatial types available for it. In this case you may need to choose the appropriate spatial type.

Drawing a Bounding Box

There are a surprisingly broad number of ways to draw a box. From pure "UI" items, such as "click and hold to release" or "two clicks, to a variety of more extreme methods. Some do just the "extreme" points (which can be used for segmentation too), etc.

Automations

There are generally two types of automations tools you may interact with as an annotator, static and interactive. Starting with static, a popular example is Pre-Labels. This is where the model has already made a prediction.

Depending on the use case, you may be asked to correct static automations, add more detail to them, or otherwise interact with them. By static I mean this is usually a "once and done". You receive the file in some state, such as 2 bounding boxes added, you update it once, and mark it complete. This is different from interactive, where you may be able to continually work with it until some result is achieved.

Anytime you see a pattern this is a great opportunity to help improve the model. Are you always correcting a similar seeming error? Communicating that to the Admins or Data Science team can go a long way to improving

the model. This can be relevant even if the Schema is correct - it could be something internally wrong with the model that needs fixing.

Interactive Automations

An example of interactive automation is drawing a box around an area of interest and automatically getting a polygon drawn closely around the object. More generally, an interactive automation is where you add more information into the system, and then some process runs based on the new information you added. Sometimes this can be done iteratively until some desired state is reached. This can sometimes be a “try again” type thing, where you keep trying to draw a box until it gets the right polygon, or it can be a “memory” based system where it continues to change based on your ongoing inputs.

Usually as an annotator you won’t need to actually “code” any of these interactions. They will often be provided as UI tools or hotkeys. You may need to know a little about the operating parameters of the automations. When they work well, when they work poorly, when to use what automation if there are many available etc.

Here I will cover a few of the most popular approaches.

1. Box to Polygon
2. Points to Polygon
3. Full image or general prediction
4. Tracking? (under the video section?)
5. Interpolation?

TK (expansion of the above list)

Semantic Segmentation

(Content TK.)

Auto Bordering

Automatically detects edges to create 100% coverage masks.

This is generally faster and more accurate than trying to either draw the border manually.

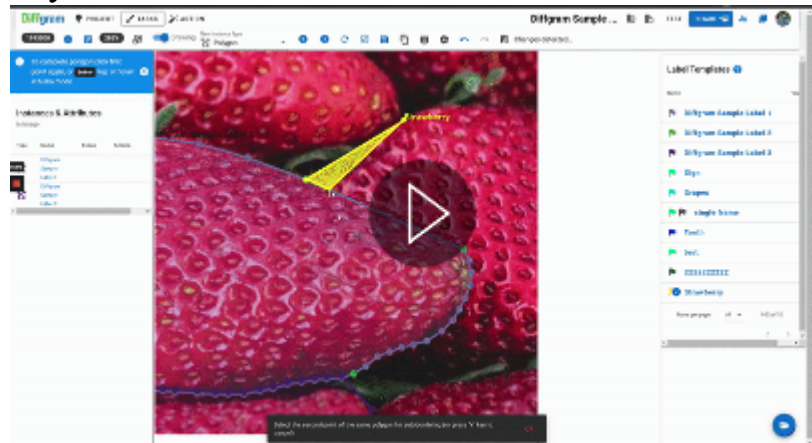


Figure 3-10. An example of auto bordering

Steps:

1. Select a point on the intersecting shape.
2. Select the exit point on the intersecting shape

OR

1. “Draw over”, eg draw over an existing object, and expect that it will auto border around the intersection of the points.

Video

Introduction to video content TK.

Motion

We annotate video to capture meaning in motion. Cars moving, basketball shot being taken, factory equipment operating. Because of this motion, the default assumption when annotating is that every frame is different.

Consider [this raw video](#).

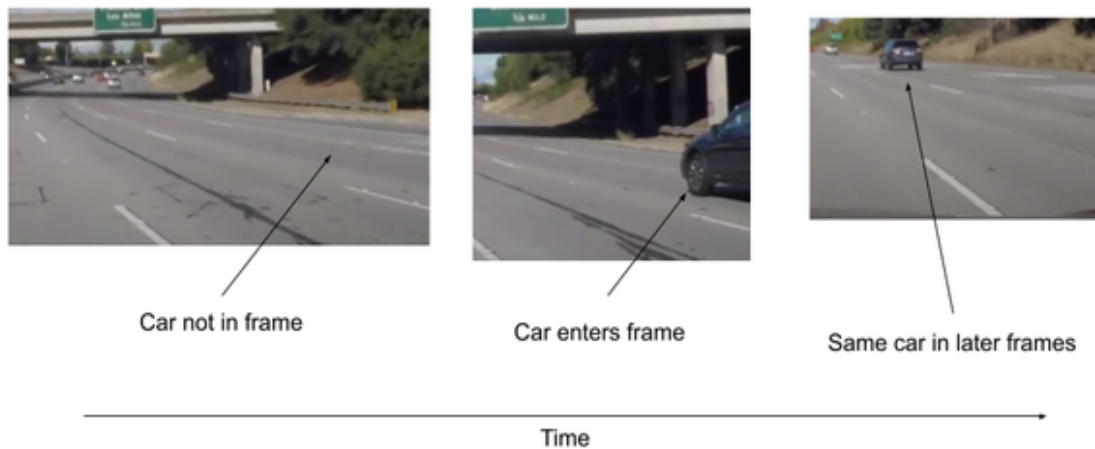


Figure 3-11. Video frames of a highway and car in motion

The car is not in the initial frame. It moves into the frame. Then later in time it is still in the frame. More generally objects move in and out of the frames. This happens at different points in time. Attributes of the car may change too - for example in one frame it may be fully visible and in the next it's partially occluded.

Ghost Instances - Basics of Tracking objects through time

There are several concepts to discuss.

Capturing Time Series

Also known as: Sequences, Tracks

There are different opinions on specifics of how to capture this. Generally speaking the goal is creating some relation between annotations in multiple frames.

One approach is:

1. Create (or select) a series. Each series can be unique to the video, unique to a set of videos (eg multiple cameras), or globally unique across videos.

Generally, this forces that every object is part of a sequence. This works well if objects are generally present in more than 1 frame but can be a little bit cumbersome if objects are routinely only in one frame.

Generally, a sequence enforces additionally constraints:

1. A sequence can only appear once in a given frame. For example Series 12 cannot appear twice in the same frame. This may not always be correct - for example an object may be partially occluded and could be represented by two or more spatial types. (Picture a bus blocked by a post)
2. A sequence must be of the same label type. While an attribute may change the “top level” concept should generally be the same between frames.

Static Objects

Sometimes a video will have a static object, E.G. a retail store with a shelving unit or other display that doesn't move (or doesn't move often), or an intersection with a traffic light that's not moving.

You can represent this by:

1. A single keyframe ie at frame x (eg. #898).
2. A keyframe at the start and end ie (9, 656). With the assumption that the object enters at frame 9 and exits at frame 656. More generally this pattern is (entry frame, list of frames, exit frame).
3. An attribute, such as “enters” or “visible”. (or % visible).

These are all ways to effectively tag it as a “static object”.

Persistent Objects - Football Example

A video may have multiple objects in it. For example, 2 different cars, apples, football players, etc.

From the human perspective we know a football player, such as the player in Fig. 3.12 , is the same person, in frame 0, 5, 10 etc. In every frame, he is still Ronaldo.



Figure 3-12. A football player in a frame; Source: <https://unsplash.com/photos/ztwaJXBWCC0>

But this is not as clear to a computer. So to help it, we create a sequence object. i.e. “Ronaldo”. Since it’s the first object we created, it gets assigned sequence #1.

If another player “Messi” was also in the frame, we could create a new sequence for him, and he would get #2. Another player would get #3 and so on.

The key point being that each sequence represents a real-world object (or a series of events) and it has a number that’s unique for each video.

Series example

We can create a new “Series” to create meaning over time.

Imagine a video with 3 objects of interest. To represent this we create 3 Series.

1. Series #1 has frames (0, 6, 10) because the object enters the video at frame 0, something about it changes at frame 6, and then leaves the video at frame 10. The object is changing over time in each keyframe.
2. Series #2 has frames (16, 21, 22...). Object also changing.
3. Series #3 has frames (0) only. It's a static object that doesn't move.

A Series may have 100s of Instances.

Video Events

A few ways to represent events

1. Create a new Series for each event
2. Create a new frame for each event. Eg (12, 17, 19) would mean 3 events, at frame 12, 17, and 19.
3. Use attributes to declare when an “event” occurs.

From a UI standpoint the main trade off here is how many events you expect and how complex the rest of the Schema is. As a rule of thumb, if there's less than 50 events, it may be “Cleaner” to keep them all as separate series. If there are more than 50 events, it's generally better to have a single sequence, and use either frames or events.

Attributes can work well for complex event cases. The main downside is that they are often less “discoverable” vs keyframes which can be more visible to “jump” to a specific point in the video.

In terms of annotation speed, pay close attention to hotkeys for video events. Often there can be hotkeys to create new sequences, change sequence, etc.

In terms of common annotation errors, be sure you are on the desired sequence. Often thumbnails will help visually determine this. You can also jump back or forward to other frames in the sequence to verify it.

Detecting Sequence Errors

Imagine you are reviewing a video that was pre-labeled (either by another human or an algorithm). To aid with this, tools like Diffgram automatically change the color of the sequences. This means you can play the video, and watch for color changes in the sequence. It can be surprisingly easy to catch errors this way. For real world cases with overlapping instances often playing the video with this color feature is the easiest way to detect issues. You also may be alerted to a potential Series error through an automated process or another annotator raising a ticket on it.

Figure 3.13 shows an example where the same vehicle has its sequence number changed wrongly.

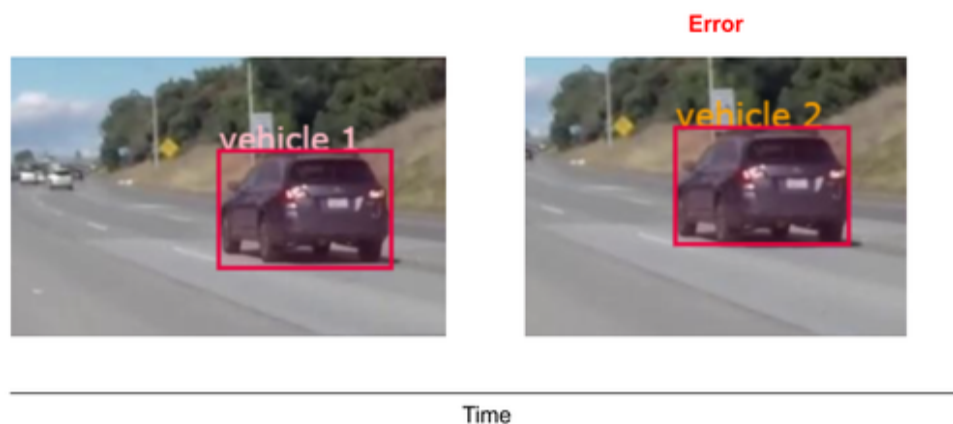


Figure 3-13. A sequencing error where the same vehicle has been labeled with different numbers.

To correct it, I open the context menu of the instance and select the correct sequence. As shown in Fig 3.14.



Figure 3-14. An example of the process to correct an invalid sequence.

In this example, I know it's correct because:

1. The car is most similar to it's prior frame.
2. The car is visually similar to the thumbnail of the series.

Common Issues in annotation

(Content TK.)

Declaring what “should” exist vs what you can actually see

We as humans may take in a given scene, such as a driving roadway, and assess where the shoulder of the road / vegetation is. While purely looking at the image pixels, there is not significant visible evidence for it!

TK:

- Classic example would be the “reasonableness” of shapes...

- The choices of various methods impose a variety of constraints.
For example, object detection predicts a rectangle shape, whereas most real-world objects are not rectangular.
- “in view” vs “allow outside”
- Team Mechanics

Summary

Next, we will look at Training Data Management and Operations.

Chapter 4. The Day-to-Day Practices of Training Data

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 4th chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at jleonard@oreilly.com.

Introduction

Congratulations! You have made it to the fun part! In earlier chapters, I covered the scientific baseline of training data. In this chapter, we’ll move forward from that technical introduction and enter into the art of training data. Here you can start making more subjective choices. I’ll show you how we work with the art of training data in practice as we walk through scaling to larger projects and optimizing performance.

First, it’s worth noting that practice makes permanent. Like any art you must master the tools of the trade. With training data there are a variety of tooling options to become familiar with and understand. I’ll talk about some of the trade offs, such as open source, deployment options and explore popular tools.

Training data tooling is concerned with storing and retrieving training data. This includes human workflow, annotation, and annotation automations as well as exploring, debugging, and managing training data. This is usually different from data science tooling, which is centered around taking that data to create, optimize, measure, and deploy models.

A quick note that applies especially to this chapter. This is a rapidly evolving area. Best effort is made to make this both practical while recognizing that it is changing.

First, I'll cover core ideas around dataset and task organization. Let's get started!

The Components

In software development there are often popular “stacks”, or sets of technology that work well together. It is still hotly debated what the canonical stacks for Training Data are. So I will speak to some of the general areas of responsibility and provide examples.

Some products cover most of these areas in one platform. Whether you buy an off the shelf platform, or build your own through discrete tools, you will need to cover each of these buckets.

You will need at least one, and sometimes multiple, products in each bucket to achieve the required results. The data type may dictate this, for example one annotation tool may only do text and another may only do visuals. Alternatively, there are tools that cover virtually every annotation type, but then may lack automation or exploration.

Looking at the detail of the training data stack, depicted in [Figure 4-14.1](#), we see the nine areas it's broken down into, spanning from ingestion, to annotation, to security.



Figure 4-1. Fig 4.1: Training Data Stack

Components Overview

Ingest:

Ingest raw data, prediction data, metadata and more.

Store:

Collaboration across teams between machine learning, product, ops, managers, and more.

Workflow:

Manage Annotation Workflow, Tasks, Quality Assurance and more.

Annotation:

Literal data annotation UIs for images, video, audio, text, etc.

Annotation Automation:

Anything that improves annotation performance, such as pre-labeling or active learning. See Chapter 6 for more depth.

Stream to Training:

Getting the data to your models.

Explore:

Everything from filtering uninteresting data to visually viewing it.

Debug:

Debugging existing data, such as predictions and human annotations.

Secure & Private:

Data lifecycle including retention and deletion, Personally Identifiable Information, Access Controls.

The Order of Components Used Varies

You may ingest data, explore it, and annotate it in that order. Or perhaps you may go straight from ingesting to debugging a model. Generally, Ingest is the first step. Beyond that the order varies. After streaming to training, you may ingest new predictions, then debug those, then use annotation workflow.

We'll continue to explore each of the nine elements from the training data stack in detail throughout this chapter, but first, I'll present helpful information around getting started and considerations that should be taken into account as you scope your project.

Now that we understand the training data stack in principle, what does it mean in practice?

Training Data for Machine Learning

Training Data is a sub focus within the broader context of Machine learning.

Usually, machine learning modeling and training data tools are different systems. Some offer a form of all in one. Usually the trade off if they are integrated is that of power, the more integrated it is the less flexible and powerful it is. As an analogy, in a word doc I can create a table. This is distinctly different from the power of formulas that a spreadsheet application brings.

This chapter will focus on the major sub areas of training data specifically assuming that the model training is handled by a different system. Streaming to training is one of the leave off points for Training Data.

Growing Selection of Tools

There are an increasing number of notable platforms and tools becoming available. Some aim to provide broad coverage while others cover deep and specific use cases in each of these areas. There are tens of notable tools that fall into each of the major categories.

As the demand for these commercial tools continues to grow I expect that there will be both a stream of new tools entering the market and a consolidation in some of the more mature areas. Annotation is one of the more mature areas. Data exploration in this context is relatively new.

I encourage you to continuously explore the options available that may net different and improved results for your team and product in the future.

Ingest

Ingestion is the first and one of the most important steps. At a super high level this means getting your data into your training data tools. Why is this hard? A number of reasons, including formats, volume of data, and the many ways to do it.

Manual Import

The most basic approach is for sensors to capture and store the data independently of the training data tooling, as shown in Fig 4.2. This could mean mobile phones, computers, dedicated cameras, etc. Commonly this data is stored on a hard drive or cloud bucket. There may be some organization via folder structure or similar. This can also mean a dedicated team or sub-system designed for grouping data from other teams in one central place that does nothing but store and retrieve the data.

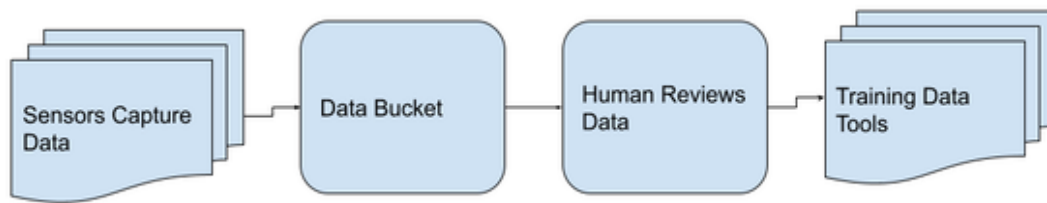


Figure 4-2. Basic Manual Ingestion Process

Then, as a separate step, the data is “imported” into the training data tooling. The unspoken assumption here is that often only the data desired to be annotated is imported. This approach is easy to reason about. It’s very durable in that it places virtually no pre-integration requirements. It works on old data, new data etc.

The forced assumption that only data designated for annotation is imported is limiting. It makes it hard to effectively use exploration and prep methods. The data is duplicated at rest. There are likely security issues since the raw storage (first copy) will often have different security rules than the training data tooling (second copy).

Currently, there are some cases where it may be impractical to stream the data directly to a tool and this may be the only practical option.

Direct to Training Data Tooling

The new way, and I believe generally better, is to treat the training data tooling as a database, and send it first and foremost to the training data tooling as shown in Fig 4.3. The data can still be backed up to some other service at the same time, but generally this means organizing the data in the training data tooling from day one.

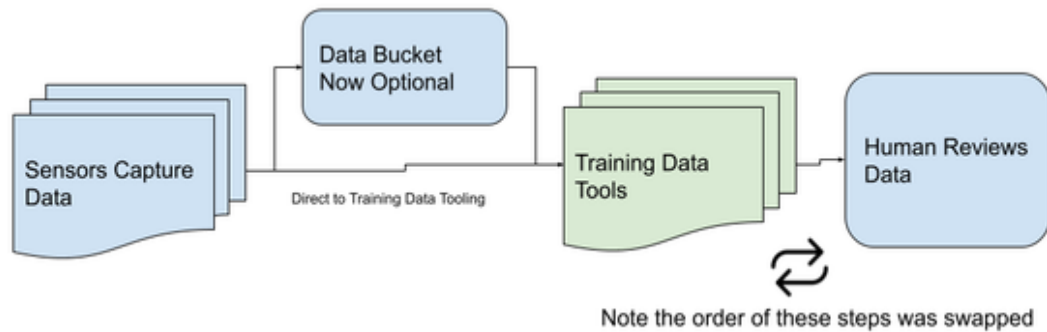


Figure 4-3. New Direct to Training Data Tooling Process

There are multiple trends driving this shift. At the highest level, it's a shift from model centric machine learning to data centric. This means focusing on improving overall performance by improving the data instead of just improving the model.

This approach allows for training data tooling to drive data discovery, what to label, and so much more. It avoids data duplication. It unblocks teams to work as fast as they can instead of waiting for discrete stages.

There are two distinct parts to this.

1. Swapping the order of when humans review the data. Instead of deciding what raw data to send, it *all* gets sent to the tooling first. Then the human reviews the data inside the tool(s).
2. Optionally, the training data tooling takes on the role of central data bucket. This is sending the data directly to the training data tooling. In actual implementation there could be a series of processing steps, but the idea being that the final resting place of the data, the source of truth, is inside the training data tool instead of another system.

Generally speaking, the tighter the connection between the sensors and the training data tools, the more potential for the tools to be effective. Every other step that's added between sensors and the tools are virtually guaranteed to be bottlenecks.

Having the data all in one place

One theme you will start to see as you work with different tools is that the first step for pretty much any of them is to get the data to the tool. As a brief tangent, in theory an alternative here is to bring the program to the data, but in practice for training data I have yet to see this work.

A closely related theme is that of exporting the data to other tools. Perhaps you want to run some process to explore the data, and then need to send it to another tool for a security process, such as to blur personally identifiable info. Then you need to send it on to some other firm to annotate, get the results back from them into your models etc. At each of these steps there is always a mapping problem. Tool A outputs in a different format than Tool B inputs as. And even if the mapping is pre-configured, there is the physical transfer time.

I have talked a bit about data scale before, but as a quick reminder, this type of data transfer is often on the order of magnitude more than in other common systems. The best rule of thumb I can think of is that each transfer is more like a mini database migration. That's much closer to the truth of it!

Sometimes this is unavoidable. A growing trend is for tooling providers to offer more and more services in one place to avoid this. If the same system already knows how to access the data, then there is:

1. No time spent mapping data
2. No need to "transfer" data - since the data was already validated and prepared for use for all subsystems.
3. No risk of data duplication.

How is this related to ingest? Well, the advantage of a tool that covers more of these sub areas is that it can store the data with this in mind from day one. The application considers those uses from day 1 and plans accordingly.

As an example, consider indexes. An application designed to support data exploration can create indexes for data discovery automatically created at

ingest. When it saves annotations, it can create indexes for discovery, for streaming, for security etc., all that same time.

TK: that format can be used for the other roles.

Avoiding a game of telephone

Often, as data is transferred between these tools, it looks like a game of telephone as shown in [Figure 4-14.4](#). The data keeps getting garbled so while one tool may know about xyz properties, the next tool does not import it, and it likely won't export all of the properties that it stores or imports.

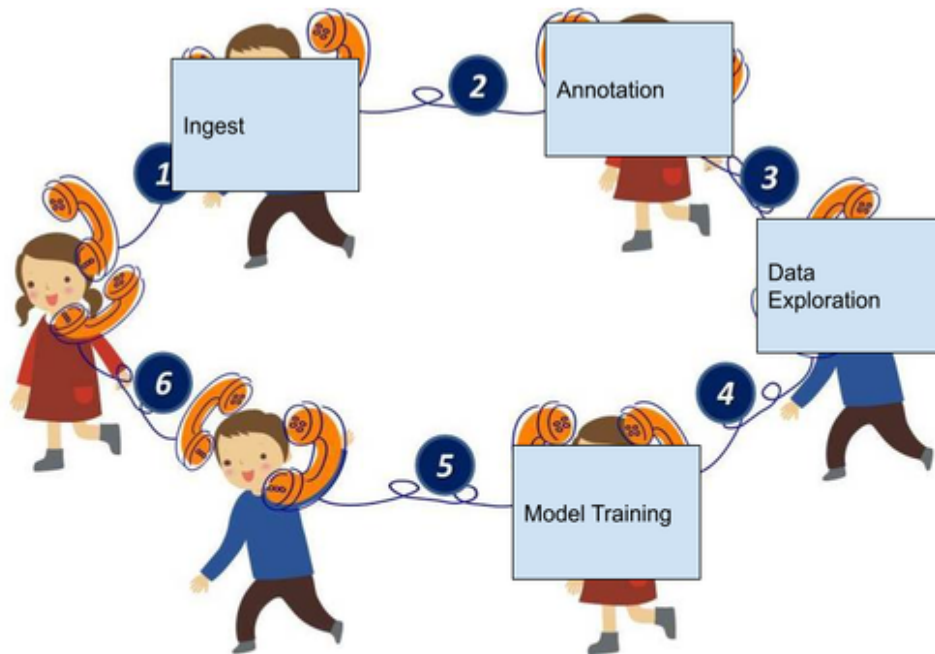


Figure 4-4. When data is transferred between tools it can look like a game of telephone, resulting in garbled data

Like telephone (chinese whispers) “Errors typically accumulate in the retellings, so the statement announced by the last player differs significantly

from that of the first player, usually with amusing or humorous effect.” Except in this case it’s not humorous!

This is partly why ingestion takes on such significance. Some questions to think about during the system design:

1. How much distance is there between {sensors, predictions, raw data} and my training data tools?
2. What percent of overall predictions made are we usefully capturing in our tooling?
3. How many times is data duplicated during the tooling processes?

To achieve this data driven approach, often a lot of iteration and data is needed. The more iteration and the more data the greater the need for great organization to handle it. Another reason is that often more and more predictions are being done and that “pre-labeled” data is available.

Raw Storage Notes

It is common to store raw data on cloud buckets. Not all tooling providers support all of the major clouds - Google GCP, Microsoft Azure, and Amazon AWS. Other open source offerings like Hadoop have even more minimal support.

Some people like to think of these cloud buckets as “dump it and forget”, but there are actually a lot of performance tuning options available. At Training Data scale raw storage choices matter. If you are a cloud guru then feel free to skip this subsection.

Storage Class

Under different names, each cloud provider offers various hot and cold tiers. There can be as much as a 4x price delta between the warmest and coldest tiers. Usually with a few clicks you can set policies to automatically move old files to cheaper storage options as they age.

Storage Zone

People regularly store data on one side of the Atlantic Ocean and annotators on the other side access it. However, it's worth considering where the actual annotation is expected to happen, and if there are options to store the data closer to it.

Storage Support

Not all annotation tools have the same degree of support for all major vendors. AWS is the most widely supported. Keep in mind that you can typically manually integrate any of these offerings, but this requires more effort than tools that have native integration.

Support for accessing data from these storage providers is different from the tool running on that provider. Some tools may support access from all three, but as a service the tool itself runs on a single cloud. If you have a system you install on your own cloud, usually the tool will support all three.

For example, you may choose to install the tool on Azure. You may then pull data into the tool from Azure which leads to better performance. However, that doesn't prevent you from pulling data from Amazon and Google as needed.

Ingest Wizards

A new emergence of UI based ingestion wizards.

This started originally with file browsers for cloud based systems. And has progressed into full grown mapping engines, similar to smart switch apps for phones, where I use an app to move all my data from android to iphone or versa.

At a high level a mapping engine steps you through the process of mapping each field from one data source to another.

Mapping wizards offer tremendous value. They save having to do a more technical integration. They typically provide more validations and checks to ensure the data is what you expect (picture like seeing a email preview in gmail before committing to open the email). And best of all once the mappings are set up, then can easily be swapped out from a list without any context switching!

The impact of this is hard to understate. Before you may have been hesitant to try a new model architecture, commercial prediction service, etc. because of the nuances of getting the data to and from it. This dramatically relieves that pressure.

What are the limitations of wizards? Well first some tools don't support them yet so it may not be available yet. It may impose technical limitations that are not present in more pure API calls or SDK integrations.

A few takeaways:

- Be aware these wizards exist.
- Wizards are typically the better default option.

One of the biggest gaps in tooling is often around the question “How hard is it to set up my data in the system and maintain it?” Then comes what type of media it can ingest? How quickly can it ingest it?

This is a problem that's somewhat distinct from other software. You know when you get a link to a document and you load it for the first time? Or some big document starts to load on your computer?

Store

Why is storage different from ingestion? If we are ingesting it aren't we also storing it? Yes and no. Store means storage and retrieval.

One way to think of this is that often the way data is interested in a database is different from how it's stored at rest, and how it's queried. Training data

is similar. There's processes to ingest data, different processes to store it, and different again to query it.

Is the tooling actually storing my data? Or is it only storing references and I must manage storage of some artifacts outside of the system?

Does the system store the data in a database or does it get “dumped” into a JSON type format after each batch?

Versioning

There are 3 primary levels of data versioning, Per Instance, Per File, And Export. Their relation to each other is shown in Fig 4.5.



Figure 4-5. Versioning High Level Comparison

Here we introduce them at a high level.

Per Instance History

Instances are never hard deleted. When an edit is made to an existing instance, Diffgram marks it as soft delete and creates a new instance that succeeds it, as shown in [Figure 4-14.6](#). For example, use this for deep dive annotation or model auditing.

Instance History



- 5  [7231342] Instance Edited
UI/WebApp 04-07-2021 19:59:08
User:  Anthony Sarkis
- 4  [7231339] Instance Resto...
UI/WebApp 04-07-2021 19:58:47
User:  Anthony Sarkis
- 3  [7231337] Instance Deleted
UI/WebApp 04-07-2021 19:58:36
User:  Anthony Sarkis
- 2  [7231336] Instance Edited
UI/WebApp 04-07-2021 19:58:34
User:  Anthony Sarkis
- 1  [7231335] Instance Created
UI/WebApp 04-07-2021 19:58:29
User:  Anthony Sarkis

✕ CLOSE

Figure 4-6. Left: Per Instance History in UI.

Instance Diff

		@@ -1,7 +1,7 @@	
1	1	{	
2	2	"x min": 706.	
3	-	"x max": 913.	
	3	+	"x max": 973.
4	-	"v min": 315.	
	4	+	"v min": 351.
5	-	"width": 207.	
	5	+	"width": 267.
6	-	"height": 200	
	6	+	"height": 236
7	7	}	

Figure 4-7. Right: A single differential comparison between the same instance at different points in time.

Per File & Per Set

Each set of Tasks may be set to automatically create copies per file at each stage of the processing pipeline. Automatically maintain multiple sets relevant to your exact task schema.

You may also programmatically and manually organize and copy data into sets on demand. Filter data by tags, such as by a specific machine learning run. Compare across files and sets to see the diff on what's changed.

Add files to multiple sets for cases where you want the files to always be on the latest version. That means you can construct multiple sets, with different criteria, and instantly have the latest version as annotation happens.

Crucially this is a living version, so it's easy to always be on the "latest."

For example, use these building blocks to flexibly manage versions across work in progress at the administer level.

Per Export Snapshots

Every export is automatically cached into a static file. This means you can take a snapshot at any moment, for any query, and have a repeatable way to access that exact set of data.

Combine with webhooks, SDK, or Userscripts to automatically generate exports. Generate them on demand anytime.

For example, use this to guarantee a model is accessing the exact same data.



The screenshot shows a table with the following columns: Date, Status, Directory Name, Job, File Count, Masks, Kind, and Actions. A single row of data is displayed, representing an export job. The 'Date' column shows 'Wednesday, January 22nd 12:38:49 pm'. The 'Status' column has a green checkmark. The 'Directory Name' is 'Luckfrill', and the 'Job' is 'Specklepuppy'. The 'File Count' is '2'. The 'Kind' is 'Annotations'. The 'Actions' column contains a download icon and a 'Download format' dropdown menu set to 'JSON'. At the bottom right, there is a pagination control showing 'Rows per page: All' and '1/1 of 1'.

Date	Status	Directory Name	Job	File Count	Masks	Kind	Actions
Wednesday, January 22nd 12:38:49 pm	✓	Luckfrill	Specklepuppy	2		Annotations	 Download format JSON

Figure 4-8. Export UI Listview Example

Workflow

Now that the dataset and storage are ready, you need something to label the data. Usually this includes at least some form of user interface (for instance, Photoshop) and some process and issue management (because normally multiple people are involved).

Generally, the assumption is that questionable data - raw, machine generated, or otherwise not yet analyzed by an admin. - comes into the system and human supervised data comes out.

In Chapter 3 I briefly introduce the concepts of creating tasks for human annotators. Here I will expand on that.

Templates:

The task template is a bundle of the label schema, related datasets, and other configurations like permissions. Also known as: Job, Project

Workflow Processes

Another area where there is little consensus is around the best “process for annotation”. In general there is a trend towards users being able to create some form of “pipelines” in which there are different instruction sets, people etc., depicted at different stages. Exploring the depth and breadth of this type of feature is very important, especially for larger scale use cases.

WHY PIPELINES ARE CONFUSING: THERE ARE MANY DIFFERENT TYPES

Pipelines are rather “in vogue” at the moment. You are likely to have many pipelines for different contexts. For example, you may have a training data pipeline - that’s for people-oriented tasks. Then there may be a model pipeline - how a training process works, how the best models get to production etc. And throughout all of that there may be some kind of generic Airflow type pipeline that moves data from some other system to the training data system in the first place.

Template Anatomy

First, let’s wrap our head around the general organization structures. Assuming we have gone to all the trouble of creating our label schema, getting datasets together etc. It’s reasonable to assume that a template will have more than one task.

Because often the schema within the same Project changes, and because we often have multiple stages of annotation, a Project contains multiple Task

Templates as shown in Fig 4.8.

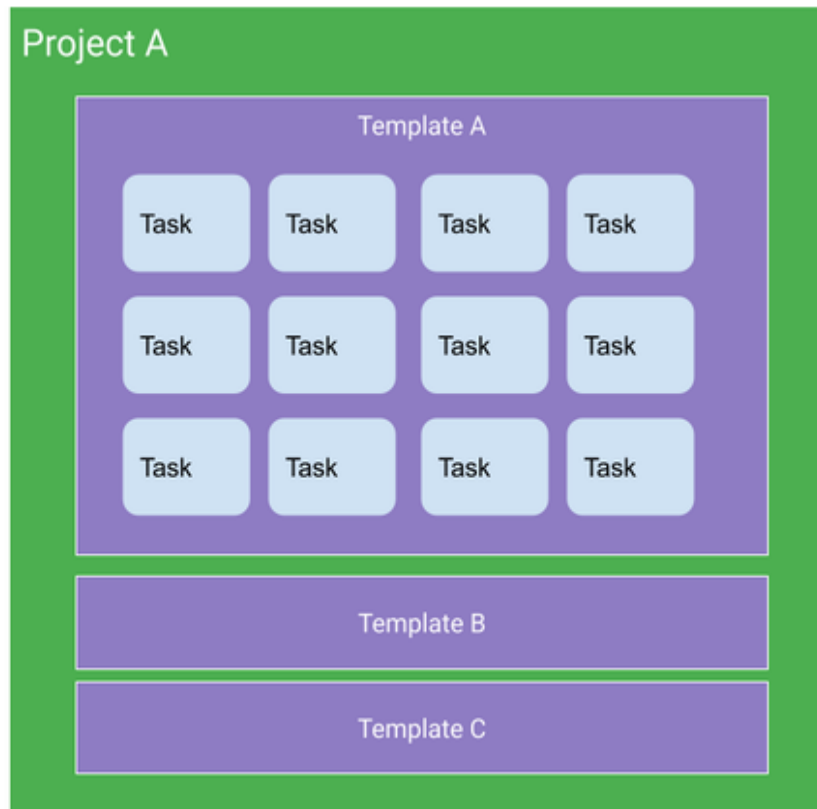


Figure 4-9. Task Structure

Workflow Management

Workflow is different from exploration. It's the concept of organizing work for the sake of doing annotation.

There are 3 overlapping concepts that help define the general shape of Datasets

1. Folders and static organization
2. Filters and dynamic organization
3. Pipelines and processes

Folders and static organization

When I think of management I often think of organization. For computer data, I picture files and folders on a desktop. Files are organized by being put into folders. For example if I put 10 images in a folder “cats”, I have in a sense created a Dataset of cat images.

[TK: visual showing a desktop file browser and images of cats]

Filters and dynamic organization

A dataset may also be defined by a set of rules. For example I could define it as “All images that are less than 6 months old”. And then leave it to the computer to dynamically create the set on some frequency of my choosing. This overlaps with folders. For example I may have a folder called “annotated_images”, of which I further filter to only show the most recent x months.

Pipelines and processes

These definitions may also become more complex. For example, medical experts have a higher cost than entry level people. And running an existing AI has a lower cost still. So I may wish to create a Data Pipeline that goes in the order: AI, Entry Level, Expert.

Arranging purely by date would not be as useful here, since as soon as the AI completes its work, I want the entry level person to look at it. Same with when the entry level person completes their work, and so on.

At each stage of the process I may wish to output a “folder” of the data. For example, say that we start with 100 images that the AI sees. At a moment in time, an entry level person has supervised 30 images. I may wish to take just those 30 images and treat that as a “set”. Of course the moment that a person annotates the 31st image, now the set should have 31 images.

In other words, the stage in the process it's at, the status of the data, and it's relationship to other elements helps determine it's set. To some extent this is like a hybrid of Folders and Filters, with the addition of “extra information” such as statuses.

The implementation of pipelines can sometimes be complex. For example, the label sets may be different.

TK: additional information

Streaming Data for Workflows

We have discussed concepts around perpetually improving models. But how do we actually achieve it? There's a few ideas at play here, I'll unpack each.

Overview of Streaming

The 10,000 foot view goal of streaming is to automatically get human annotations “on demand”. While people often jump to thinking of real time streaming, that's actually a different (albeit related) idea. Instead, think of a project where one person on the team has defined the Label Schema. But the data is not yet ready - perhaps because the engineer who needs to load it hasn't loaded it yet - or perhaps because it's novel data that has not yet been captured by a sensor. While those sound very different - from the training data perspective it's the same problem: the data is not yet available.

The solution to this is to set everything up in advance, and then have the system automatically generate the “concrete” tasks (from the template configuration) when the data becomes available. The overall flow of this interaction is shown in Figure 4.9.

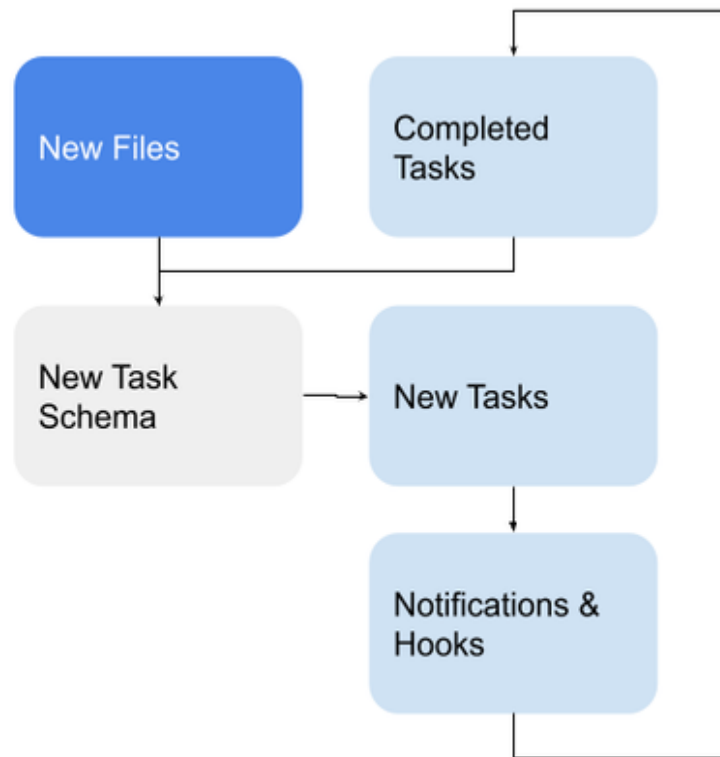


Figure 4-10. Streaming Structure

The Dataset Connection

How do we know when the data is available? Well, first we need to send some kind of signal to alert the system that new data is present. But how do we know how to route the data to the right Template?

Introducing the Empty Dataset (Technically: Abstract Dataset).

Let's jump to code for a moment to think about this. Imagine I can create a new dataset object in Python:

```
my_dataset = Dataset("Example")
```

This is an empty set. There are no raw data elements.

Sending a single file to that set

Here I create a new dataset, a new file, and add that file to the set.

```
dataset = Dataset("Example")
file = project.file.from_local("C:/verify example.PNG")
dataset.add(file)
```

Relating a Dataset to a Template

I create a new Template. Note this has no Label Schema. - it's an empty shell for now. Then I have that template "watch" the dataset I created. What this means is that everytime I update a file to that set, that action will create a "callback" that will trigger task creation to that set automatically.

```
template = Template("First pass")
template.watch_directory(my_dataset, mode='stream')
```

Putting the whole example together

```
# Construct the Template
template = Template("First pass")
dataset = Dataset("Example")
template.watch_directory(dataset, mode='stream')
file = project.file.from_local("C:/verify example.PNG")
dataset.add(file)
```

Here I created a new template (for humans) and the new dataset where I plan to put the data. I instruct it to watch it for changes. I then add a new file to the system - in this case an image. Note that at this point the file exists in the system, in whatever default dataset is there - but in the dataset that I want. So in the next line, I call that dataset object specifically and add the file to it, - thus triggering the creation of a concrete task for human review.

Notes: Practically speaking, many of these objects may be a .get() (eg an existing set). You can target a dataset at time of import (doesn't have to be added separately later). These technical examples follow the Diffgram SDK (Version TBD) which is licensed under MIT open source approved license. Other providers and closed source vendors may have different syntax or feature sets.

Expanding the example

```
template_first = Template("First pass")
template_second = Template("Expert Review")
dataset_first = Dataset("First Pass")
dataset_ready_expert_review = Dataset("Ready for Expert Review")
template_first.watch_directory(dataset_first , mode='stream')
template_first.upon_complete(dataset_ready_expert_review,
mode='copy')
template_second.watch_directory(dataset_ready_expert_review,
mode='stream')
```

Here I create a 2-pass template. The “2” because the data will first be seen by the first template, and then later by the second template. This is mostly reusing elements from the prior examples, with the `upon_complete` being the only new function. Essentially that function is saying “whenever an individual task is complete, make a copy of that file, and push that to the target dataset.”. I then register the watcher on that template like normal.

There is no direct limit to how many of these can be strung together - you could have a 20 step process if needed here.

Non-linear example

```
template_first = Template("First pass")
dataset_a = Dataset("Sensor A")
dataset_b = Dataset("Sensor B")
dataset_c = Dataset("Sensor C")
template_first.watch_directories(
[dataset_a, dataset_b, dataset_c], mode='stream')
```

Here I create 3 datasets that are watched by one template. The point here for organization is to show that while the Schema may be similar, the datasets can be organized however you like.

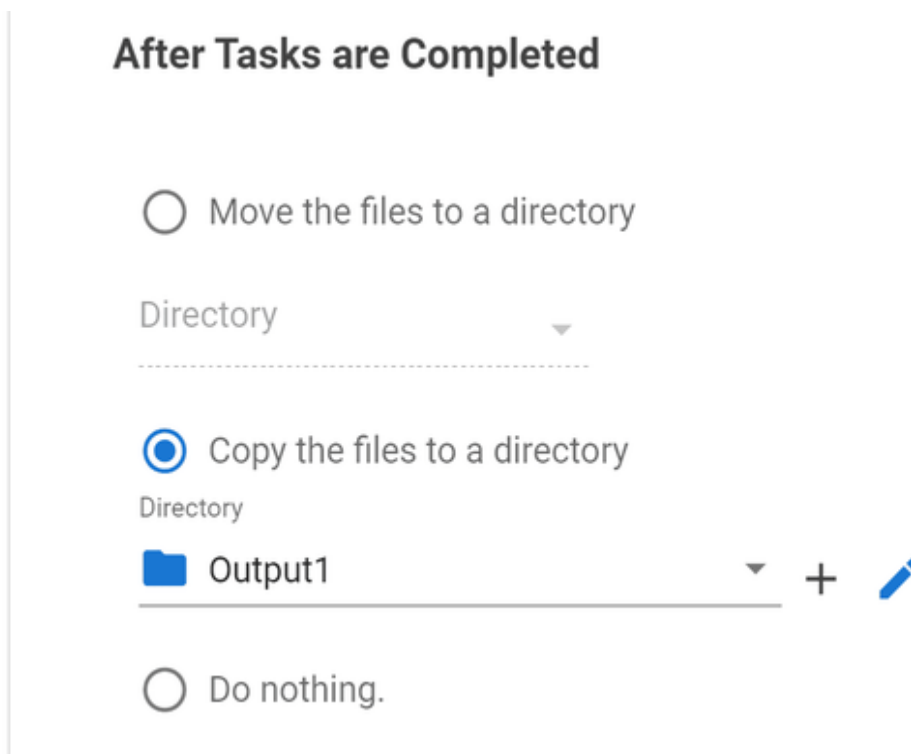
Hooks

For complete control of this process, you can write your own code to control this process at different points. This can be done through registering webhooks, userscripts, etc.

For example, a webhook can be notified when a completion event happens, and then you can manually process the event by (eg filtering on a value, such a number of instances). You can then programmatically add the file to a set. (This essentially expands on the copy/move operation `upon_complete()`.)

User Interface

An example of how to achieve the `upon_complete()` in a User Interface is shown in Fig 4.10.



The image shows a user interface titled "After Tasks are Completed". It contains three radio button options: "Move the files to a directory", "Copy the files to a directory", and "Do nothing.". The "Copy the files to a directory" option is selected. Below this option is a text input field labeled "Directory" containing the text "Output1". To the right of the input field is a dropdown arrow, a plus sign, and a blue pencil icon. The "Move the files to a directory" option also has a "Directory" label and a dropdown arrow, but it is not selected.

Figure 4-11. Task Completion UI Example

Annotation

Are the media types I need supported? Spatial types supported?

In a 2021 survey we asked the question “What is an acceptable learning curve for Annotation Tooling?” as shown in Fig 4.11 65% said “some kind of learning curve is ok, as long as it doesn’t require formal training”. In general this is the target that I see most tooling providers aiming for.



Figure 4-12. Fig 4.11: 2021 Training Data Survey

Depth of Labeling

While there is a growing consensus around some of the “Core” features there is still a wide gap among providers - even in technical documentation.

For example, as shown in [Figure 4-14.12](#), rendering a video like YouTube and asking questions about the entire video is very different from frame specific labeling. Yet both often get shepherded under “Video Labeling”. Explore the depth of support needed carefully for your use case.

This is generally a binary “has it or doesn’t category”. In some cases, a little more depth may be easy to add. Because this area is rapidly changing many tooling makers will be happy to work with you to add the depth here that you desire.

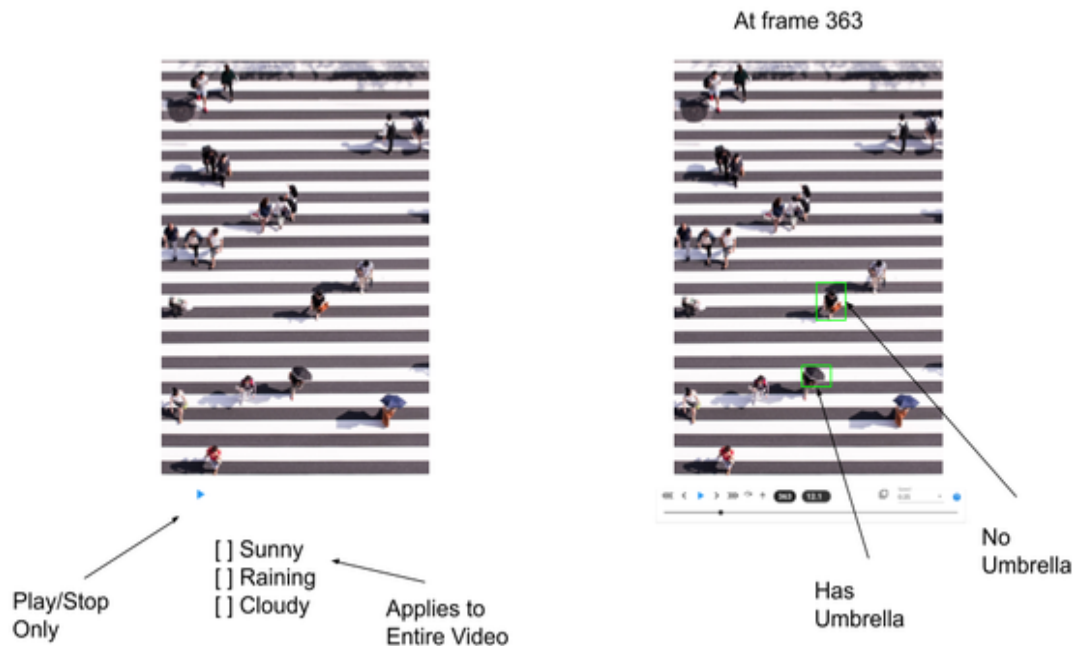


Figure 4-13. Depth of Labeling - Comparison of Whole Video vs Frame
https://unsplash.com/photos/n31JPLu8_Pw?utm_source=unsplash&utm_medium=referral&utm_content=creditShareLink

Do you need to customize the interface?

Most tools assume you will customize the Schema. Some also allow you to customize the look and feel of the UI, such as how large elements or where they are positioned. Others adopt a “standard” UI, similar to how office suites have the same UI even if the content of the documents are all different.

Reasons to customize the interface including wanting to embed it in the application and having special annotation considerations.

Most tools assume a large screen device like a desktop or laptop will be used.

How long will the average annotator be using it?

A simple example is hotkeys. If a subject matter expert is using the tool a few hours a month then hotkeys may not be all that relevant. However, if

someone is using it as their daily job, perhaps 8 hours a day 5 days a week, then hotkeys may be very important.

To be clear, most tools offer hotkeys so that specific example is likely not worth worrying about. More generally, the point is that, by accidents of history or intent, most tools are really optimized for certain classes of users. Few tools work equally well for both casual and professional users. Neither is right or wrong, just a trade off to be aware.

Annotation Automation

One approach is for the tooling to implement specific, known methods for singular tasks. This usually means a fairly constrained problem. Another approach is for general primitives to be provided and for the user to assemble their own singular task method or use an off the shelf singular method based on that general approach.

I'll also present helpful context to the many auto machine learning methods available

Strategy

Speedups are discussed in detail in Chapter 5. Here I briefly discuss the high-level strategies products employ. The major difference in speed up strategy is open vs closed. Open strategies generally provide samples and allow you to edit it to your desired use case.

For example - Diffgram includes the popular “bodypix” model built in - you can run it without any technical knowledge. But if you have a better model, or need to adjust something, your or the data scientist(s) on your team can.

Another approach is to implement specific approaches more “deeply” into the technology. I discuss the pros and cons of this in Chapter 5.

Stream to Training

This is the question “How to get my data out of the system?”. All systems offer some form of export. Is it a static one term export? Is it direct to tensorflow or pytorch memory?

This is different from exporting in that it implies a direct connection.

After Training Data

Finally, to do the actual training and operations of the model, something like Determined AI takes over. At the time of writing there are many hundreds of tools that fall in this category. This category may also become quite broad depending on how you define deployment.

Modeling Integration

At some point we will want to do actual machine learning with our data. While there are many more advanced ways, the go-to choice is often to export a snapshot of the data and train on that. If this sounds too “old school” keep in mind that you can wire this up to be iterative - so the snapshot could be done hourly, daily, etc. For large datasets this can be a “patch” type operation - only updating what’s changed.

Often the more challenging part is on the other side of the pipeline - loading predictions back into the annotation system. For more on Importing, see Chapter 3 Pre-Label Prep.

Model Run Also known as: Predictions

Running a machine learning model on a sample or dataset. For example, given a model X, inputting an image Y, and returning a prediction set Z. In a visual case this could be an object detector, an image of a roadway, and a set of bounding box type instances.

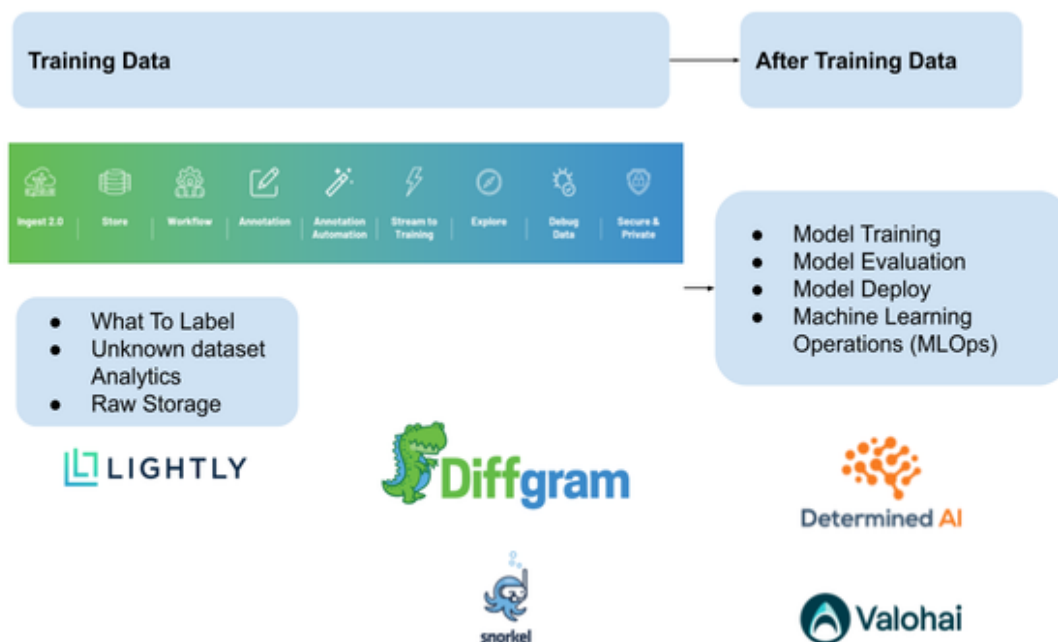


Figure 4-14. 13: Training Data process and tools

Explore & Debug Data

To some extent, explore can be thought of as a “supercharged” version of looking through files on a regular file browser. Typically this means with the aid of various tools designed for this specific domain.

As an analogy consider looking through marketing contacts in a marketing system vs a spreadsheet. You may still see the same “base” data in the spreadsheet, but in the marketing tool it will provide other linked information and offer actions to take, such as contacting the person.

Exploring data includes manually writing queries and viewing it. It can also include running automatic processes that will walk the data, such as to discover insights or filter the volume of it. It can be used to compare model performance, debug the human, and more.

While it’s possible to do some limited exploration by manually looking at raw data in a regular old file browser, that’s not really what I’m referring to here.

To explore the data it must have already been ingested into a training data tool. Data exploration tools have a lot of variance, especially around proprietary automatic processes and depth of analysis.

The data can often be viewed as a set of samples or a single sample at a time.

An important consideration when reflecting on exploration:

1. The person who does the exploration may, or may not, be involved in the annotation process. More generally, the person doing the exploration may be different from the person who did any other process, including uploading, annotation etc.
2. The organization and workflow of data to do annotation may not be useful to someone using the data for actual model training.
3. Even if you are directly involved in all of the processes, the data is separated by time. For example, you may conduct a workflow for annotation over a period of months, and then on the third month do further exploration, or may explore the data a year later etc.

To put this concretely, if I'm concerned about annotation I care about what's the status of "batch #7"?

When I'm exploring the data, I may want to see the work of all batches 1-100. At that point, I don't necessarily care what batch was created, I just wanted to see all examples of some label. Put more broadly, it's in part a different view of the data, that crosscuts multiple datasets.

Put simply, the exploration process may be separate by time and space from annotation.

TK: additional detail around the following concepts

Exploration can be done at virtually any time.

1. You can inspect data prior to annotation of a batch, such as to organize where to start.

2. You can inspect data during annotation to do quality assurance, to simply inspect examples, etc.

Generally, the goals are to:

1. Discover issues with the data
2. Confirm or disprove assumptions
3. Create new slices of the data based on knowledge gained in the process

The basic explore loop

1. Run some process
2. Take some action

Typical explore processes

1. See or hear the data first hand

Typical explore actions

1. Flag a file or set of files for further human review. For example, missing annotations.
2. Generate or approve a new novel slice of the data. For example a reduced dataset that may be easier to label.

Similar Image Reduction

If you have many similar images you may want to run a process to reduce it to the ten percent most interesting. The key difference here is often it's an unknown dataset, meaning there are few or no labels available yet.

I used to think of this as a “data prep” step. However, I have realized it makes more sense to think of getting all your data to your training data

tooling as the first ingest step, and then further processing it from there. Really, this step is automatically exploring the data, taking a slice of it, and then presenting that slice for further processing.

At the time of writing, doing this type of stage is definitely considered more of an “advanced” step. Most training data tools already tackle the basic pre-processing needed to get raw data in workable formats. Currently, there are very few tools available that truly fall in this category.

Often the organization methods necessary during the creation and maintenance of training data are less relevant to the creation of models. The people who create the training data, may not be the people who create the datasets. And again, others may take the sets and actually train models.

Can I access a slice of the data without downloading it all?

Can I access the data without installing any additional tools?

Can I compare the model data without involving a data scientist?

Using the Model to Debug the Humans

A key additional aspect to importing data is tagging which “model run” the instance’s belong to. This is to allow comparison, e.g. visually as shown in Fig X, between model runs. It can also be used in Quality Assurance. We can actually use the model to debug the training data. One approach to this is to sort by the biggest difference between ground truth and predictions. In the case of a high performing model, this can help identify invalid ground truth and other labeling errors.

Fig 4.14 Example of a model prediction (Solid) detecting a car where the ground truth was missing it (Dashed). This type of error can be automatically pushed to the top of a human review list because the box is far from any others. An example algorithm is comparing the nearest Intersection over Union (IoU) to some threshold, in this case it would be very high since the box hardly overlaps with any of the green ones.

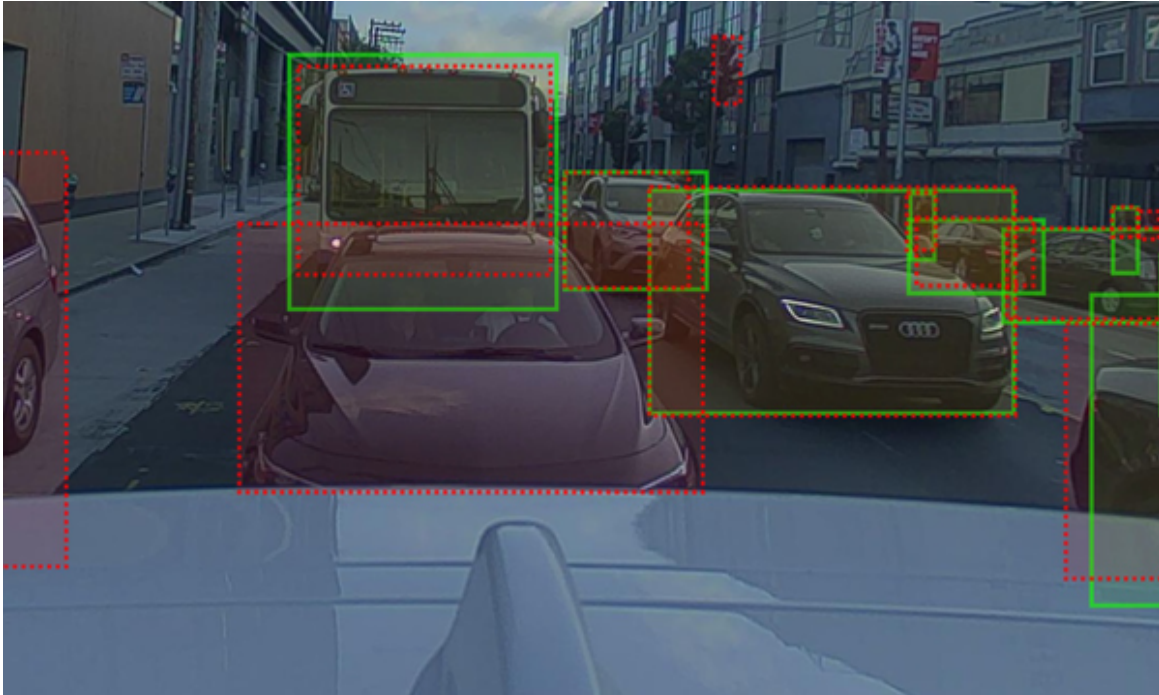


Figure 4-15. Example of a model prediction (Solid) detecting a car where the ground truth was missing it (Dashed).

https://dashboard.scale.com/nucleus/ds_bwhjbyfb8mjj0ykagxf0/di_bwhjbzzb8mjj0ykagzzg

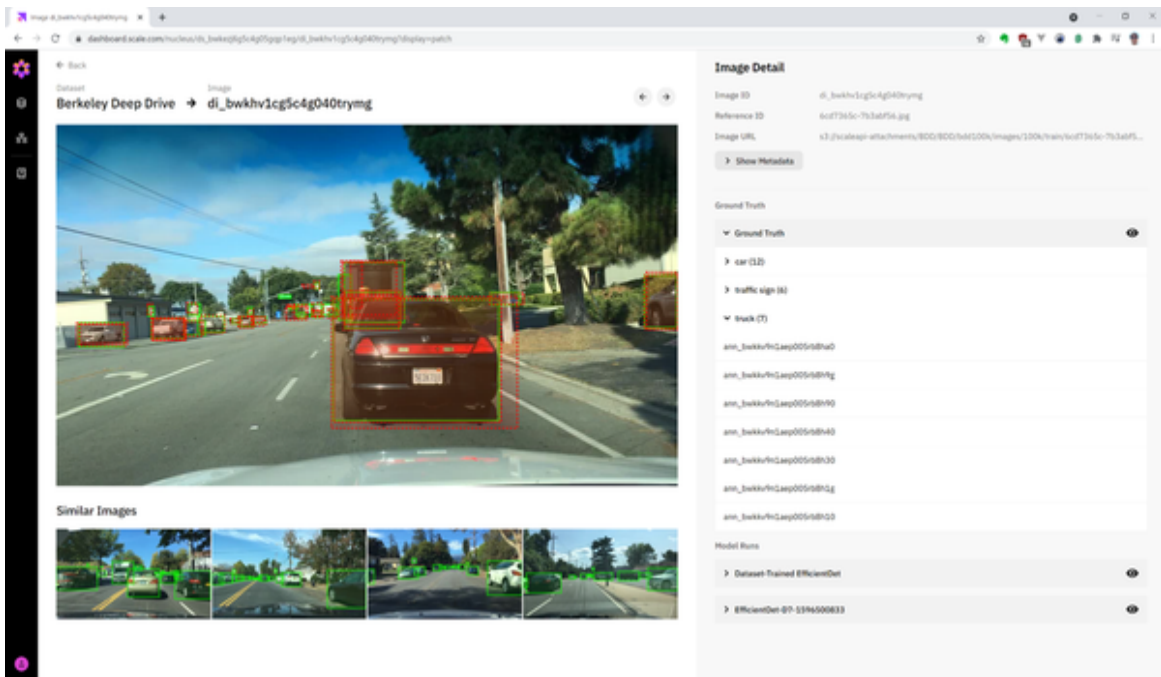


Figure 4-16. Example of Interface Showing Modeling Integration - Comparison of 2 Models and Ground Truth

To further help picture the relationships here, consider that the ground truth changes at a slower frequency than the model predictions. Sure errors in ground truth may be corrected, more ground truth added etc. But for a given sample, in general the ground truth is static. Whereas we expect during development that there will be many model runs. Even a single automatic process (AutoML) may sample many parameters and generate many runs.

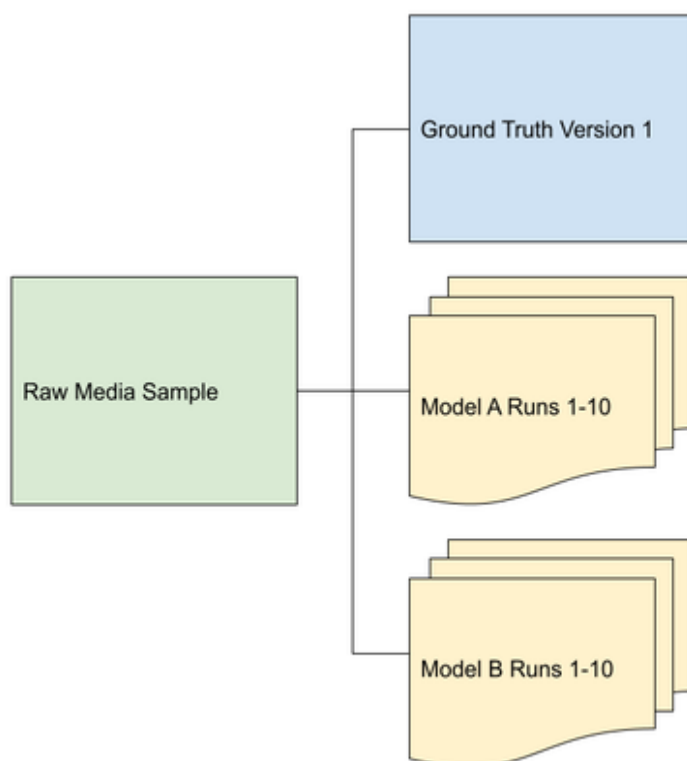


Figure 4-17. Relation of Raw Media to Models and Ground Truth Instance Sets

A few practical notes to keep in mind here. There is no requirement to load all the model predictions back into the training data system. In general, the training data is already used in a higher level evaluation process, such as determining the accuracy or, precision, etc. For example, if there is an AutoML process that generates 40 models and identifies the “best” one, you could filter by that and only send the best predictions to the comparison system.

Similarly, A Ground Truth set is not strictly required either. For example if a production system is predicting on new data there will not be ground truth available. It can still be useful with a single model to visually debug it this way, and it gives flexibility for other cases such as shipping a new version of the model, and wanting to spot check it by running both in parallel during development efforts etc.

A model is not a model run

A model is typically the raw weights, also known as - the direct output of a training process. WhereasWhere as a model run adds context, such as what settings (e.g. resolution, stride) were used at runtime. Therefore it's best to think of each model run as a unique ID. Because of course a model with the same structure but trained differently (data, parameters, etc) is different. And a static model is still unique because the context (e.g. resolution or other prep) may have changed, and the model "runtime" parameters (e.g Stride, internal resolution setting etc) may have changed. This can get context specific very quickly. In general it avoids a lot of confusion to have a unique ID for each set of {model, context it runs in, settings}.

Dataset is not related to Model

A few potentially subtle distinctions to be aware of here.

There is no strict relation or hierarchy to a "Dataset" and a "Model" - it's better to think of it as a per sample level. The reason for this is that, for example, a model version may have been run on Sample 1 but not Sample 2. And both samples may be in the same dataset!

A set of predictions is not really a dataset

A model run may generate a set of predictions, - but calling this a dataset kind of misses the point - because the real dataset is the amalgamation of all of the predictions as well as the ground truth, etc. And generally this structure is defined more by other needs than what sample or batch of samples happened to have been run. While in some contexts this difference

may be mincing words, in my opinion it's best to reserve the "Dataset" concept in this case for a real, "complete" dataset, not a partial artifact of some process.

TK: more about getting data to training systems

TK: Secure & Private Data

Content TK.

Summary

Content TK.

Chapter 5. Annotation Automation

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 5th chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at jleonard@oreilly.com.

Introduction

Too much work? Is annotation tedious? Quality poor?

In this chapter I show you how to solve these problems with automations.

The first topic is Pre-Labeling - the idea of running a model before annotation. I’ll cover the caveats you should be aware of and step through extensions of the idea such as ‘micro-model’ labeling.

Next, Interactive Automations are when a user adds information in order to help the algorithm. For example drawing a box to automatically get a tighter location marked by a polygon. Interactive improvements are often most relevant to spatial locations - however that’s just the start. The end goal of Interactive Automations is to make tedious UI work a more natural extension of human thought.

Quality Assurance (QA) is one of the common uses of training data tools. I cover exciting new methods like using the model to debug the ground truth. Other tools automatically check base cases and look at the data for general reasonableness.

Pre-Labeling, Interactive Automations, and QA tools will get you far. After covering the foundations, I’ll walk through key aspects of data exploration and discovery. What if you could query the data and only label the most relevant parts? This area includes concepts like filtering an unknown dataset down to manageable size and more.

There are many domain specific options that can be implemented as well. For instance, if you have multiple sensors, you can get a “two for one” deal by labeling one sensor, and using known geometry to estimate into the other sensors. Incorporating interpolation and object

tracking can speed up Video. Using dictionaries and heuristics can help with text. I'll walk through all of these at a high level so you are confidently aware of them.

I will touch on data augmentation, common ways it's used, and cautions to be aware of. When we augment data, we derive new data based on the existing base information. From that viewpoint it's easier to think of the base information as the core training data and then the deriving process - augmentation - as a machine learning optimization. So while a portion of the responsibility here is outside the scope of training data we must be aware of it.

Simulation and Synthetic data have situation specific uses but we must be upfront about the performance limitations.

Overall, there are many high quality approaches that will substantially automate your training data process. There are also limitations and risks to consider when implementing automation. I will provide a high-level overview of the most common methods you can actually use today, what results to expect, the trade-offs involved, and how they work together. For the most popular methods I will take a deeper dive into some of the specifics and for the rest I will provide a general introduction.

We have a lot to unpack and experiment with in this chapter. Let's get started by taking a closer look at the project planning process and techniques that are commonly used today.

Getting Started

The motivation to use automation approaches are the problems encountered working with Training Data. From high labor costs, lack of available people, tedious work, or even cases where it's nearly impossible to get enough raw data.

Some automations are more practical than others. I outline what people actually use for most projects. Followed by what kind of results you can expect and not expect. This is rounded out with the two most common confusions about automations - fully automatic labeling and proprietary methods.

I wrap this section by looking at the costs and risks. This section is a view of how the concepts map together, and ultimately to how they actually help your work. It can also help direct your reading and act as a reference to quickly look up common solution paths.

Motivation: When to use these methods?

Here I state the typical problem that leads people to seek the following method.

*T
a
b
l
e
5
-
I
.
R
e
f
e
r
e
n
c
e
o
f
P
r
o
b
l
e
m

a
n
d
c
o
r
r
e
s
p
o
n
d
i*

n
g
A
u
t
o
m
a
t
i
o
n
S
o
l
u
t
i
o
n

Problems	Solutions
----------	-----------

Too much routine work	Pre-label
Annotations too costly	
Subject matter expert labor cost too high	

Data annotated has a low value add	Pre-Label
	Data Discovery

Spatial annotation is tedious	Interactive Automations
Annotator day to day work is tedious	Pre-Label

Annotation quality is poor	Quality assurance tools
	Pre-label ^a

Obvious repetition in annotation work	Pre-label
	Data-Discovery

It is near impossible to get enough of the original raw data Simulation & Synthetic

Raw data volume clearly exceeds any reasonable ability to manually look at it. Data Discovery

^a To avoid tedious work and reduce annotator fatigue

Note: Check if the method is designed to work with Schema or Location

Some automations work on both the schema (label, attributes) and spatial location (box, token position). Some work on only one or the other. For example, object tracking is generally oriented towards spatial and doesn't usually help with meaning. When it matters I will highlight this difference, otherwise it usually comes down to the implementation.

As an example of this impact, a method that offers a 2x improvement on the spatial location will be of little importance if 90% of the time is in the Schema - defining the meaning of what is actually present. This is relevant to all automations that impact annotations directly.

What do people actually use?

With so many new concepts and options it can be easy to feel overwhelmed.

While these methods are always changing here I highlight the top methods I have seen people use successfully and are the most broadly applicable across all projects. The two questions I will most try to answer:

1. What are the best practices?
2. What do you need to know to be effective and successful with training data automations?

Most projects usually can use these techniques:

These approaches may be commonly used however it does not mean they are easy, always applicable, etc.

You must still have sufficient expertise to use the approaches effectively - expertise you will gain in this chapter.

1. Pre-labeling¹
2. Interactive automations

3. Quality assurance tools
4. Data discovery
5. Augmentation of existing real data.

Domain Specific

These techniques depend on the data and sensor configuration. They can also be more costly, or require more assumptions about the data.

Special purpose automations, like Geometry, Multi-sensor approaches.

Media specific, like Video Tracking and Interpolation

Simulation & Synthetic

A note on Ordering

In theory there are some order to these methods, however, in practice these methods have very little universal order of usage. You can pre-label as part of a data-discovery step, during annotation, or much later to grade production predictions. Sometimes data discovery is only really relevant after some significant percent of the data has been labeled. Therefore, I have put the more common methods towards the start of the chapter.

What kind of results can I expect?

Being a new area it's common for there to be few expectations about what these methods can do. Sometimes people invent or guess at the expected results. Here I unpack some of the expected outcomes of using these methods.

1. First, is the method itself
2. Next is the expected result when well implemented
3. And finally the "Is not" covers the most common issues and confusions

*T
a
b
l
e*

*5
-
2
.
E
x
p
e
c
t
e
d*

*r
e
s
u
l
t
s
f
o
r
e
a
c
h*

*a
n
n
o
t
a
t
i*

o
n

a
u
t
o
m
a
t

i
o
n

t
e
c
h
n
i

q
u
e

Method	Does:	Does not:
Pre-Labeling	Reduce the less meaningful work Usually within a single sample - e.g. an image, text, or video file. Shifts the focus to correcting odd cases A building block for other methods.	Solve labeling entirely. A human must still review the data and do further work with it.
Interactive Automations	Reduce tedious UI work. For example, drawing a box, and getting a polygon drawn tight around an object in the box.	Completely eliminate all UI work.
Quality Assurance Tools	Reduce manual QA time on ground truth data. Discovery novel insights on models.	Replace human reviewers entirely.
Data Discovery	Makes human time focused on the most meaningful data.	Work well in cases where there is already a well functioning model and the main goal is grading it

Avoids unnecessary similar annotations.

Augmentation	Provide a small lift in model performance. Work without having base data.
--------------	---

Simulation & Synthetic	Cover otherwise impossible cases. Provide a small lift in model performance.	Work well for cases where raw data is already abundant or situations are relatively common.
------------------------	---	---

Special purpose automations, like Geometry, Multi-sensor approaches.	Reduces routine spatial (drawing shapes) work by using geometry based projections. Some limited methods can work largely independent of humans, others usually require labeling of the first sensor and fill the rest.
--	---

Data type specific, like Object Tracking (Video), Dictionary (Text), AutoBorder (Image).	Reduces routine work by utilizing known aspects of the media type.
--	--

Common Confusions

Before we dive into this let's address two of the most common confusions.

Fully Automatic Labeling

The myth: "We don't need labeling, we made it automatic."

When an existing AI predicts something that is a form of automatic labeling. In this chapter we are concerned about how to create that AI model in the first place. Meaning some form of non-automatic labeling first. If someone was to ever solve truly automatic labeling of arbitrary data they will have created the elusive idea of Artificial General Intelligence. While this may eventually happen it is out of scope of commercial concerns today.

Proprietary Automatic Methods

Sometimes true, sometimes myth: "Use our method for 10x better results"

The intent of this chapter is to convey the general methods and concepts that are available. As you will see, as these methods stack with each other, it is possible to get relatively dramatically better results than purposefully going as manually and slowly as possible.

However, there are a vast number of concrete implementations. The single most common theme I have seen with secret vendor specific approaches is that usually they are very narrow in scope - for example it may work for only one type of media, one distribution of data, one

spatial type etc. It can be difficult to verify in advance if your use case happens to meet those assumptions. Statistically it's unlikely.

The best way around this is to:

Be aware of the general approaches as explained in this chapter.

Trend towards tools that make it easy to run the most up-to-date research and your own approaches.

Risks

All automations, even those operating as expected, introduce risk. Here are a few that specific to annotations.

Lack of Net Lift

This is when automation doesn't actually help.

This is surprisingly common, especially ones that involve user interface.

Worse Results

Automations can make results worse.

For example the super-pixel approach can lead to blotchy annotations that are less accurate than a traced polygon. Don't assume the failure state is equal to manual annotations.

Cost Overruns

As detailed in the next section automations have many costs, from time to implement, hardware cost, human training cost etc.

Method Specific Risks

Every method is unique and comes with it's own unique risks.

Even seemingly similar methods can have different effects, for example correcting a pre-labeled box is different from a pre-labeled polygon, and different again from an attribute classification.

Treating it too Casually

Sometimes these automations feel like just a little time saver, or some kind of obvious thing.

We have to remember that every automation is like a mini system. That system can have errors and cause problems.

Costs Expected

Automations can sometimes feel like magic.

Wouldn't it be nice if the computer could just do x for us? This magic feeling often seems to shroud some of these methods in an almost religious zeal.

My point with this section is to help you evaluate the total costs of the automations. This is especially important because all automations have costs and some are proposed with the most casual air.

In cases outside of training data a single automation may have a big planning team, documentation, risk analysis, etc.

Yet somehow here, there seems to sometimes be an expectation that these automations will just work. It's almost like we expect there to be magic, and so we quickly lose our rational thinking cap and just keep paying since any price appears fine as long as it finally works it's magic.

Setup Costs

Virtually all automations require some form of setup.

Even the ones with the most minimal of setup require a degree of training and understanding around their assumptions.

Scope

Imagine you have a lawn to mow. It's a small front lawn about the size of a parking spot. You can get a push mower or a ride-on-mower. The ride-on, being about as big as the parking spot, is so effective it can pretty much just start up and instantly cover the entire lawn.

Naturally the ride-on-mower is orders of magnitude more expensive than the push mower, it requires storage, supplies, gas, maintenance, etc. So while once in place it will mow the lawn the fastest, it's startup costs and ongoing costs far outweigh the benefits.

Setting a good scope of automations is important. A few questions to consider:

1. Does the automation require technical integration? Can it be done from a UI or through a wizard?
2. What are the expected startup and maintenance costs?
3. Is an off the shelf method adaptable to our needs? Do we need to write a specialized automation?

If this sounds like a project management discussion that's good - because that's what each automation is - a project. Obviously a giant lawn mower is too large for a small lawn. So what are the right sizing metrics for training data?

Compare to Best Practices not manual annotation

Completing an entire project with 100% manual annotation is rare. By manual annotation, I mean no model review, no pre-labeling, no UI assistance features, none of the methods in this chapter. Yet automation methods are often presented as a percent savings relative to this mythical 100% manual annotation.

Instead of thinking of it as a comparison to manual annotation, think of it as comparing to best practices. And best practice is to use these automation methods appropriately.

Benchmark questions:

1. Are we reasonably aware of all commonly available methods? Of the ones relevant to our case, what percent of them are we using?

For example if there are four methods available and you are using all four then at the strategic level you are already there.

1. By survey, how unique do annotators feel the work they are doing is? Do they feel they are making small adjustments or adding foundational value with every annotation?

This can still be quantitative - but the point is to get away from lower level metrics that are easily gamed or noisy.

1. What is the ROI of our automation methods?

For costs, consider all supplies (e.g. hardware), vendor costs, administration and data science costs. For the benefit, if possible try the actual next best alternative. This requires being clear eyed about the comparison.

Account for Correction Time

For example, if a spatial speed up method is being used, try just drawing the polygon points directly by tracing the object. When you account for correction time, which method was actually faster.

I have seen research papers that reference “clicks”. They say something like: We reduced the number of clicks from 12 to 1 therefore it’s 12 times better. But in reality, there is another tool that allows you to trace the outline, meaning that there are no clicks, and it’s just as fast to trace the outline as it is to fiddle with the errors the automation makes.

T
a
b
l
e

5
-
3
.
E
x
a
m
p
l
e

c
o
m
p
a
r
i
s
o
n

o
f
t
i
m
e

t
o

d
r
a

w

v

s

t

i

m

e

t

o

c

o

r

r

e

c

t

Tool	Time to Draw	Time to Correct
Trace Outline	17 seconds	N/A
Magic Tool Automation	3 seconds	23 seconds

As you can see in this example the magic tool still takes some time to draw. In this case it takes more time to correct. Meaning that while it appears to be faster it's in fact slower.

Subject Matter Expert Required

It's also worth understanding that virtually all of these methods still end up requiring a similar level of subject matter expertise. An analogy is that collectively the tooling represents something akin to a word processor. It saves you having to buy ink, to mail letters, etc. But it doesn't write the document for you. It automatically formats the characters you instruct it to write.

The challenge with automation methods here is that

The startup costs and ongoing costs of setting up these automations is unknown or hard to predict

The good news is that many of these improvement methods stack. The bad news is that most of the methods make a variety of assumptions, either about the data, your level of effort, etc.

Sometimes people discussing these methods will speak to savings relative to 100% manual annotation.

It's worth saying right from the get go that 100% manual annotation is rare.

Essentially, these automations over time appear to be shifting from being thought of as "savings" and more just "standard expectations".

This is not meant to take away the magic from some of these methods. It's quite incredible how quickly a dataset can be improved by careful combination of these methods.

Even today, instead of thinking of these as savings, think of it more as "making what would otherwise be impossible, possible".

Let's dive in!

Pre-Labeling

Pre-Labeling is usually focused on labels *within* a sample. For example bounding boxes or segmentation masks in an image. Pre-Labels can also be used for data discovery, see the data discovery section for more.

TK: to reduce routine work

Standard Pre-Labeling

Let's imagine we have a model that is sort of good at predicting faces. It usually gets good results, but sometimes fails. To improve that model we may wish to add more training data. The thing is - we already know that our model is good for most faces. We don't really want to have to keep redrawing the "easy" examples. How can we solve this?

Pre-Labeling to the rescue! Let's unpack this diagram Fig 5.1. First some process runs, such as a model prediction. Humans then interact with it. We then typically complete the "loop" by updating our model with the new data.

Initially the user reviewing the data is shown all 5 predictions. A user may directly declare a sample to be "valid" - or the system may assume that any samples not edited are valid by default. However it's marked, the net result is the second stage - 4 correct examples and 1 incorrect that needs to be edited.

The general theme of this method is that over time it focuses the human user on the hard cases, the "net lift".

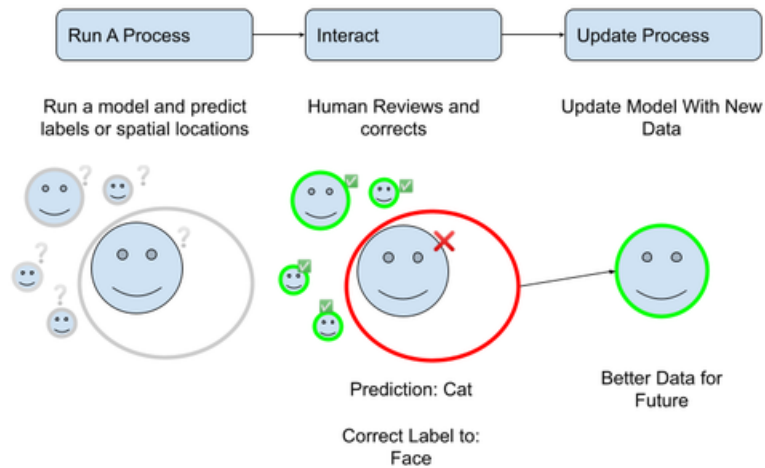


Figure 5-1. Pre-Label Overview

Benefits

Why is pre-labeling so popular? Well there's a few benefits

1. It's fairly universal - it's not tied to any specific method, and can work across many domains
2. It's relatively easy to implement and understand.
3. It indirectly acts as a quality assurance control. People are looking at the actual predictions. This relationship to the actual model can be very valuable.
4. It works on both meaning and spatial.

Caveats

Requires an existing relevant model

The primary caveat is that you need an "initial" model. In general, pre-labeling is a "secondary" step in the process. This can be offset that some methods and contexts require very small amounts of data - for example maybe even a handful of images and you can start - but it's still a blocker. Transfer learning does not solve this - but it does help reduce how many samples are needed to start using this new model.

Can introduce new errors modes

Instead of looking at data with "fresh eyes" it may be easy to miss something that "looks good" but actually is wrong.

Sometimes can be slower

Editing may be slower than creating a new thing for some cases. If a model is predicting a complex spatial location and it requires lots of time to correct, it may be faster to simply draw it “right” from the get go.

Requires some setup or reliance on “built in” methods

While some tools offer built in learning in general it’s best to use our existing model. What’s the point of “correcting” a model that’s not actually going to be used? The downside is that wiring up the connections (to import to the annotation tool) can take a bit of setup time.

Technical Setup Notes

TK

Micro Model Pre-Label

Pre-Labeling with a different model.

An extension of pre-labeling is to use a model to predict something it knows, and use it as a starting point for predicting something unknown. Essentially this divides a responsibility (eg spatial) into some existing model that we don’t plan to update, and focuses our effort on adding new information.

The micro model gets discarded - it’s only used within the scope of annotation automation - saving time on the dimension it’s good at.

Usually a micro model is not trained appropriately to be used as the actual primary model.

For example, imagine we have a face detector that’s great at predicting faces - but doesn’t know anything else. We can run that model first, get the spatial locations of the faces, and then add the new information like happy, sad, etc.

While it may seem subtle, it’s a big difference from standard pre-labeling. We aren’t specifically trying to correct the spatial locations, we are strictly using it as a time saver to get to labels and attributes.

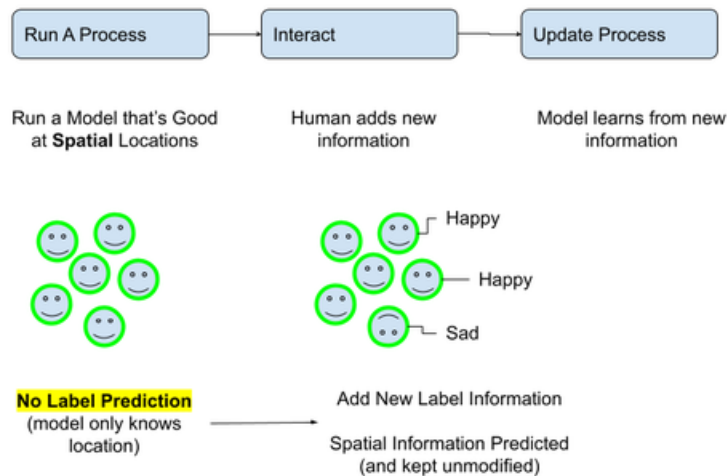


Figure 5-2. Caption TK

Imagine a sports broadcast of a basketball team - identifying and tracking every player in each frame manually would be a lot of annotation work. There are fairly good “people detectors” that can run, tracking algorithms etc. These are responsible for the spatial location. Then the human annotation can be focused on the relevant items - like what action the players are doing - things that a generic tracking/detector algorithm don’t provide.

Benefits

Use off the shelf models

Because our goal here is to add new information (not to correct an existing model) - we can use off the shelf models. This means that even if we don’t have any training data for our Happy/Sad model, we can get started with an existing face detector. This can mean faster setup time. Also because we aren’t directly concerned about improving the model, if built-in models are available, they can be used directly without drawback.

Clear separation of concerns

Either it helps speed up the process - or it doesn’t.

Caveats

1. An off the shelf model may not exist - or may require upgrading to be useful
2. Usually not directly helpful for improving spatial detections
3. Usually not very interactive - fairly static.

The “One step early trick”

Speaking to this idea more generally, it’s the concept of taking something and getting a prediction “one step early”.

1. For example, generic object detector without knowing the class.
2. Generic “event” detector without knowing the class

Typically this is combined with some other method, eg pre-labeling.

Quality Assurance Pre-Labeling

While technically not an automation I must mention one of the most common use cases for training data - quality assurance. Essentially this means loading existing predictions, often from production models, into a system for correction.

What’s the difference between model Quality Assurance and Pre-Labeling?

For Industry type use cases, the main difference is just the intent behind it. There is a similar technical process to load the data, the actual correction mechanisms are usually fairly similar in the UI.

With Quality Assurance generally the intent is to still improve future models, and this is again similar to pre-labeling.

How to get started Pre-Labeling

1. Generally the cycle looks like:
2. Identify your existing models
3. Map the data from the model to the training data tooling
4. Create tasks using the existing data
5. Annotate
6. Export and use the data to retrain new models.

Interactive Annotation Automation

Interactive automations to reduce otherwise tedious UI work - eg where it’s clear to the human what is correct and it’s just trying to get that data understood by the system faster.

Introduction

When a user interacts and adds information in order to help the algorithm. In general they can make otherwise tedious UI work a more natural extension of human thought.

If I as the user provide the interaction first, then we can get “interactive” speed up approaches.

Consider wanting to get polygon - semantic segmentation - data for complex shapes. Instead, what if I could just tell the computer what area of the image I’m looking at, and it would figure out the shape?

Another assumption is that the user is standing by ready to correct. So it’s not as if the user does something, then moves on, rather that the user “reviews” the output, and modifies it accordingly.

The key test compared to other methods is that it should be *impossible* for the method to run, without some kind of user input. Whereas say full image or full video methods can be run as a background computation without direct user input.

The crucial difference here is that it’s usually not the model of interest actually running. In fact many of these methods work without any machine learning at all. Popular examples include

Box to Polygon (Grabcut, DXTER)

Tracking algorithms

Contrast, brush, etc.

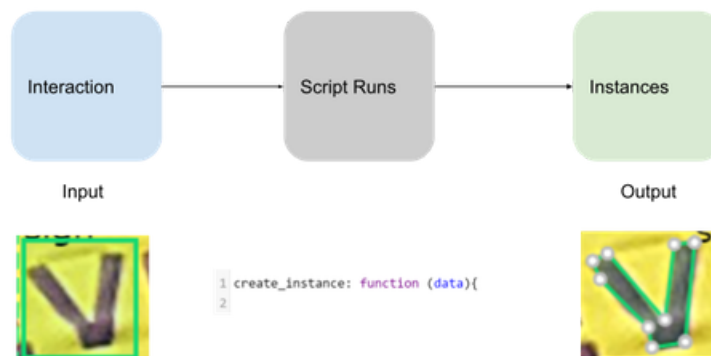


Figure 5-3. Caption TK

Pros

For complex spatial locations this can be extremely effective.

It usually does not require a trained model, or an existing model.

Caveats

Can sometimes be “recreating the wheel”. Eg if you have a model that’s good at detecting people - it’s probably better to use that to start then to manually identify each person (even if

it's a single click).

Often requires more user training, patience etc. Sometimes these methods can take so long to run that they don't really save much time.

To really get these methods working well, it often requires some form of engineering work over and above wiring input/output.

The big idea about creating your own

Some tools provide integrated methods, or off the shelf approaches. For those tools, you may not have access to modify how they work. In those case, the following examples will serve as a guide to the inner workings behind the scenes.

Open Source Diffgram provides a compiler to enable you to write your own automations. This means you can use your own models, the latest open source models, adjust parameters to your desire, etc.

Technical Setup Notes

How to try this yourself

For the scope of these examples I'm using open source Diffgram's Automation library. You can download and try it yourself at: <https://github.com/diffgram/diffgram>. *** The specific reason I'm including Diffgram here is that it includes a compiler that can make these custom things work.

The example code is all in Javascript.

In most cases for the sake of brevity most of the examples are pseudocode. Please see the linked examples for full executable code examples.

Interactive on Drawing Warm up

TK

What is a Watcher? (Observer Pattern)

Any time a user does something, such as creating an annotation, deleting an annotation, changing labels etc. we think of it as an event.

We do so at a semantically meaningful level for training data.

For example, a `create_instance` event is more meaningful than a regular `mouse_click` if I want to do something - such as running a model - after the user creates the instance.

How to use a Watcher:

We must first define a function that will do something upon a `create_instance` event.

```
create_instance: function (data){ // your function goes here }  
create_instance: function (data){
```

```
    console.log(data[0])  
  }  
}
```

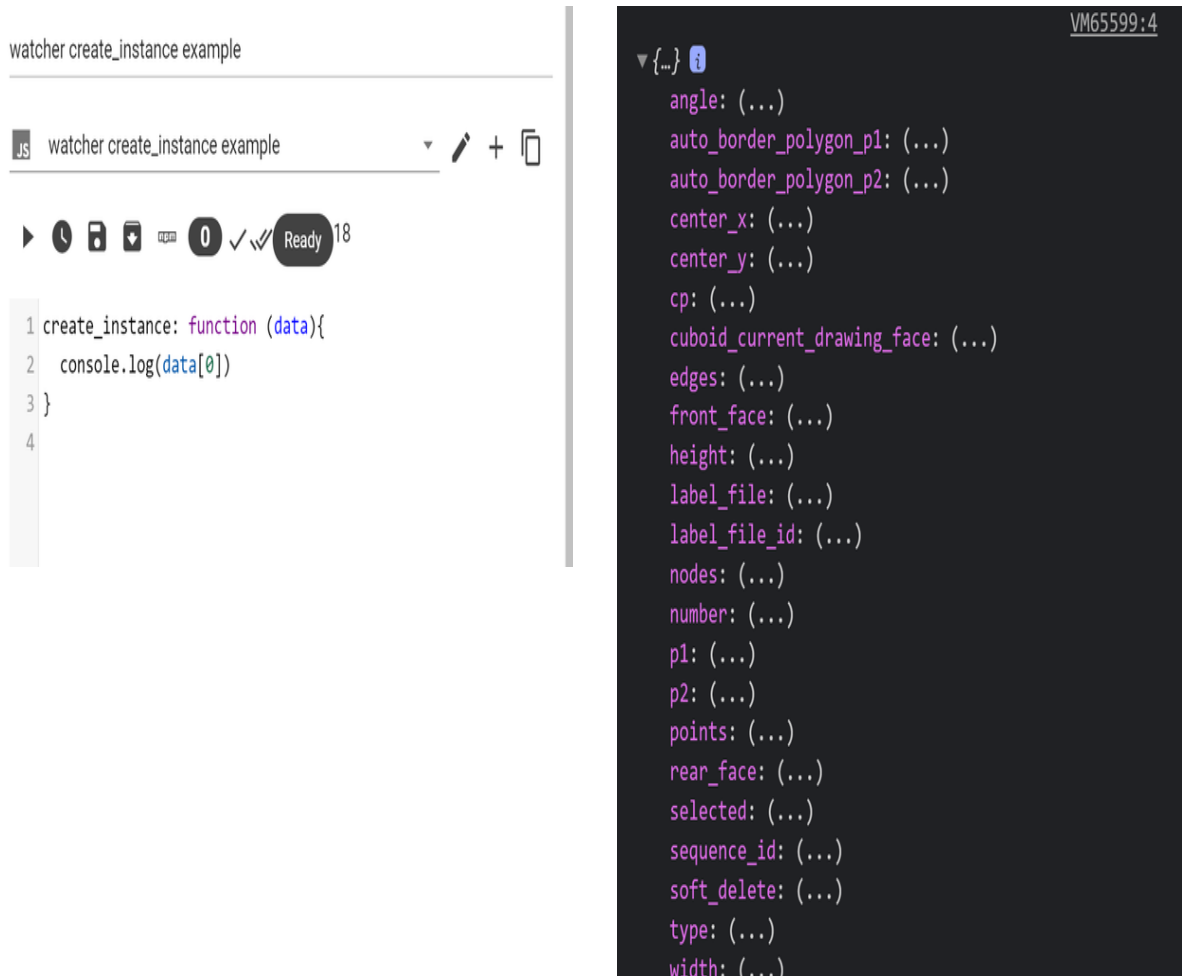


Figure 5-4. Example of Automation Interaction UI

After we enabled this script, and the user draws an example annotation, we see the example output in Fig X.xxxx

IMAGE TO COME

Figure 5-5. TK: Fig X.xxxx Console - Example of Logged Members of an Annotation

Interactive Capturing of a Region of Interest

Here is an example of code that captures the canvas region of interest based on a User's annotation. While this may seem fairly simple, it's the first step towards running an algorithm based on just the cropped area.

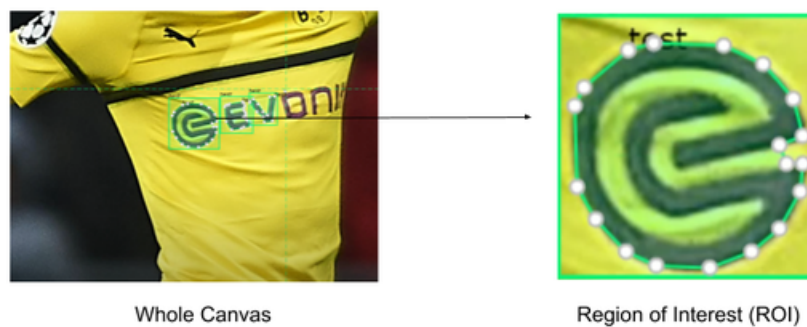


Figure 5-6. Example of Whole Canvas Vs Region of Interest

The code to capture this looks like this:

```
create_instance: function (data){
  let ghost_canvas = diffgram.get_new_canvas()
  let instance = {...data[0]}
  let roi_canvas = diffgram.get_roi_canvas_from_instance(
    instance,
    ghost_canvas)
}
```

More generally the idea here is to use some input from the user to pre-process what we feed to our model.

Interactive Drawing Box to Polygon Using Grabcut

Now, this example will take our region of interest canvas, run the standard open cv grab cut algorithm and output polygon points, which will then be converted into a human editable annotation.

Crucially, you can imagine replacing grabCut() with a preferred algorithm of your choice.

Simplified pseudocode:

```
let src = cv.imread(roi_canvas);
cv.grabCut(src, //args) // replace with your model or preferred algorithm
cv.findContours( //args) // because it's a dense mask
cv.approxPolyDP( //args) // reduce to useful volume of points
points_list= map_points_back_to_global_reference() // because roi is local, but we
need the location relative to whole image
diffgram.create_polygon(points_list)
```

[Full Example](#)

Full Image Model Prediction Example

[The full example is available here.](#)

Zooming out from the directly interactive example, we can also run file level models. You can imagine having a series of this models available, and the user perhaps choosing at a high level which one to run.

The high level idea is to get the information from the user interaction, the raw media, and then run the automation.

An example in Javascript could look something like this

```
this.bodypix_model = await bodyPix.load()
let canvas = diffgram.get_new_canvas()
let metadata = diffgram.get_metadata()
segmentation = await this.bodypix_model.segmentPerson(canvas, {});
Points_list = get_points_from_segementation()
diffgram.create_polygon(points_list)
```

Example - Person Detection for Different Attribute

Now, going back to a more simple example, we can also use these automations to simply run models.

Here, we used the BodyPix example in a diffgram userscript. It ran on the whole image and segmented all the people. Here we now select the person to add our own attribute “On Phone?”. This is an example of using a model that’s not directly our training data goal to avoid having to “recreate the wheel” for parts that are well understood. You can imagine swapping this model for other popular ones, or using your own models for the first pass here.

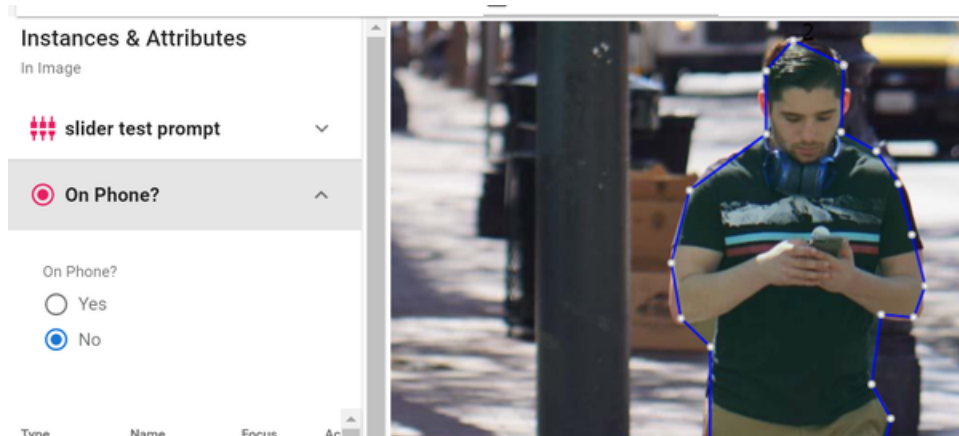


Figure 5-7. Caption TK

How to get started with Interactive

TK

Quality Assurance (QA) Automation

Novel Training Data QA tools to debug ground truth data and reduce Quality Assurance costs.

Using the Model to Debug The Humans

The big idea here is that models can actually surface ground truth errors. Sounds crazy? So the way to think about this is essentially that if a model has many examples of truth, it can overcome small errors. Intuition here is sort of like how we can sometimes spot errors in a quiz answer sheet because even though it says it's the answer, we think about it and feel confident that it's not the answer.

In practice, one way to do this with images is to use Intersection over Union (IoU). If the model prediction doesn't have a nearby ground truth, then either the model is wrong, or the ground truth is. This can be surfaced by applications as a ranked list.

(Identifying the biggest delta between predicting and existing groundtruth)

Automated Checklist Example

While this may sound simple a checklist is an essential part of the QA workflow.

This can literally be a checklist for human review, or if possible can be programmed. Some software may already do some Tools may include some of these by default, but often the best checks are set up more as "test cases", where you define parameters relevant to your specific data (like a Checksum).

1. Is the count of samples reasonable?

2. Are the spatial coordinates reasonable?
3. Are all attributes selected or approved by human?
4. Are all instances approved by human?
5. Are there any duplicate instances?
6. (Video specific) Tracks where an instance is missing. Eg in frame n, missing in frame n+1, reappears in frame n+2, and not marked as end sequence
7. Dependency type rules, eg if class A is present, then class B should also be present, but class B is missing. This is different from heuristics in that it's usually more about reviewing existing instances vs generating them.

Checks based on looking at the data of samples

1. Reasonableness check. For example, if the class is a "person", and 95% of the pixels in the spatial location of the sample are all a similar color, that's unlikely to be correct. Or if the majority are in a different histogram distribution than what's expected (eg an object that is never red has red pixels) etc.

Data Discovery - What to Label Exploration

to help identify what to label.

1. Identifying outliers and filtering similar data without running our model

The main difference between choosing based on data is it's actually looking at the data itself. Eg this images visually is similar to this other one. Where as the metadata does not need to look at the actual image. Eg this was taken at night, and this was taken at day.

Choosing Based on Data

TK

Human Review

All methods involve some form of human review at some point. If you don't choose any method you are likely in essence doing human review by default.

Human review is often a good thing - even if it's not automated. Some automations make fairly careless assumptions about the data that may not be relevant to your case. For example if you only have so many medical scans of patients, maybe you do want to use 100% of your data.

While automation approaches for data discovery can be a very important part of the process, I think of it as a more advanced topic, and that the basics of understanding your data with human

review is the first step. Usually the best reason to know you need to use data discovery is that your raw data volume so clearly exceeds any reasonable ability to manually look at it.

Pre-Label Based

One commonly overlooked method is to use an existing network to pre-label at the whole file level. For example, imagine the intent is to label only day time images, but for some reason the metadata of the data is unavailable. You can run a network to tag images as “nighttime or daytime”, and then only label the daytime images.

Similarity Based

The long story short of that is you get similar images identified, eg each batch of these is similar so can sample from each.

And if you already know you are low on an image of a certain type you can also try querying based on that image.

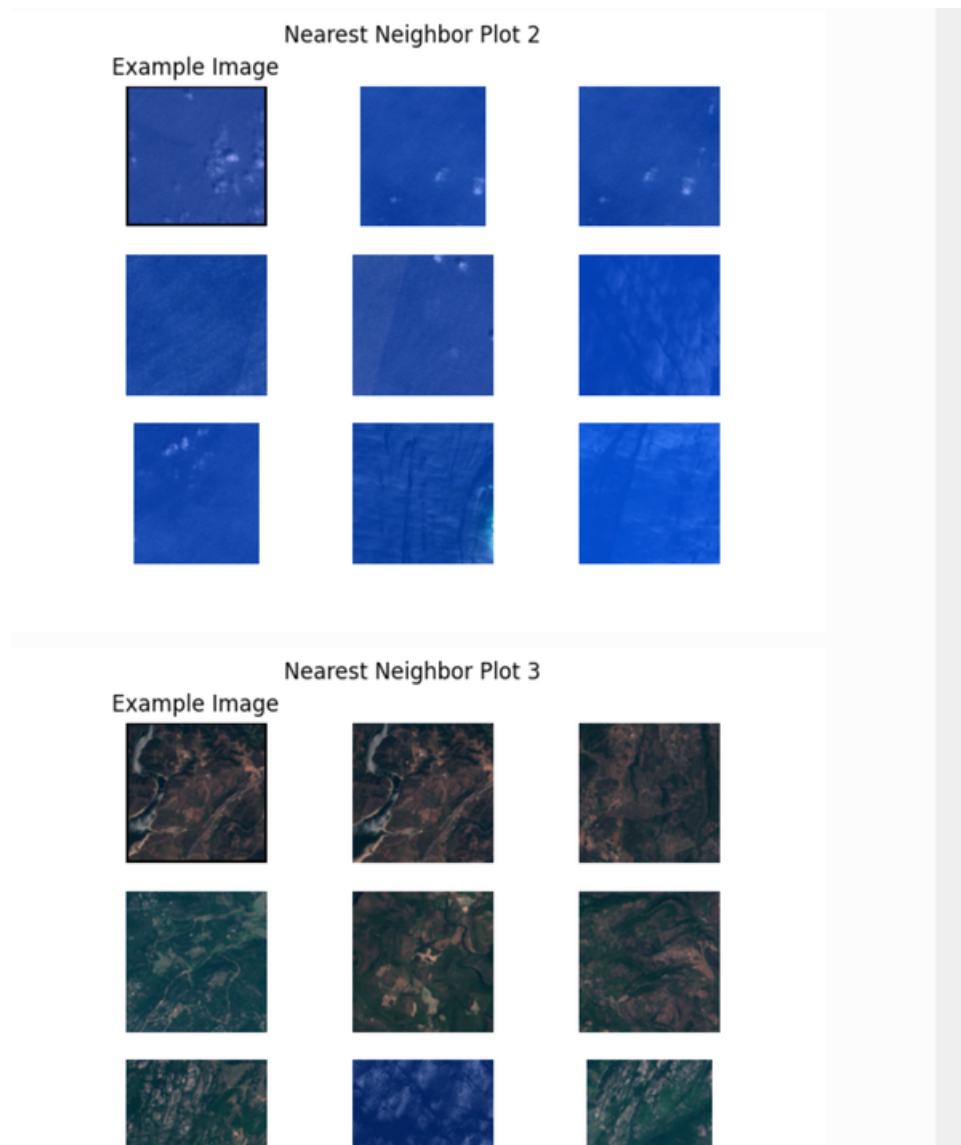


Figure 5-8. Example of similarity comparison.

Choosing Based on MetaData

So for example if you want to do something like this automotive company does you can query based on metadata

like time of day, microscope resolution, position, etc etc. etc.

Make sure to add the meta data during ingest, and then can query it on the explorer tab.

This is more user directed, but can still dramatically reduce the amount of data you need to label



Figure 5-9. Example of Query TK maybe replace Diffgram one to avoid license issue

Simulation & Synthetic Data

Synthetic data shows promise but is not a replacement for real data. In general - all synthetic data methods still require some form of real data to be useful. That's why they are best thought of as an automation method on top of real data.

Let's get the big question out of the way real quick - does simulated data work? The super short answer is "Yes, but probably not as well as you may be hoping".

The next question is "Will it eventually work really well?". This one is actually easier - it's unlikely. And there's a simple thought experiment to reflect on this.

To simulate the data, to the level of realism that is required, is in the level of hard problems as Artificial General Intelligence. It's relatively easy to get photo realistic looking renders - but there's a big difference between those simulated renders and real life.²

Typically simulated data, if used well, can provide a small lift in performance.

It can also be sometimes used for situations where it would be impossible, or very difficult, to get the data beforehand.

Another way to think about this is that using a simulation to automatically create training data is somewhat akin to using a heuristic. Instead of humans supervising and saying what right is, we are back to trying to build a bottom up type model. Just one level removed.

Simulations are not perfect - Training Data still needs human review

Shown in Fig 5.9 is an example from a major company presented publicly. They claimed it was "perfect" data. As you can see there is a patch over the cross work - part of what the touted as being more realistic - but then they forgot to account for that in the training data generation - generating perfect crosswalk lines over the imperfect pavement.

Were they trying to teach it that the black splotch was actually a cross walk? This doesn't make sense. Of course perhaps they were meaning to post-process it later to project a perfect cross walk. And perhaps some super specialized network could theoretically do this. But at least from the training data perspective this is just a plain error.

This is just a small example but it shows that assuming that a simulation will render perfect data is far from correct.

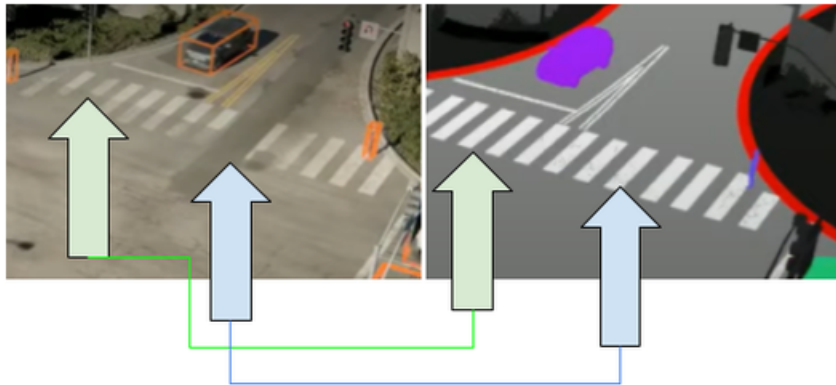


Figure 5-10. Example of training data that was claimed to be perfect but has clear labeling mistakes.

For the impossible and rare cases, I would really think of it less as “automatically creating training data” and more as “automatically creating scenes, for which humans can then create training data”.

For example, a system may be able to automatically identify which pixels are from which object. But - if the simulation is sufficiently random, that may not be relevant for events. In other words, it's important to think about what the simulation does actually know.

A way to think about this is if I have a simulation of shelves at a supermarket, those shelves will look similar, unless the simulation is engineering specifically to render different shelves. But what dimension will they be different in? Will those differences be relevant to the production data? Who is programming that in? Remember that even subtle shifts in the data can have huge effects on the result.

Clear Uses of Simulation:

1. Product images. If you already know what the product looks like, that should be a clear enough case for a shopping type robot.
2. Rare scenes of autonomous driving.

Pros

1. Simulations can improve performance
2. Simulations can create otherwise impossible or rare cases

Cons

1. The improvement is usually only by a small amount. Think on the order of a relative 0-10%. There may be case specific exceptions to this so it's worth some research.
2. Often the fidelity of simulations is a lot further from the real world than may first appear. For example what looks visually good in a video, may be drastically different pixel for pixel.
3. Simulations are often costly to setup, maintain, and operate.

Things to think critically about when considering simulations

1. What are we actually simulating? Are we simulating lighting conditions? Camera angles? Whole scenes?

Media Specific

Many automation methods work well for all media types. Here I cover some of the highlights of how popular methods intersect with specific media types. Then cover some common domain specific methods. This does not cover all media types. The main intent of this section is to:

1. Give awareness to the relationship between specific media and automation types
2. Provide an introduction to some of the domain specific types

What methods work with which media?

A ✓ indicates that usually the method will match. For example using pre-labeling with data discovery approaches for images is better defined than for video. I have included more extensive footnotes here to expand on rationale where required and keep the chart easy to read.

As you can see, generally most of these methods can stack well together. Of course each method takes a degree of work and understanding. There is no requirement to use all of them. You can have a very successful project that completely skips a lot of these.

*T
a
b
l
e*

*5
-
4
.*

*M
e
d
i
a*

*f
o
r
m
a
t
s
c
o
m
m
o
n
l
y*

*u
s
e
d*

*w
i
t
h*

each technique

Methods / Data Type	Video	Image	3D	Text	Audio
Pre-Labeling	✓	✓	✓	✓	✓
Interactive Automations	✓ ^a	✓	✓	Sometimes ^b	Research Area
Quality Assurance Automations	Research Area	✓	✓	✓	✓
Data Discovery	Sometimes ^c	✓	Research Area		
Augmentation	Sometimes ^d	✓	Research Area	Research Area	✓
Simulation & Synthetic	✓	✓	✓ ^e	Research Area	Research Area
Media Specific ^f	Varies - Check Method				

Domain Specific Special purpose automations, like Geometry, Multi-sensor approaches.	Few commonly used methods	Sometimes ⁹	Few commonly used methods
---	---------------------------	------------------------	---------------------------

- a May take more effort to set up.
- b There are a variety of “real time” model training concepts, but often this is more single user focused.
- c If the method requires conversion to images may prevent use of video specific automations. If possible to think in terms of discovering segments, and keep video form, may improve compatibility.
- d Many augmentation methods are focused on images, so may not be applicable for event detection or other types of motion prediction tasks.
- e Because most simulations are 3D by default.
- f Like Object Tracking (Video), Dictionary (Text), AutoBorder (Image). Because the methods are unique to each media type, double check when choosing a specific method.
- g Many post-processing methods but usually not in context of training data

Considerations

Just because you are using a certain data type, eg Video, doesn’t mean that an automation will be available for that case. For example maybe you have no problem identifying people in videos, but identifying specific interactions may be harder.

Video Specific

If you are interested in *event detection* type actions, some of these automations are not recommended. For example object tracking, would confuse a situation where you want to know exactly which frame an event occurs.

Object Tracking

The big idea here is to look at the data itself, and track an object throughout multiple frames. There are a variety of tracking concepts, many need only a single frame to provide an effective track forward through time.

Interpolation

The idea here is humans create keyframes and the in between data is filled in.

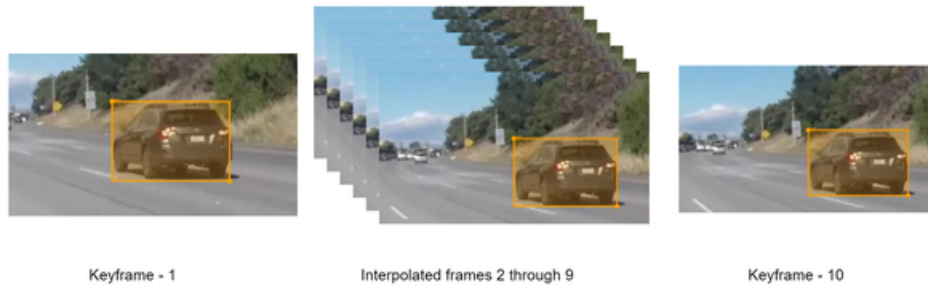


Figure 5-11. Example of Interpolation

Polygon and Segmentation Specific

TK

AutoBordering

I covered how to use AutoBordering in Chapter 3. Here I briefly want to state that using techniques like that to snap to edges, as simple as it may seem, is a great use of practical UI automations.

Language (NLP) Specific

TK:

Heuristics (covered earlier in this chapter)

Dictionaries

Augmentation

Augmentation is modification of real data. For example skewing existing images, changing the brightness, introducing artifacts, etc.

In general there seems to be a consensus towards doing augmentation at runtime - and more minimally than the exuberance of early approaches. To put it simply augmentation is often a crutch, or provides a relatively small lift. It can still be useful and it's good to be aware of when and how it can be used.

Like simulated data, augmentations generally can provide some form of lift on the data. But think in terms of 0-10%.

Better Models are Better than Better Augmentation

One general theme is that as the models and algorithms get better, augmentations become less effective. This may not always be true but is a good rule of thumb.

A simple way to think of this is that there are nearly infinite subtle combinations of data values, be it text, pixels, etc. So trying to train on ever growing size of subtly different data is a losing proposition to just having a better understanding of what it is.

Think about it from the human perspective, I don't need to see 100 different variations of essentially the same thing in order to recognize it.

To Augment or Not To Augment

The first thing to appreciate with any form of Augmentation is that it significantly alters the playing field.

Next, the best evaluation of any augmentation method is what "net lift" it provides. This is sometimes trickier to measure than is first apparent.

For example, imagine a system that "pre labels" instances. The Spatial location is pre labelled reliably. Expert users then add additional information, such as what type of crop disease it is. The "net lift" in this case, is primarily time savings. So if the cost (literal cost, complexity, etc) to setup the pipeline, run the model, etc. is low enough then, every spatial location that's pre-labeled is a direct time savings. In practice, if a robust model is available for this, or is already being run in production etc, then this is valuable.

Alternatively consider some negative scenarios.

An approach (eg superpixel similar) is used to more "quickly" label spatial locations. Except, it introduces artifacts and hard to correct errors. It's easy for it to look "ok" at a glance, so these errors go unnoticed until significant time is spent on the dataset. This may cause rework and/or hard to understand production errors.

A pre-label approach that requires constant correction. Correction is ~3x slower than just drawing the spatial location correctly the first time. So, in the abstract, the pre-label must be very accurate to break even, and more so to provide net lift.

Any scenario in which an "external" model is used. It introduces bias into the new training data - polluting it so to speak. As a practical example, imagine a person detector that appears to reliably detect people. It's used as the "spatial location" first pass, to which your customer Labels are then added. If that detector has a bias, perhaps favoring an ethnic majority, it may be very difficult to spot, since it's "hidden". You may need an extra QA step expressly to check for this potential.

At a high level, the general rules of thumb for these augment approaches are:

Explore with caution - there are always trade offs and augment methods hide the trade offs well

Consider the "net lift", taking into account costs such as the added complexity, reduced flexibility, literal compute/storage costs etc

Many of the approaches are very specific - it may work very well for one case, of one spatial type, etc. while failing on many others

Other general observations

The most effective options generally seem to favour spatial types over class labels

Generally it's best to create a "Seed" set, before exploring any type of 'model' based approaches there

Sometimes "Augmented" training data is somewhat hard to avoid, eg in the case of 'correcting' a production pipeline

Runtime Augmentation

It's worth keeping in mind that in general, any type of augmentation must be reproducible. In that context, it's usually more compute and storage effective to have the literal images created at training time, than stored as part of normal training data.

Patch and Inject Method (Crop and Inject)

The concept here is to take patches - meaning crops or subsections - of the data and creating novel combinations. For images, you can image placing rare classes in a scene. This is sort of like a hybrid simulation where real data is simulated in different places.

There is little research on this. In general - a model should be robust to these types of scenarios.

Domain Specific

TK

Geometry Based Labeling

In some cases, you can use known geometry of a scene, sensors, etc. to use geometry based transformations to automatically create some labels. This case is highly dependent on the specific context of your data.

Multi-Sensor Labeling Automation - Spatial

The main idea here is to use mathematical projections to assume where something is in space based off where the sensors physically were. For example, if you have 6 cameras, it can be possible to construct a virtual 3D scene. Then you can label 1 camera, and project that into the other 5. Often this means projecting into 3D and back into 2D.

Note this does not require a 3D oriented sensor like LiDAR or RADAR. But in general it's considered easier if one of those sensors is present.

Pros

1. In the most optimal case, you get something like 5:1. Keep in mind you still need to review and often correct the projections, so in reality it's more like 3:1 at it's best.

Cons

1. It requires having multiple sensors, extra metadata, the ability to project into 3D

Spatial

This is similar to the multi-sensor labeling, but is more focused on cases where large extents of geometry focused concepts. Like lane lines.

Heuristic Based Labeling

In general heuristic labeling has been NLP focused.

Dictionary based labeling

User defined heuristics

This is a controversial area. The main problem is that the more well defined a heuristic is, the more it looks like we are coding and doing work there, and basically re-inventing feature engineering. Since one of the whole points of deep learning based approaches is automatic feature engineering, this essentially is self defeating.

Now this is not to say heuristics based methods are without merit - far from it. There are some incredibly interesting research in this area, especially for text based applications.

If you have written a perfect set of heuristics, then why even train a machine learning model at all?

¹ For experienced folks, grading production predictions is also rolled into this concept

² Anecdotally, I have run this by both research and industry oriented people and there's usually common agreement on it's truth.